软件定义的信号测试与验证平台

摘要

本项目面向便携式测试及通信等领域验证需求，基于 ZYNQ-7020 SoC 构建统一软件定义的多域信号测试与验证平台。采用软件定义的架构，单平台即可集成任意波形发生器、示波器、高精度宽频带实时频谱仪、频率特性分析仪与数字滤波器等核心功能。PL 端负责高速 ADC 采集与并行信号处理：构建 FFT、FIR、IIR 实时算法流水线；通过 FFT 硬件加速卷积预算实现 Chirp-Z 变换，在关注频段内实现任意频率分辨率细化；基于 Vitis HLS 设计矩阵乘法与线性方程组求解硬件加速器，加速基于 Levy 最小二乘的传递函数拟合；构建由 AXI 控制、双端口 BRAM 存储波形表的 DDS IP 核，波形数据支持 PS 端动态改写，实现用户自定义任意波形发生；通过 AXI VDMA 驱动 LCD 显示，并利用 AXI DMA 实现 PS-PL 高速数据交互。PS 端负责数据流调度、人机交互控制及部分数据分析，通过 AXI4-Lite 配置 ADC 采集、FFT 等 PL 模块的工作状态与参数。信号发生器支持任意函数波形的发生。频谱仪与示波器覆盖 DC 至奈奎斯特频率，实现精细时域和频域分析。频率特性分析仪可测量未知系统的幅频、相频与冲激响应，并拟合传递函数。

目录

摘要 **1**
第一部分　作品概述 **1**
1.1 功能与特性 1
1.2 应用领域 2
1.3 主要技术特点 3
1.4 主要性能指标 3
1.5 主要创新点 5
1.6 设计流程 6
第二部分　系统组成、功能说明及接口设计 **7**
2.1 整体介绍 7
2.2 硬件系统介绍(FPGA/PL) 8
2.2.1 硬件整体及接口设计介绍； 8
2.2.2 ADC 采集模块 ip 核集群介绍 11
2.2.3 DDS/DAC 输出 ip 核集群介绍 12
2.2.4 硬件算法加速器集群介绍 15
2.2.5 滤波器功能集群介绍 37
2.2.6 LCD 显示器功能集群介绍 41
2.3 软件系统介绍（ARM/PS） 42
2.3.1 软件整体介绍 42
2.3.2 Chirp-Z 变换 42
2.3.3 示波器 Sinc 插值 48
2.3.4 频率特性测试仪 50
2.3.5 Zoom-FFT 软件部分 52
2.3.6 人机交互 54
第三部分 性能与资源优化 **55**
3.1 性能与资源优化概述 55
3.2 DDS 输出时序优化 55
3.3 BRAM 资源优化 58
3.4 AXI 传输速度优化 59
3.5 Chirp-Z 卷积运算精度优化 61
3.5 时钟输出精度优化 63
第四部分 大模型协作记录 **67**
4.1 协作模型与工具声明 67
4.2 核心交互案例与自我纠错轨迹 67
4.3 协作效果评估与反思 70
4.4 团队成员原创部分说明 70
第五部分 技能包提炼过程 **72**
第六部分 完成情况及性能参数 **75**
6.1 整体介绍 75
6.2 工程成果 75
6.2.1 PL 侧 FPGA 硬件设计成果 75
6.2.1 软件界面设计成果 77
6.3 特性成果（功能、性能参数等量化指标） 78

6.3.1 信号发生器 78
6.3.2 示波器 82
6.3.3 实时频谱分析仪 85
6.3.4 频率特性测试仪 90
6.3.5 滤波器 94
第七部分 总结 97
7.1 可扩展性 97
7.2 心得体会 97
第八部分 复现说明 99
附录 参考文献 102

第一部分　作品概述

1.1 功能与特性

单平台集成任意波形发生器、示波器、高精度宽频带实时频谱仪、频率特性分析仪与数字滤波器。同时拥有统一软件定义架构，可以基于本架构，以极低的硬件成本，构建任意其他功能和算法。

信号发生器支持基础波形（正弦波、方波、三角波、锯齿波）、多次谐波、高级波形（sinc 等）、通信调制波形（AM、FM、PM、2ASK、2PSK、2FSK 等），幅度、相位、频率等通用参数和各个波形特殊参数的均可配置；示波器可以对信号的时域参数进行测量，并拥有 Sinc 插值、自动设置、水平系统和垂直系统调整的功能。频谱仪可实现 FFT 频域实时分析、Chirp-Z 变换精细频谱分析、Zoom-FFT 分析，同时拥有良好的交互功能，支持点数、采样率等测量相关参数调整。支持坐标范围调整、采样率调整、峰值寻找等交互功能。支持选择对信号加窗的窗函数类型；频率特性测试仪，可测未知系统幅频响应、相频响应、冲激响应、阶跃响应在装置绘图，并显示指定阶数的传递函数拟合函数；数字滤波器，支持直接将本装置作为一个 FIR/IIR 滤波器使用，接入别的系统或者本系统。



图 1-1　功能示意图

1.2 应用领域

面向便携式现场测试、通信链路验证、雷达信号验证、高校实验教学、工业电子维护与科研。可用于波形激励、时/频域观测、频谱监测、系统辨识与数字滤波器验证；适用于外场设备调试、射频模块测试、通信调制质量评估、雷达回波模拟与接收链路验证、自动测试系统。

典型场景：高校实验教学与学生工程实践训练。作为本科学生极难获得昂贵的且数量大单体仪器。如下图所示，本人在开发时仪器多且昂贵，且仅在特定的空闲时间能获取使用权。且不少学校缺乏良好的实验室条件，会受到物理空间的限制无法摆放。在本场景该装置有充分的应用空间。

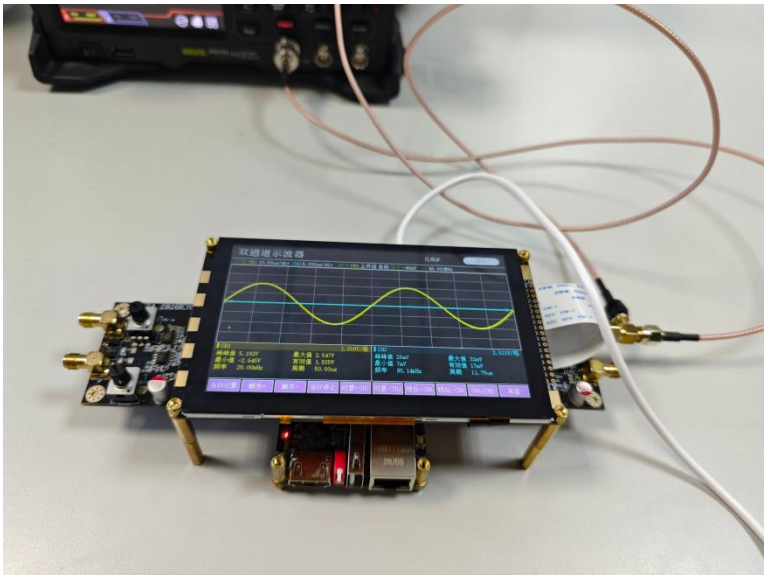


图 1-2 装置图片

1.3 主要技术特点

平台采用统一软件定义架构，PS 与 PL 通过 AXI4-Lite、AXI DMA、AXI GPIO 和 AXI VDMA 协同。PL 端实现高速 ADC 采集与并行信号处理流水线，包括 FFT、FIR、IIR 实时算法；利用 FFT 硬件加速卷积运算实现 Chirp-Z 变换，在关注频段内实现任意频率分辨率细化，受采样率与数值精度约束。基于 Vitis HLS 设计矩阵乘法与线性方程组求解硬件加速器，加速 Levy 最小二乘传递函数拟合。采用双端口 BRAM 驱动 DDS IP，波形表可由 PS 动态改写，支持任意函数信号发生。通过 AXI VDMA 驱动 LCD 显示，AXI DMA 实现 PS-PL 高速数据交互。PS 端负责数据流调度、人机交互、参数配置与部分数据分析，可供调节参数丰富。PS 端通过 AXI4-Lite 控制 ADC、FFT、FIR、IIR、DDS、PL 时钟（控制采样频率）等 PL 模块的工作状态与参数。

1.4 主要性能指标

本装置秉承低成本创作原则，均选用性价较高的器件。在器件参

数范围内尽力发挥其理论的最佳性能。选用 ADC 芯片为 AD9238，最大采样率 65Msps。选用 DAC 芯片为 AD9767，最大速率为 125Msps。

信号发生器：

DAC 采样率	120Msps
DAC 位数	14bit
最高频率	正弦波 10MHz，更高可保证不失真，幅度不保证
频率分辨率	0.011Hz
相位分辨率	0.0015rad
输出电压范围	-5v-5v,Vpp 最大 10V，可调
通道数	2

示波器：

ADC 采样率	最大 64Msps，最小 5Msps,可变
ADC 位数	12bit
最高可测量频率	25MHz
存储深度	最大 1M
输入电压范围	-5v-5v
通道数	2

高精度宽频带实时频谱仪：

说明：ADC 参数、采样率、最高可测量频率、带宽参见示波器

FFT 点数	16384 点，可变
频率分辨率	与采样率和 FFT 点数有关
幅度误差	1%
窗函数类型	矩形、汉宁、汉明、布莱克曼
Chirp-Z 变换最大采样点数	8192
Chirp-Z 变换分辨率	采样点数为 8192 时，最小为指定频率范围除于 8192
Chirp-Z 变换幅度误差	1MHz 以内 0.5%

Chirp-Z 变换频率误差	1MHz 以内 5Hz 以内

频率特性测试仪：

扫频分辨率	最小 500Hz，可调
扫频时间	分辨率 500Hz，500Hz-500kHz，15s
扫频范围	分辨率 500Hz 时，8MHz，随着分辨率增加而提高
幅度误差	0.3dB 以内
相位误差	3 度以内
传递函数拟合阶数	2
显示曲线类型	幅频曲线、相频曲线、冲激响应、阶跃响应

数字滤波器：

FIR 最大阶数	9
IIR 最大阶数	7
采样频率范围	最大 64Msps，最小 5Msps

1.5 主要创新点

1.软件定义统一架构;单平台集成五类仪器，集成度高；使用多种 AXI4 总线可灵活传输参数、数据和工作状态；性能和精度不弱于单一功能台式仪器。

2.采用先进算法，如 Chirp-Z 变换、levy 最小二乘法、Sinc 插值算法，显著提高测量精度。

3.使用 FPGA 硬件加速核心算法，FFT、卷积、矩阵乘法和 FIR。实现形式包括 Vitis HLS、RTL 代码、ip 核；

4.使用双端口 BRAM 和 DDS 实现任意函数波形输出；

5.优秀的人机交互功能：使用的 AXI VDMA 驱动的 LCD 屏幕，操作

丝滑，全触控,功能丰富。

1.6 设计流程

1. **需求分析,指标定义**;分析想要实现什么，需要达到什么指标

2. **总体架构和算法设计：**划分系统子功能模块并设计通信接口

3. **MATLAB 算法仿真：**仿真算法是否理论成立

4. **PL 开发：**包括自定义 IP、官方 IP、Vitis HLS ip、RTL、BD

5. **PS 驱动与交互开发：**C 语言嵌入式开发

6. **指标及算法测试：**测试该装置性能，用 MATLAB 导入实际采样数据验证算法的正确性。

7. **优化迭代：**不足的未达到指标要求和与理论差距过大的部分继续改进

第二部分　系统组成、功能说明及接口设计

2.1 整体介绍

系统架构采用 AMD ZYNQ 7020 异构 SoC 作为核心处理平台。PL 端通过可编程逻辑实现高速信号处理——包括 ADC 控制、DAC 输出、DDS 直接数字频率合成、16384 点 FFT、FIR/IIR 数字滤波、频谱分析、频率特性扫描。PS 端采用双核 ARM Cortex-A9 运行，负责任务调度、触控 UI 交互、数据处理与存储。PL 端与 PS 端相互配合，PL 端采用图形化、模块化的编写形式。

本系统依据在 PL 端划分为 5 个功能集群，如下图所示,分别为：**ADC 采集模块 ip 核集群、DDS/DAC 输出 ip 核集群、硬件算法加速器 ip 核集群、滤波器功能 ip 核集群、LCD 显示功能 ip 集群**。各个功能集群均拥有与 PS 端通信的接口，**且各个集群之间的数据通信必须经过 PS 端**（除滤波器功能 ip 核集群）。这么做有以下依据：

1. 各个集群之间功能相对独立
2. 各个集群内部功能实时性要求高
3. 集群之间通信数据流向复杂，且涉及跨时钟域；用 PS 端易于控制数据流，处理跨时钟域方便，降低设计难度。
4. 便于 PS 端对各个模块参数和功能的配置

最终以 PS 端为中心，**通过 AXI 总线向四周连接模块化的 PL 端的 IP 核对应的功能，形成“PL 包围 PS”的策略。极大提高各个模块的可复用性，每一个功能都能通过 PS 端配置各个模块的工作状态，**

并定制其大量数据流的流向满足其使用的需求，实现软件定义的测试架构。

功能框图

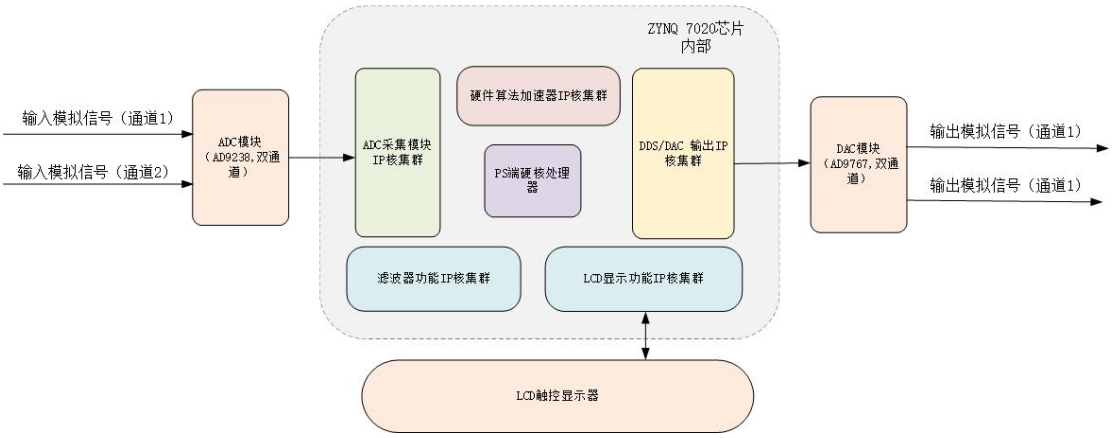


图 2-1　功能框图

2.2 硬件系统介绍(FPGA/PL)

2.2.1 硬件整体及接口设计介绍；

对于系统 FPGA 硬件架构，我们应该关注三个重点：数据流软硬件通信接口设计、控制流软硬件通信软硬件通信接口设计和多时钟域设计。

数据流软硬件通信接口总体设计如下图所示。其设计思路是：对于不间断大量数据传输，均使用 AXI4-Stream 通信协议，通过 AXI DMA ip 核，将其转换为 AXI4 协议与 PS 端进行通信。其包括：ADC 采集模块 ip 核集群、硬件算法加速器 ip 核集群；对于显示器驱动，我们采用针对视频图片数据优化的 AXI VDMA ip 核；对于 DDS，其传输的数据为波形数据且传输频率较低,且数据传入的目的

地为 AXI BRAM，所以我们使用 AXI BRAM 控制器完成数据传输；对于滤波器功能 ip 核集群，因为对实时性要求较高，数据流不通过 PS 端，故不涉及软硬件交互接口。

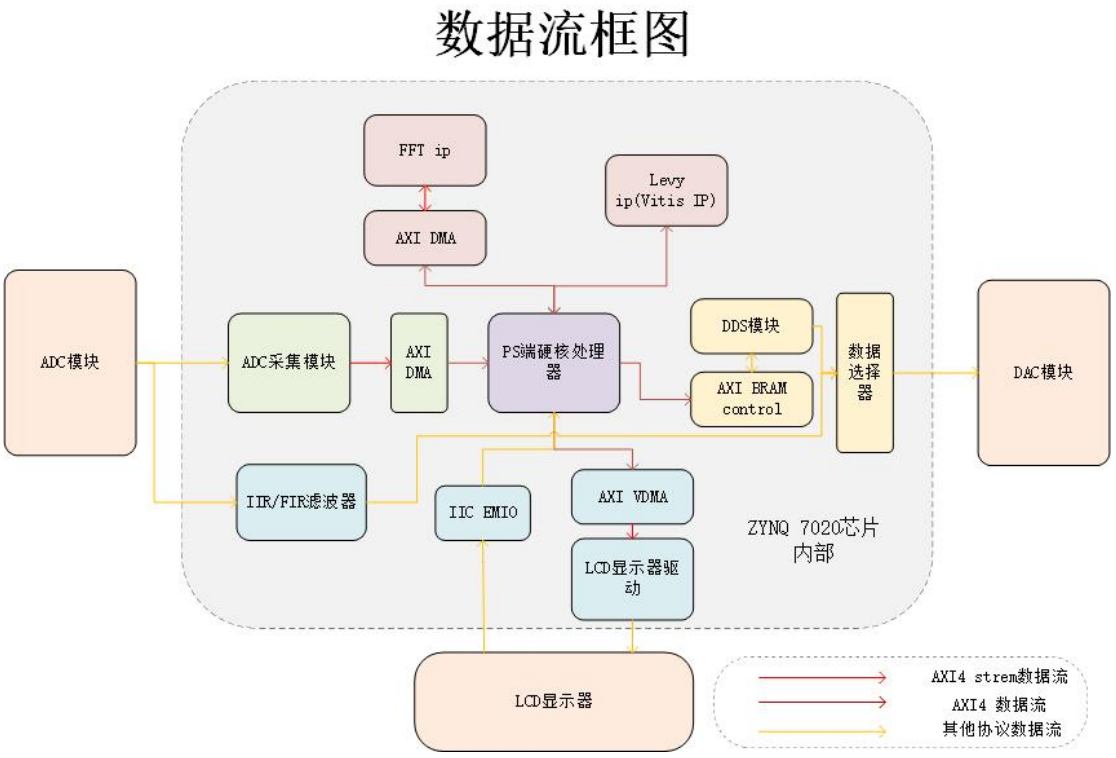


图 2-2 数据流框图

控制流软硬件通信接口总体设计如下图所示。其设计思路是：对于参数传递，使用 AXI4 Lite 控制接口。其包括：FFT 的点数、ADC 单次采样点数、ADC 采样时钟频率、IIR/FIR 系数、DDS 幅度频率相位数据等等；对于工作状态传递，往往单个模块只需要 1-2bit 足够，故使用 AXI GPIO 传递信号。其包括：ADC 采样的启停和数据选择器的状态。

控制流框图

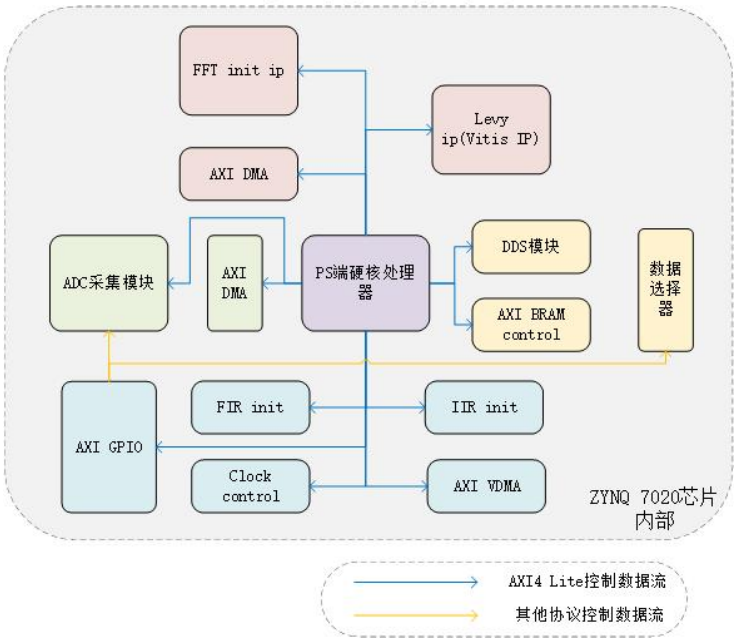


图 2-3 控制流框图

多时钟域设计如下图所示。我们使各个模块工作在适合自己的时钟频率下。其中 ADC 采集模块 ip 核集群核滤波器功能集群的时钟频率可以依据需求改变,该模块时钟由 clk_wiz ip 核提供。需注意：在数据跨时钟域进行传输时，应有相应的跨时钟域的缓冲功能。

时钟域

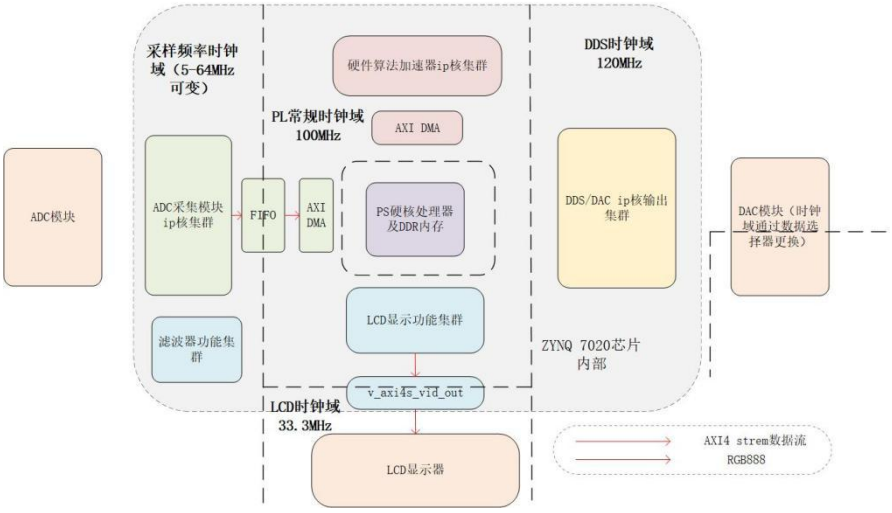


图 2-4 时钟域

2.2.2 ADC 采集模块 ip 核集群介绍

ADC 采集模块 ip 核集群统实现思路是在 PL 端建立采样时钟域。该采样时钟可以通过配置 clk_wiz ip 核改变。因为 AXI DMA 等其它模块工作在 100Mhz，所以 AXI DMA 与 ADC 模块之间通过一个异步 AXI Stream Data FIFO 用于缓冲，用于解决跨时钟域数据传输的问题。故整体的数据流向为：通过 PL 端 ADC 将采集到的数据，AXI Stream Data FIFO 传入 AXI DMA，再传到 PS 端。

同时，位于前端的 ADC 模块单次的采样点数可以由 PS 端通过 AXI4-Lite 改变。为了防止 DMA 传输阻塞，PS 端通过 AXI GPIO 模块，控制前端 ADC 模块 AXI4 Stream 协议的传输的启停，做到仅在检测到 AXI-GPIO 下降沿的时候，ADC 侧 AXI4S 的 tvalid 才被允许拉高，激发数据传输。具体的实现如下图所示。

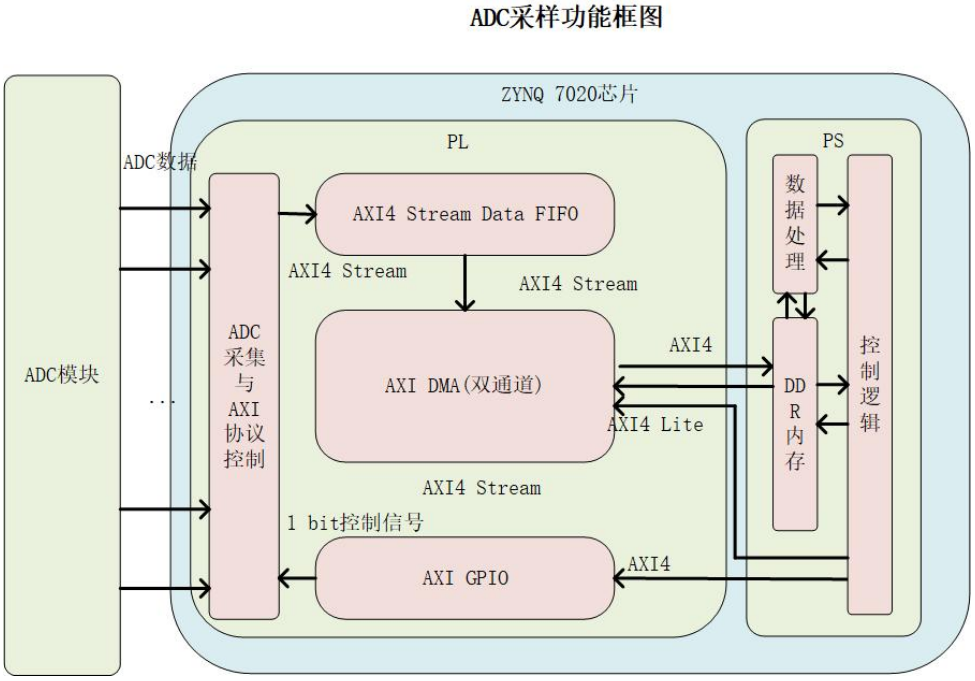


图 2-5　ADC 部分功能示意图

此外 ADC 采集模块可以通过 AXI4-Lite，配置 ADC 单次采样的点数，格式如下图表所示：

寄存器	功能	数据范围
slv_reg0	采样点数	0-2^20，整数

2.2.3 DDS/DAC 输出 ip 核集群介绍

DDS 的技术原理围绕相位数字化调控展开，核心由相位累加器、波形查找表（ROM）、数/模转换器（DAC）和低通滤波器（LPF）四大单元组成，整体工作流程为四步： 第一步将相位累加器在系统时钟驱动下，根据输入的频率控制字（相位步长）进行周期性相位累加，输出的相位值作为波形查找表的寻址地址； 第二步在波形查找表中预先存储了目标模拟信号（如正弦波）的相位-幅度数字量化值，根据相位地址输出对应的幅度数字量； 第三步数/模转换器将离散的幅度数字量转换为带有阶梯纹波的模拟电压信号； 第四步模拟低通滤波器滤除模拟信号中的高频杂散和阶梯纹波，最终得到平滑、连续的目标周期模拟信号。

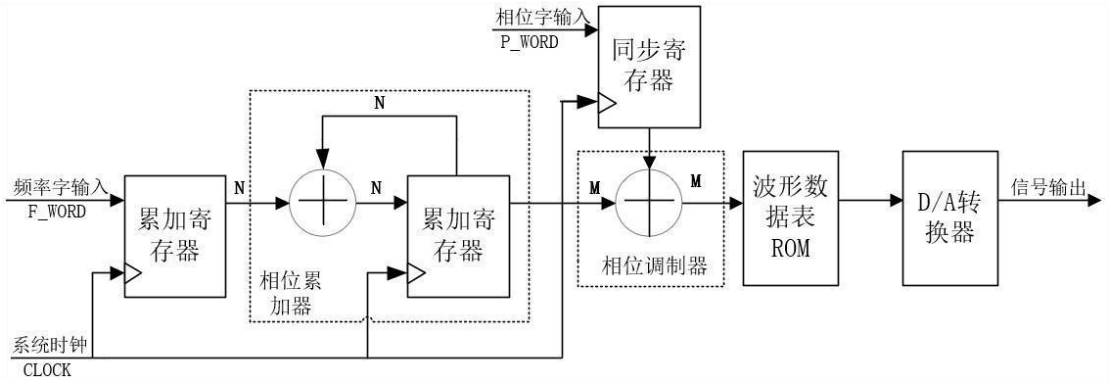


图 2-6 DDS 原理图

对于累加寄存器，频率关键字 F_WORD 由 PS 端输入，系统时钟 CLK

设定为 120MHz,累加寄存器的位数取 32 位。

该系统的输出频率由频率关键字 F_WORD 决定，其 Fout 由下面式子决定，SYS_CLK 代表系统时钟，N 为累加寄存器的位数:

$$F_{out} = \frac{F_WORD * SYS_CLK}{2^N}$$

即 F_WORD 表达式为:

$$F_WORD = \frac{2^N * F_{out}}{SYS_CLK}$$

对于相位调制器，其位数 M 取 12 位，由于 M 的位数与 N 的位数不相同，实际操作中要取 N 的高 12 位。相位控制关键字 P_WORD 也由 PS 端输入。输出信号的初相位与相位控制关键字 P_WORD 的关系如下：

$$P_WORD = \frac{\varphi}{2\pi} * 2^M$$

其中φ代表初相位，M 代表相位调制器的位数。

信号发生模块集成两个 DDS ip 核。其中一个 IP 核波形数据由 matlab 产生 coe 文件并固化在 ROM ip 中，其中固化有 4 种波形：正弦波、方波、三角波、锯齿波。根据我们选用 DAC 芯片 AD9767 为 14 位，我们通过 MATLAB 生成幅度在 0-16383 之间，每周期采样点在 2 的 M 次方的正弦波、方波、三角波和锯齿波波形数据，即 4096 位。写入 COE 文件，并使用 Xilinx 提供的 ROM ip 核，预先加载 coe 文件。

其波形类型、频率关键字、幅度、相位等关键参数通过 AXI4-Lite

受到 PS 端控制。AXI4-Lite 协议的格式如下表所示：

寄存器	功能	数据范围
slv_reg0	波形类型	0001：正弦波 0010：方波 0100：三角波 1000：锯齿波
slv_reg1	频率关键字 F_WORD	小于2^{32}
slv_reg3	幅度参数	0-1024 线性对应实际输出幅度
slv_reg4	相位关键字 P_WORD	0-4096，线性对应 0-2π的相位

另一个 DDS ip 核波形数据存储在一个双端口 BRAM 的 ip 核中，其波形数据可以由 PS 端产生通过 axi_bram_ctrl ip 核改写 bram 的波型数据，自编的 DDS IP 核可以读取 bram 的波形数据输出。和前面那个普通 DDS ip 核一样,同样为 4096 个点，每个点的范围 0-16384,其频率、幅度、相位由 PS 端通过 AXI4-Lite 控制，格式可以参考普通 DDS 的表。实现了信号发生器可以发生任意波形。

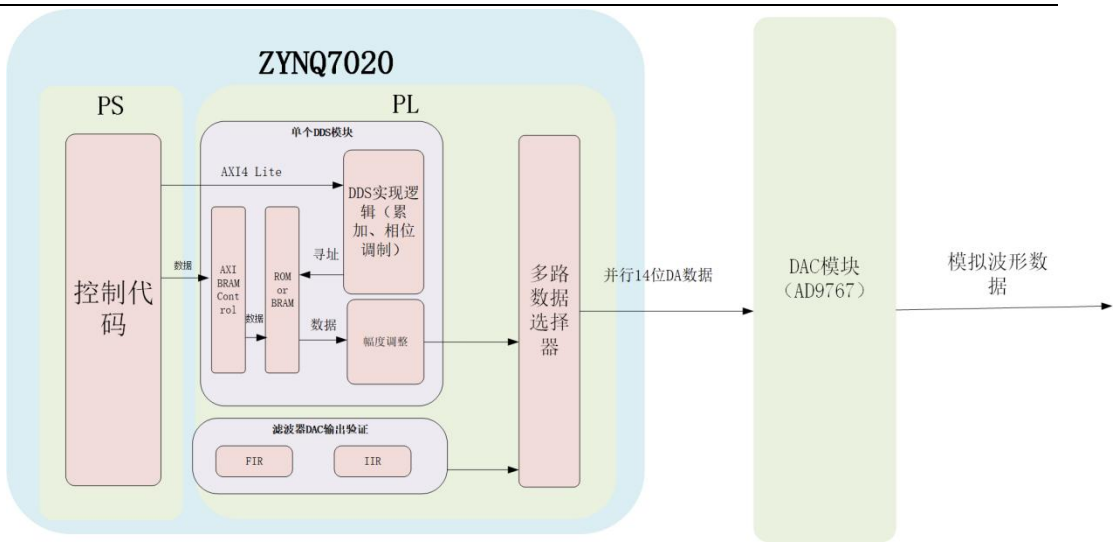


图 2-7　DDS 功能示意图

同时，在 DAC 通道一，两个 DDS 和数字滤波器的验证输出使用一个同一 DAC 个通道。为此，我们在通道一设计数据选择器，该数据选择器由 PS 端通过 AXI GPIO 控制。控制方式如图 2-8 所示：

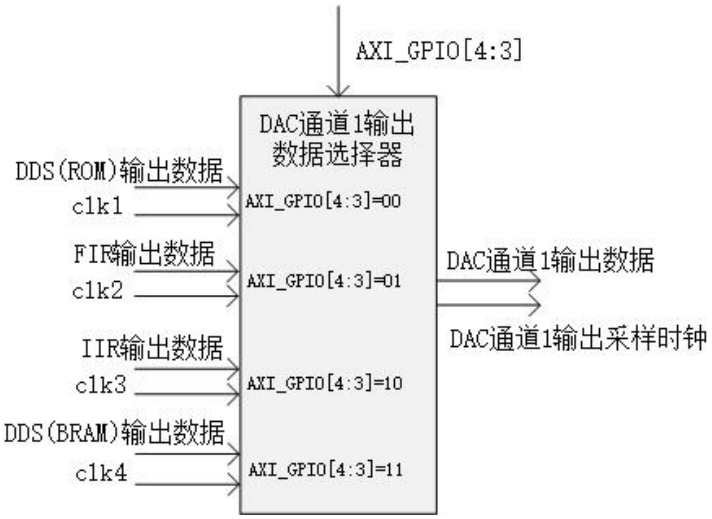


图 2-8　控制方式示意图

2.2.4 硬件算法加速器集群介绍

2.2.4.1 FFT 加速模块

我们知道，对于一个信号，想要得知其频率成分，我们需要对其做傅里叶变换。对于当代的数字系统，更经常用到的是离散傅里叶变

换（DFT），其公式如下：

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}kn}$$

其中 k=0，1，...，N-1。

如果直接按照公式计算，对于每一个 X[k]需要 N 次复数乘法和 N-1 次复数加法。总共需要计算 N 个 X[k]，因此总的计算复杂度为 $O(N^2)$。

当 N 很大时（例如 N=1024），计算量是百万级别的；如果 N=106，计算量是万亿级别，这在实时系统中是无法接受的。

于是，出现了快速傅里叶变换算法。快速傅里叶变换（FFT）的核心思想是分治策略。它利用了旋转因子的周期性和对称性，将一个大尺寸的 DFT 分解成多个小尺寸的 DFT 的组合。它通过巧妙的分治和递归结构，利用旋转因子的对称性，将计算复杂度从 $O(N^2)$ 降低到 $O(N\log_2 N)$。

对于 FFT，我们需要关注一个核心参数——频率分频率。频率分频率决定的我们能在频谱图上分辨两个信号频率的最小差值。频率分频率如果太低，会造成系统无法达到要求；太高，又会过度消耗系统资源。频率分频率Δf 公式如下：

$$\Delta f = \frac{f_s}{N}$$

其中f_s为采样频率，N为 FFT 运算的采样点数。f_s越低，频率分辨率越小。N越大，频率分辨率越小。同时，采样定理指出：为了能够

从采样后的离散信号中完全恢复出原始的连续信号，采样频率f_s必须至少是信号中最高频率分量f_{max}的两倍。故我们在工程中要充分权衡利弊和限制条件，做出最合适的选择。

FFT 通过 AMD 官方提供的 FFT ip 核实现。相比于在处理器顺序实现，FFT 算法良好的并行特性使得它在可编程逻辑电路实现不仅拥有更低的能耗，而且还拥有更快的速。为了确保精度最大化，输入数据 32 位（16 位实部+16 位虚部），FFT 结构自动选择，**FFT 输出结果不缩放，**输出数据 64 位。相位因子 W 的位数选择 32 位。配置界面如图 2-9 和图 2-10 所示：

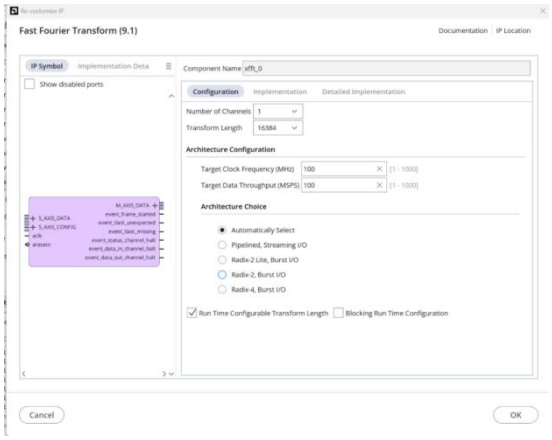


图 2-9　FFT ip 核配置界面

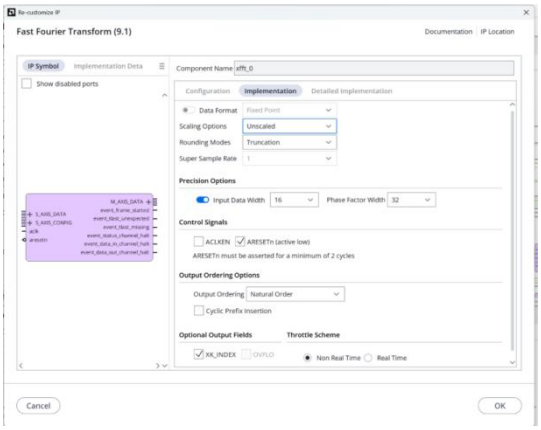


图 2-10　FFT ip 核配置界面

其输入输出数据的 AXIS_DATA 接口的 tdata 格式如下表所示：

输入数据 tdata 格式	bit15-bit0	实部
	bit31-bit16	虚部
输出数据 tdata 格式	bit31-bit0	实部
	bit63-bit32	虚部

FFT 的工作状态，如变换点数（16384 以下可调），IFFT/FFT 等，为了可以在工作中随时改变，为此编写了 FFT_init ip 核，该 ip 核可以在 PS 通过 AXI4 Lite 向其传输点数数据时,将其转换成 FFT ip 核 AXI4-Stream 接口的配置的正确时序向其配置。

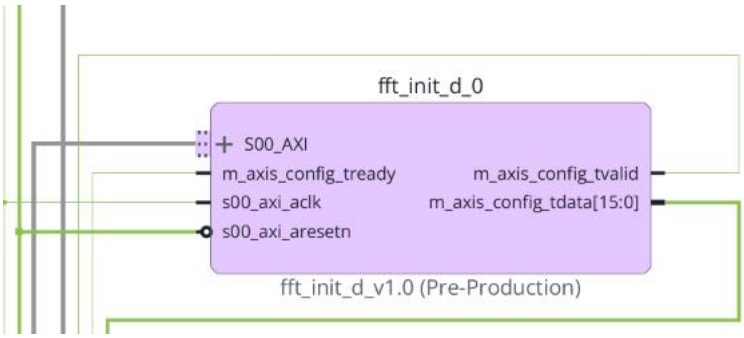


图 2-11 自编 FFT_init ip 核接口示意

综上所述，**该 ip 核可以灵活适配不同的场景和功能**，在本项目中，**频谱分析、Chirp-Z 卷积运算、频率特性幅频/相频响应分析，冲激响应逆 FFT 运算均使用该 ip 核加速运算，**该 ip 得到了广泛的应用。具体应用可以参见软件设计部分。

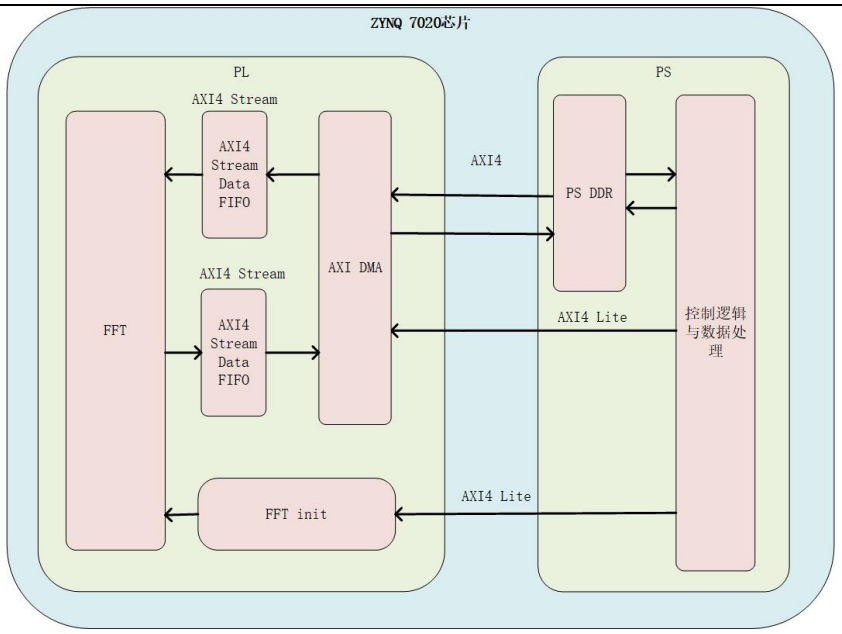


图 2-12　FFT 周围 ip 核示意图

FFT_init 的编写逻辑是基于状态机。PS 端在每次有改变 FFT 工作状态需求前，会传输一个符合 FFT ip 核 m_axis_config_tdata 格式的数据放在 m_axis_config_tdata_mid 的变量里。状态机有三个状态，状态 S0 为等待状态，m_axis_config_tvalid 保持拉低，不对 ip 核进行配置。当检测到 m_axis_config_tdata 不等于 m_axis_config_tdata_mid 时，立即将 m_axis_config_tvalid 拉高并将 m_axis_config_tdata_mid 传入 m_axis_config_tdata，状态由 S0 转移到 S1。状态 S1 为发送数据状态，若 ip 核的 m_axis_config_tready 没有拉高，则一直保持当前状态。由于 m_axis_config_tdata 只有一帧数据，若检测到 m_axis_config_tready 拉高,下个时钟周期立即将 m_axis_config_tvalid 拉低，状态由 S1 转移到 S2。状态 S2 为保护周期，在此周期内不能对 FFT ip 核做任何配置。当记满四个时钟周

期，状态由 S2 转移到 S0。

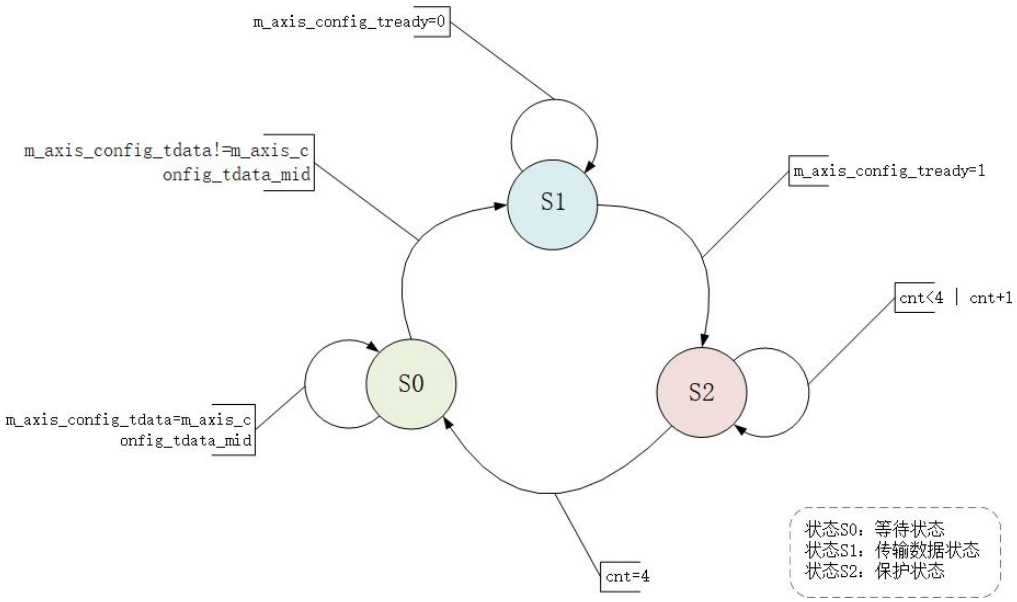


图 2-13　FFT init ip 核状态转移图

对于我们所设置的，FFT ip 的核的 m_axis_config_tdata 仅需要一帧数据，格式如下表所示：

数据位	功能	格式
Bit7-Bit0	FFT 点数配置	$\log_2 N$，N为点数
Bit8	IFFT/FFT 切换	1：FFT　0：IFFT

FFT_init 与 PS 端通信的 AXI4-Lite 参数如下图所示：

寄存器	功能	数据范围
slv_reg0	FFT 点数配置	0-14，整数
slv_reg1	IFFT/FFT 切换	0 和 1

2.2.4.2 Zoom-FFT

针对综合测试平台在窄带频谱分析中难以区分相近谱线的问题，本模块采用长记录采集、复数混频、低通抽取和短点数 FFT 的处理方法。PS 负责混频与任务调度，HLS 核完成多级半带滤波、抽取和可选 Hann 加窗，频谱运算复用原 PL FFT 核。在约 8.192 MHz 采样率、8192 点 FFT、64 倍抽取的设计配置下，有效观测时长约 64 ms，频率格距约 15.625 Hz。

普通 FFT 在采样率约 8.192 MHz、输入 8192 点时仅覆盖约 1 ms，频率格距约 1000 Hz。Zoom FFT 则先采集长记录，再保留目标窄带信息，使较短的 FFT 对应更长的观测时间。

Zoom FFT 首先通过 PS 进行复数混频（数字下变频）将 fc 附近的正频率分量搬移到零频附近，数学公式如下：

$$u[n] = x[n]e^{-j2\pi f_c \frac{n}{F_s}} = I[n] + jQ[n] \quad (1)$$

然后由 HLS 用最多六级 63 抽头半带 FIR 进行抗混叠滤波，每级抽取 2 倍。低通滤波抑制带外分量及搬移后的镜像，随后才能抽取；直接间隔取样会产生混叠。抽取倍率和输出采样率为：

$$D = 2^L, \quad F_{out} = \frac{F_s}{D} \quad (2)$$

抽取后的 N 个有效复数点经可选是否经过周期 Hann 窗送入原 FFT 核。每帧滤波历史清零，并丢弃启动瞬态；当前实现按保守规则计算启动区间 W 与最少原始输入点数 Nin：

$$W = 62\sum_{l=0}^{L-1} 2^l = 62(D-1), \quad N_{in} = W + ND \quad (3)$$

有效 FFT 记录不包含启动区间。频率格距与有效观测时长满足：

$$\Delta f = \frac{F_s}{DN}, \quad T_{eff} = \frac{ND}{F_s} = \frac{1}{\Delta f} \quad (4)$$

表 典型配置下的频谱分析比较

项目	普通 FFT	Zoom FFT 设计配置
原始输入点数	8192	528194 含启动区间
输出点数	8192	8192
有效观测时长	约 1 ms	约 64 ms
频率求值格距	约 1000 Hz	约 15.625 Hz

当 D=64、N=8192 时，W=3906，Nin=528194，完整采集时长约 64.477 ms。100 Hz 间隔约为 6.4 个频格，Hann 主瓣零点间宽度约为 62.5 Hz，稳定等幅且位于通带内的双音具备分离条件。15.625 Hz 是频率格距，实际双音分辨能力仍受窗、噪声、幅度差和时钟误差影响。

为验证 Zoom FFT 的窄带频谱分析原理，采用采样率为 8.192 MHz、频率分别为 150000 Hz 和 150100 Hz 的等幅双音信号进行 MATLAB 仿真。普通 FFT 对 8192 个原始采样点进行分析，观测时长为 1 ms，频率格距为 1000 Hz，无法有效分离间隔 100 Hz 的双音。Zoom FFT 先以 150050 Hz 为中心进行复数混频，采用与 63 抽头半带系数 Fir 算子，通过六级低通滤波和二倍抽取实现总计 64 倍抽

取，再对 8192 个有效复数点进行 FFT。两种方法均采用周期 Hann 窗。Zoom FFT 的有效观测时长为 64 ms，频率格距为 15.625 Hz，频谱中可清晰区分两个分量。结果说明，Zoom FFT 能够在保持 FFT 点数不变的情况下，通过较长记录的窄带处理提高频率分析能力。

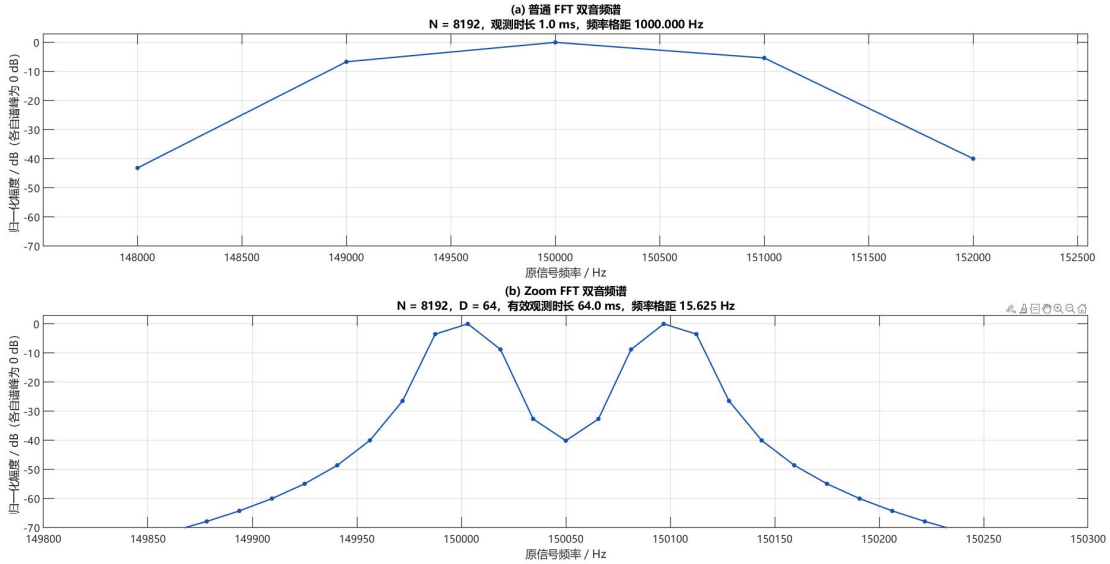


图 2-14　FFT init ip 核状态转移图

HLS 核内部执行滤波、抽取、加窗及输出缩放。**滤波抽取通过 Vitis HLS FIR 算子库调用 AMD FIR Compiler LogiCORE IP 实现。**数据链依次为原 FFT DMA 的 MM2S、FIFO3、zfft_preprocess、xfft_0、FIFO2 与 S2MM；前处理核位于 FIFO3 后、FFT 前，原 DMA、FIFO 和 FFT 核继续复用。

旁路模式原样传递数据，使普通 FFT 和 CZT 所需的 FFT/IFFT 事务共用同一条处理链。

图 2-15 给出前处理核的数据通路、存储和控制关系。PS 已完成混频，核内负责滤波、抽取、可选加窗及输出组帧；后级 FFT 在核外复用。

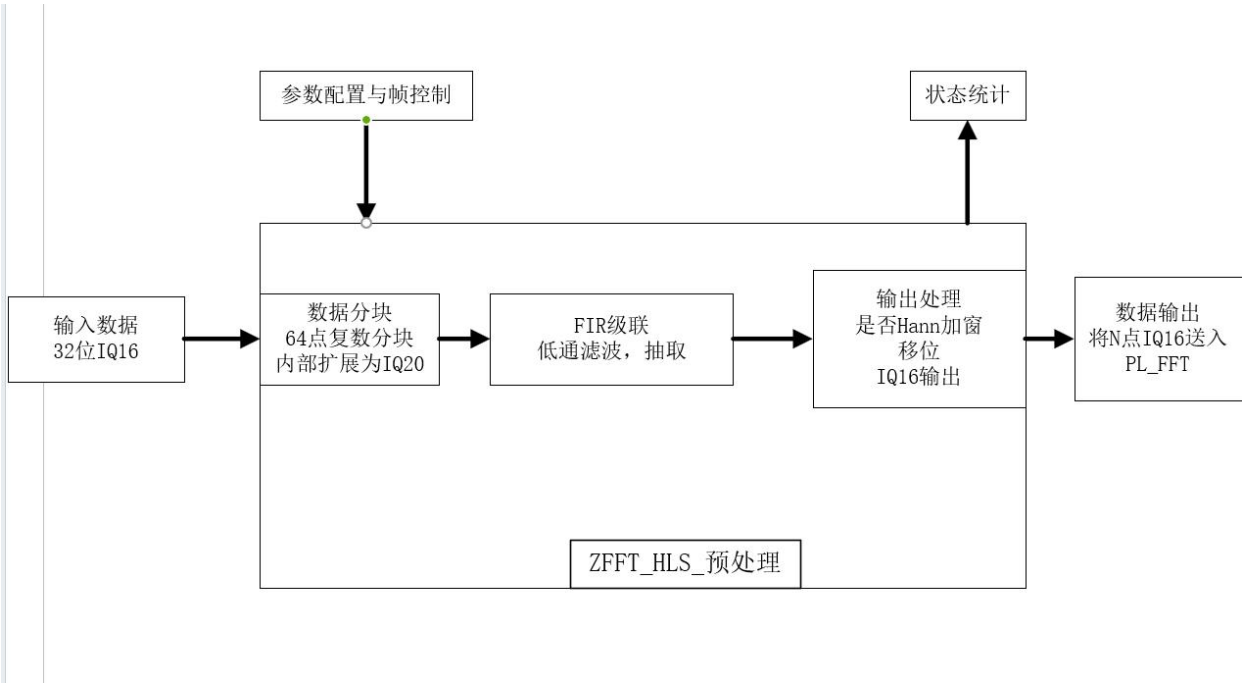


图 2-15ZFFT 前处理核内部结构

核心算法：

半带低通滤波与二倍抽取：

I、Q 两路使用相同的实系数半带 FIR。每级滤波后保留间隔为 2 的采样点，可合并表示为：

$$v_i[m] = \sum_{\ell=0}^{62} h_i[\ell]\, v_{i-1}[2m-\ell] \quad , \quad v_0[n] = u[n]$$

其中，$(h_i[\ell])$为第 i 级 63 抽头滤波器系数。滤波先抑制带外分量，再抽取，以避免混叠。

启用 L 级后：

$$D = 2\text{^}L,\ F_s' = \frac{F_s}{D}$$

Hann 加窗：丢弃滤波启动阶段的数据，取稳定后的 N 点复数序列（$v_{\text{eff}}[m]$），施加周期 Hann 窗：

$$w[m] = \frac{1}{2}\left(1 - cos\frac{2\pi m}{N}\right), \quad 0 \le m < N$$

$$z[m] = v_{eff}[m]w[m]$$

I、Q 两路乘以同一窗系数，得到送入后级 PL FFT 的复数数据。图 3-3 按一帧调用归纳参数检查、模式选择、初始化、数据流处理与完成返回。Zoom 模式先向启用的 FIR 级输入零并排空响应，清

除前帧滤波历史，再由输入、各级滤波抽取和输出任务通过 DATAFLOW 流水衔接。输出端丢弃预热数据，完成可选加窗与缩放，输出 N 点并生成末点 TLAST，必要时补零。待剩余输入和核内数据排空、各任务完成后，汇总状态与点数统计并返回，软件据此判断

本帧是否有效。

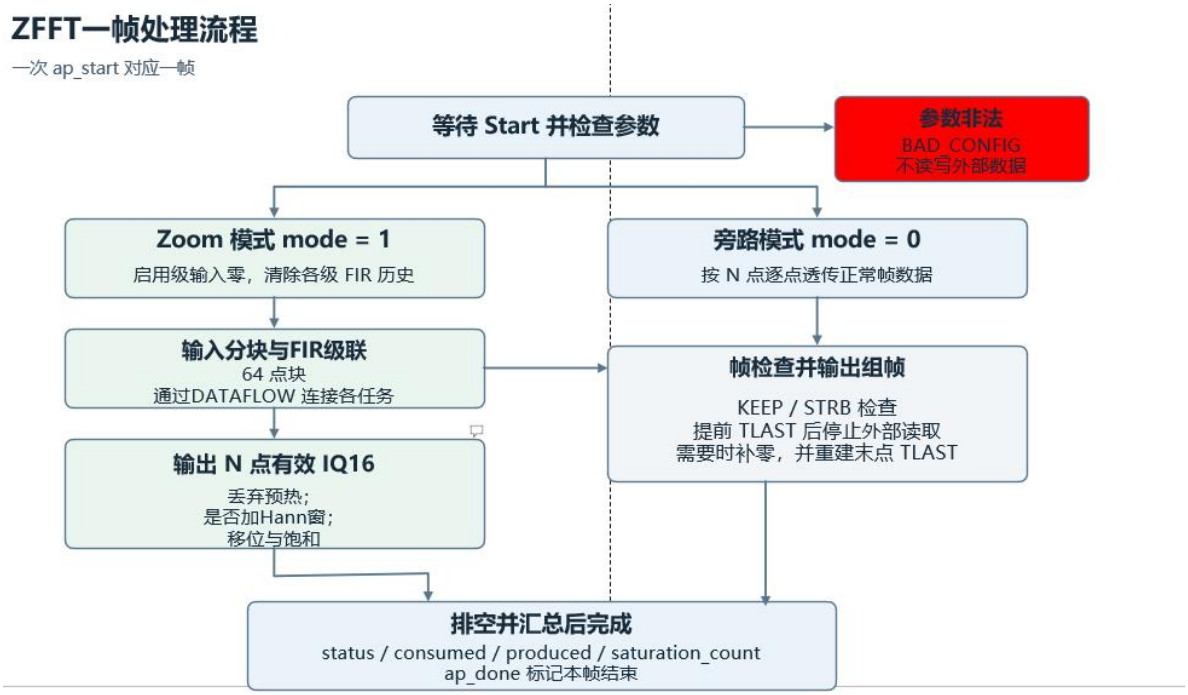


图 2-16 ZFFT 整帧控制状态转移示意

图 2-17 以 L=3、D=8 说明独立 FIR 级联与块级流水调度。各级使用独立的 FIR 实例，每个输入块的 64 个复数点依次抽取为 32、16、8 点。输入、各级滤波抽取与输出任务通过 FIFO 衔接，不同数据块可同时处于不同处理阶段，实现 DATAFLOW 流水重叠。横轴为示意时间槽，用于说明任务间的衔接关系，不代表实际时钟周期或加速倍数。

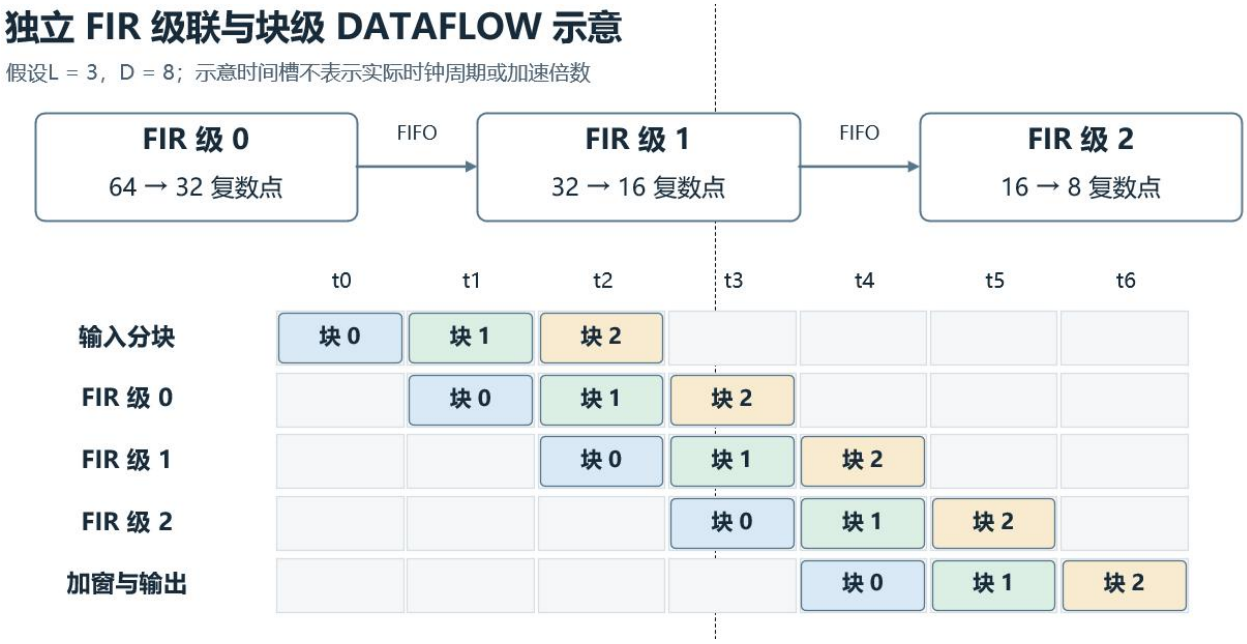
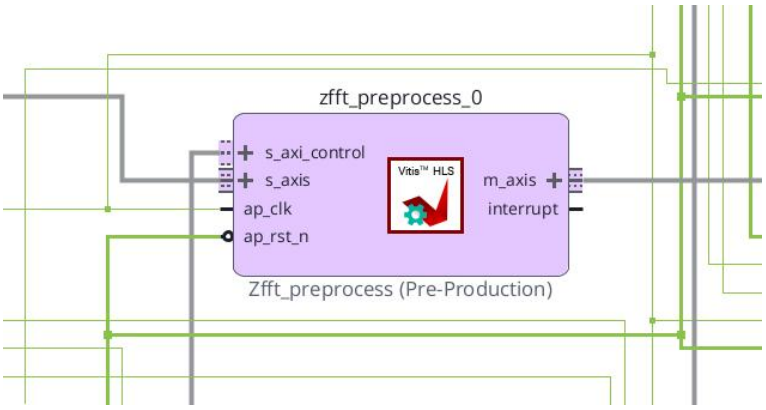


图 2-17 ZFFT 整帧控制状态转移示意

新增核 zfft_preprocess 分为六个接口：m_axis 与 s_axis 采用 AXI4-Stream 接口负责数据输入输出；s_axi_control 采用 AXI4-Lite 接口做 PS 端控制；ap_clk 为模块提供时钟；ap_rst_n 低电平有效的复位接口；并预留 interrupt 中断接口，可用于向 PS 通知任务完成等事件，为后续采用中断方式管理任务提供支持。



高速数据通过原 FFT DMA 与 AXI4-Stream 传递，模式及处理参数通过 AXI4-Lite 配置。控制参数包括 mode、decim_log2、fft_points、input_samples、window_enable 与 output_shift；

控制参数	作用	取值及约束
mode	选择处理路径	0：普通旁路，原样传递数据。 1：Zoom，执行滤波、抽取及可选加窗。
decim_log2	设置抽取级数 L	Zoom：$L=1\sim6$，$D=2^L$（2～64 倍）；旁路忽略。
fft_points	设置有效输出点数 N	Zoom：N 为 8～8192 的 2 的幂，须与后级 FFT 配置一致。
input_samples	设置输入帧样本数	Zoom：$N_{\text{in}}\geq 62(D-1)+ND$，$N_{\text{in}}\leq 1048575$。 旁路：$N$ 个数据字。单位为 32 位 I/Q 数据字，每字含一组 I/Q。
window_enable	控制周期 Hann 窗	0：关闭；1：开启，用于抑制频谱泄漏；旁路忽略。
output_shift	设置输出右移位数 s	Zoom：$s=0\sim4$，幅度约缩小 2^s 倍，降低输出饱和风险；PS 换算时补偿，旁路忽

		略。

软硬件划分兼顾参数灵活性、重复运算吞吐和资源余量。PS 保留中心频率可调的混频与任务管理；HLS 以官方 FIR 的 DSP 乘加流水完成重复滤波抽取，并通过 DATAFLOW 连接接收、各级处理及加窗输出；原 PL FFT/IFFT 继续承担频谱运算。独立 FIR 实例与 FIFO 增加 PL 资源，换取级间并发处理能力。端到端时间还包括采集、PS 混频、缓存维护及 DMA，应在同一输入记录、N、D、窗和时钟条件下分项计时后计算加速比。

2.2.4.3 levy 最小二乘法加速模块

采用实系数二阶模型，分母最高次系数固定为 1，分子至多一阶：

$$H(s)=\frac{b_0+b_1 s}{s^2+a_1 s+a_0}. \quad (1)$$

DDS 正弦激励被测系统，上游从两路信号提取同频复数相量，得到频响 H=Y/X。

频响公式为：

$$\omega_k=2\pi f_k,\quad H_k=H(j\omega_k)=u_k+jv_k=\frac{Y_k}{X_k}. \quad (2)$$

其中X_k为 ADC1 测得的输入，Y_k为 ADCO 测得的输出复数，u 为频响的实部，v 为虚部。两路均需提取同一激励频率下的复数相量，保留幅度与相位信息。

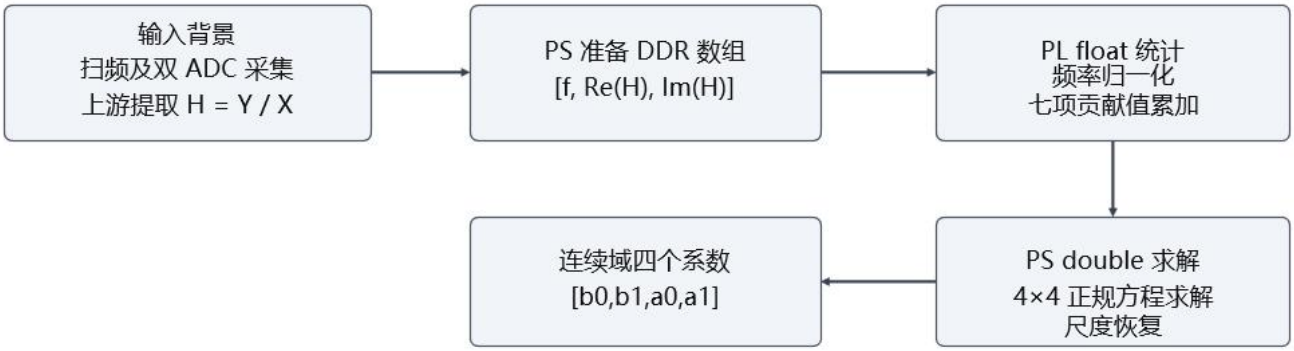


图 2-1 Levy 辨识模块功能框图

归一化频率：为减小频率平方引起的数量级差异，采用参考角频率 ωr=10000 rad/s，同时归一化频率和四个未知系数：

$$t_k = \frac{\omega_k}{\omega_r}, \quad \theta = \begin{bmatrix} c_0 \\ c_1 \\ d_0 \\ d_1 \end{bmatrix} = \begin{bmatrix} b_0/\omega_r^2 \\ b_1/\omega_r \\ a_0/\omega_r^2 \\ a_1/\omega_r \end{bmatrix}. \quad (3)$$

归一化后交叉相乘，保留模型误差对应的代数残差：

$$H_k(d_0 - t_k^2 + jd_1 t_k) \approx c_0 + jc_1 t_k.$$

将$[H_k \approx \frac{b_0+j\omega_k b_1}{a_0-\omega_k^2+j\omega_k a_1}]$带入上式并分离实部和虚部等式，每个有效频点形成两行观测方程：

$$\begin{bmatrix} 1 & 0 & -u_k & t_k v_k \\ 0 & t_k & -v_k & -t_k u_k \end{bmatrix} \theta = \begin{bmatrix} -t_k^2 u_k \\ -t_k^2 v_k \end{bmatrix}.$$

将 M 个有效频点的观测行叠加为 2M×4 矩阵 A 和右端向量 r，通

过线性最小二乘求解四个未知量：

由于存在噪声，通常不能让每一条方程都完全成立。定义残差：

$$[e = A\theta - r]$$

最小二乘就是让残差平方和最小：

$$[J(\theta) = e^{\mathrm{T}}e = \|A\theta - r\|_2^2]$$

将目标函数展开后对θ求导并令其等于 0，整理得到：

$$[A^{\mathrm{T}}A\theta = A^{\mathrm{T}}r]$$

工程中再记：

$$[G = A^T A, \qquad q = A^T r]$$

最后得到：

$$G\theta = q$$

展开正规方程后，无需传输完整观测矩阵，只需实际点数 M 和表 1 所列七项统计量。其中 t、u、v 均对应同一有效频点。

表 构造正规方程所需的七项统计量

统计量	所有有效频点的统计结果
m0	$\sum_k t_k^2$
m1	$\sum_k u_k$
m2	$\sum_k t_k v_k$
m3	$\sum_k t_k^2 u_k$

统计量	所有有效频点的统计结果
m4	$\sum_k (u_k^2 + v_k^2)$
m5	$\sum_k t_k^2 (u_k^2 + v_k^2)$
m6	$\sum_k t_k^3 v_k$

PS 据此构造 4×4 矩阵 G 和右端向量 q；统计运算随 M 增长，求解规模始终固定为四个未知量：

$$G = \begin{bmatrix} M & 0 & -m_1 & m_2 \\ 0 & m_0 & -m_2 & -m_3 \\ -m_1 & -m_2 & m_4 & 0 \\ m_2 & -m_3 & 0 & m_5 \end{bmatrix}, \quad q = \begin{bmatrix} -m_3 \\ -m_6 \\ m_5 \\ 0 \end{bmatrix}. \quad (5)$$

Levy 最小化的是交叉相乘后的代数残差，其权重与直接拟合复数频响不同。因此系数求解成功后，还需在频响层面检查模型误差。模型质量应通过重建频响与测量频响的相对 RMS 独立检查，其中带帽符号表示模型频响：

$$\varepsilon_H = \sqrt{\frac{\sum_k |\hat{H}(j\omega_k) - H_k|^2}{\sum_k |H_k|^2}}. \quad (6)$$

MATLAB 原理仿真

为验证二阶 Levy 传递函数辨识原理，选取已知系统：

$$H(s) = \frac{7.5 \times 10^7 + 1500s}{s^2 + 4000s + 10^8}$$

在 100～10000 Hz 范围内均匀生成 2048 个复数频响点，参考角频

率取 10000 rad/s。按照软硬件划分，采用单精度浮点运算模拟七项统计量的八组交错累加，再采用双精度运算构造并求解 4×4 正规方程，恢复传递函数系数。

将辨识结果与原系统的幅频、相频曲线进行对比，两者基本重合，相对复数频响 RMS 误差约为 5.298×10^{-7}，表明该方法能够准确恢复所选模型。仿真输入为无噪声合成频响，用于验证辨识原理与数值计算过程。

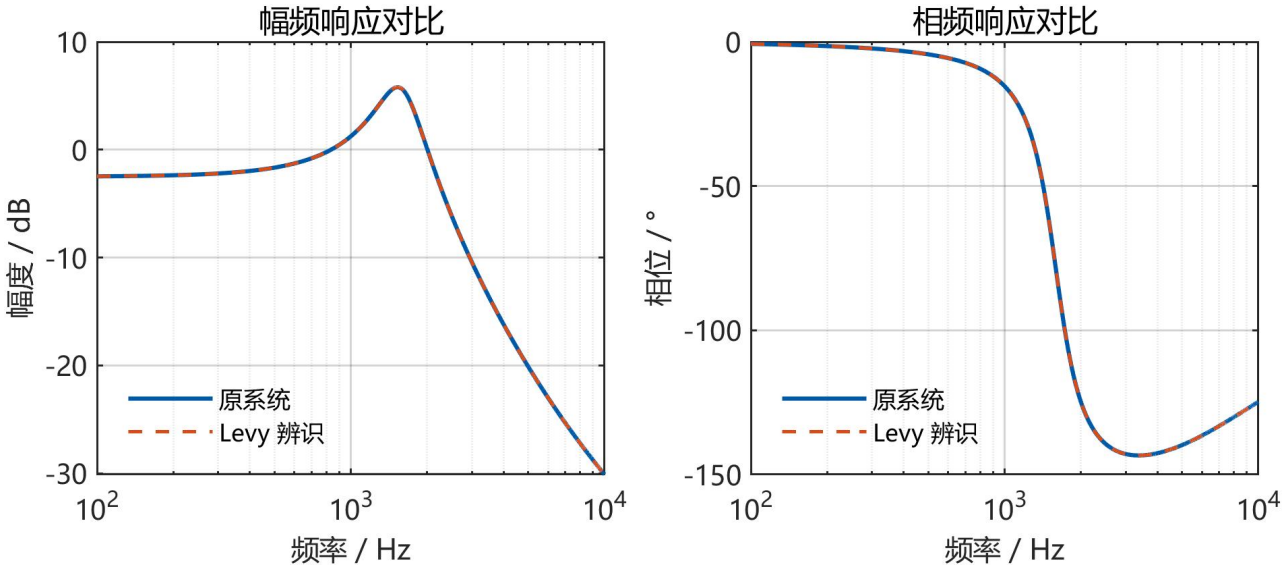


图 2-18 原系统与 Levy 辨识结果的幅频和相频对比

新增核 levy_moments_hls 包含五类接口：s_axi_control 使用 AXI4-Lite 配置地址和点数、读取状态及统计量；m_axi_gmem 作为 AXI 主接口顺序读取 DDR；ap_clk 提供时钟；ap_rst_n 为低有效复位；interrupt 提供中断通知接口，当前 PS 软件采用轮询。核内只执行频率归一化、七项贡献计算、累加与归并，小矩阵求解保留在 PS。图 3-2 给出核内结构。输入按 [f,u,v] 从 DDR 读取，完成合法性检

查后生成七项贡献，写入八组局部累加器，再归并并检查溢出。所有结果通过 AXI4-Lite 返回，统计核不另设 DDR 输出缓冲。

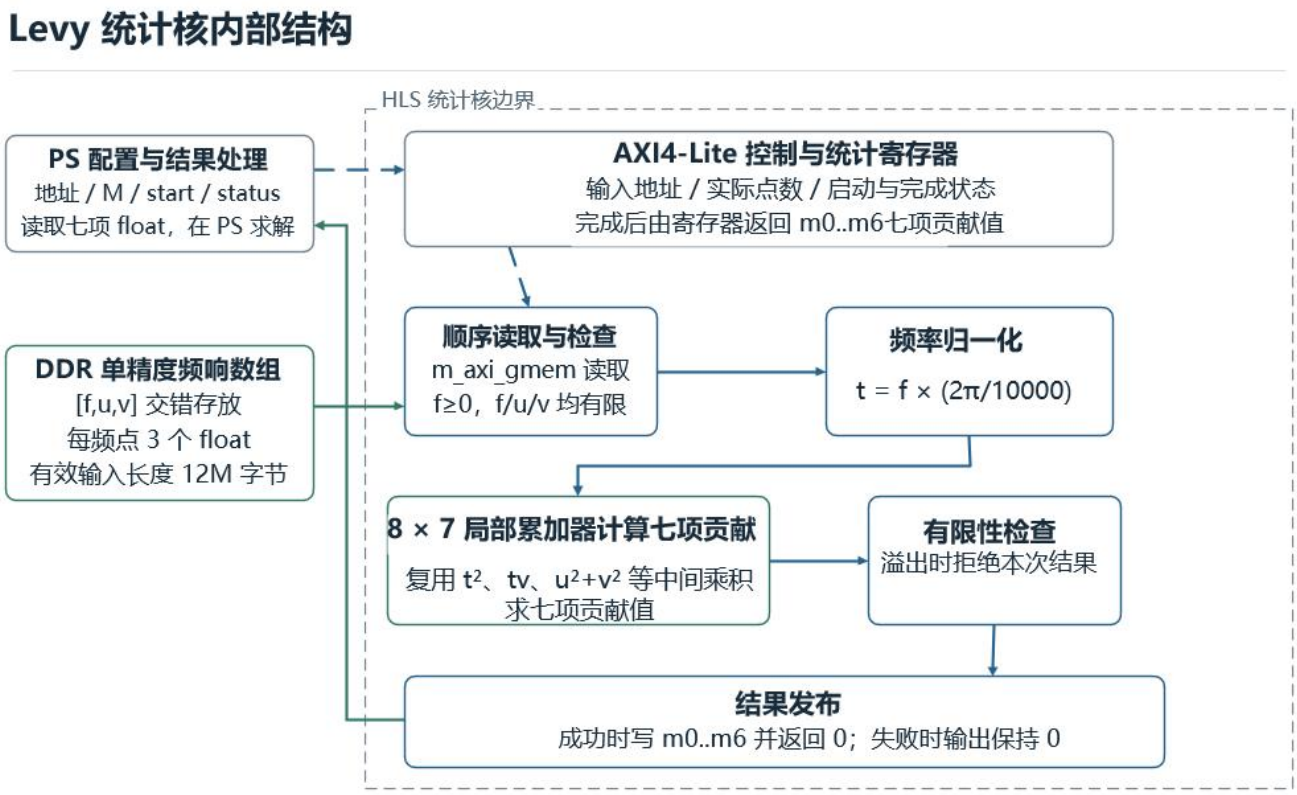


图 2-19 Levy 统计核内部结构

核心运算先复用 t^2、tv 和 u^2+v^2，再形成七项贡献。t^3v 按 $t^2 \cdot tv$ 计算，频率归一化通过乘编译时常数完成；浮点乘法和部分加法绑定 DSP，但对阶、规格化与控制仍消耗 LUT。

@资源优化通过计算分工和精度调整共同实现：将除法、开方及 QR 求解迁至 PS，PL 保留规则的单精度乘加统计。

控制通道使用 PS 的 AXI 通用接口和 AXI4-Lite，数据通道由 HLS 的 m_axi 经互连与 PS HP 接口直接访问 DDR。Levy 核

不需要独立 AXI DMA；现有 DMA 负责上游 ADC 波形搬运。

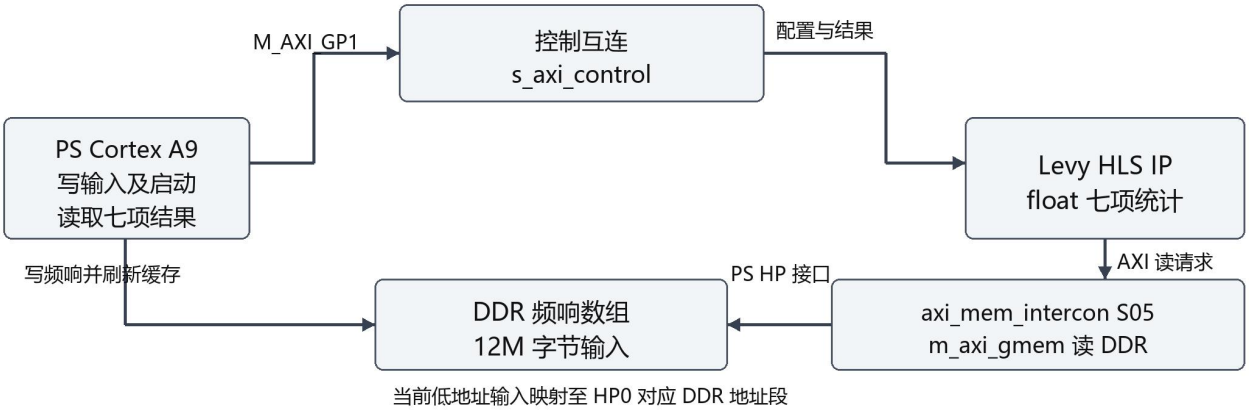


图 2-20 Levy 控制通道与 DDR 数据通道

表 输入输出与控制参数约定

接口项目	作用	数据格式与约束
samples	输入 DDR 首址	float；每点 [f,Re(H),Im(H)] 连续存放。32 字节对齐，完整数组位于 [0x00100000,0x10000000)。
point_count	实际有效频点数 M	PS 拟合接口 4～2048，PL 统计核 1～2048；输入长度 12M 字节，最大 24576 字节。
m0～m6	七项统计结果	7 个 float，共 28 字节；由 AXI4-Lite 结果寄存器读回，按浮点位模式解释。
ap_start ap_done	启动及完成标志	当前采用轮询，自动重启关闭；ap_done 读后清零，观察到完成后统一读取结果。

接口项目	作用	数据格式与约束
ap_return	PL 统计状态	0 成功；1 点数错误；2 输入非法；3 浮点溢出。矩阵判秩由 PS 完成。
coeff	PS 连续域输出	4 个 double，顺序 [b0,b1,a0,a1]；仅 LEVY_PS_OK 时有效。
调用约束	缓冲区与存储期	串行调用，输入输出不重叠；超时后保留输入，IP 空闲或受控复位前不得改写或释放。

2048 是容量上限，并非固定处理长度；波形采集的 8192 个样本也不同于扫频有效点数 M。频率必须有限且非负，频响实部、虚部必须有限；点数满足下限仍需检查矩阵是否满秩。任一点非法或统计溢出时，整次调用失败，部分统计不得作为成功结果使用。

启动流程依次为检查参数与缓冲区、确认 IP 空闲且自动重启关闭、禁用中断并清旧状态、刷新输入缓存、写地址和点数、启动并轮询完成、读取统计状态和七项结果、在 PS 求解。ap_done 读后清零，检测到完成即退出轮询。结果由寄存器返回，无 DDR 输出缓存需要失效；上游 ADC DMA 接收缓存仍需独立维护。

轮询上限为 1000000 次，并非固定时间。超时只终止 PS 等待，PL 可能继续访问 DDR；确认 IP 空闲或完成受控复位前，不得覆盖或释放输入。错误状态统一交由调用者处理，失败结果不进入

显示和后续滤波器配置。

划分依据:计算规模、数据访问规律、精度及资源。PL 处理随 M 增长的顺序乘加统计，利用流水线和 DSP；PS 处理规模固定、含主元选择与判秩分支的 4×4 求解，保留双精度并便于异常处理。方案允许一定离线等待，以降低新增 PL 资源为优先目标，整体收益仍需用相同输入和误差要求测量。

2.2.5 滤波器功能集群介绍

有限冲激响应滤波器（FIR）滤波器是数字信号处理中核心的非递归线性时不变系统，其核心定义为：单位冲激响应$h[n]$ 仅在有限个时间点（0 ≤ n ≤ N-1，N 为阶数）非零，输出仅由当前与过去有限个输入样本线性卷积决定，数学表达式为：

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

系统函数为：

$$H(z) = \sum_{n=0}^{N-1} h[n]\, z^{-n}$$

h[n]即为 FIR 滤波器的系数。

使用 FIR 对信号进行滤波时，需要考虑对系数进行量化，因为 FPGA 对于处理浮点数的速度远低于整数，固需要保证系数$h[n]$均为为整数。考虑到 FPGA 在进行 2 的整数次幂乘除时运算速度较快，量化数均为 2 的整数次幂。假设量化比特数为 m，量化过程可以用以下方式表示：

$$y[n] = (int)(\frac{1}{2^{m-1}}\sum_{k=0}^{N-1}(int)(2^{m-1}h[k])x[n-k])$$

在工程上，一般取 12bit 量化（即所有系数乘 2048）就能和原始滤波器保持良好的相似性。故在本工程，PL 硬件默认所有的参数均为 12bit 有符号数量化值。在输出时会除以 2048。

无限冲激响应数字滤波器（IIR）是一种输出信号由当前输入和过去输出共同决定的数字滤波器，其单位冲激响应为无限长序列。按处理信号维度可分为一维、二维及多维类型，具有递归型结构特征，可采用直接型、级联型等网络结构

设输入为 $x[n]$，输出为 y[n]。一般 N 阶 IIR 滤波器可写为:

$$\sum_{k=0}^{N} a_k\, y[n-k] = \sum_{m=0}^{M} b_m\, x[n-m]$$

通常写成递推形式：

$$a_0 y[n] = -\sum_{k=1}^{N} a_k\, y[n-k] + \sum_{m=0}^{M} b_m\, x[n-m]$$

传递函数为：

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{m=0}^{M} b_m\, z^{-m}}{\sum_{k=0}^{N} a_k\, z^{-k}}$$

故对于 IIR 滤波器来说，其极点的位置是任意的。在设计系数时，**一定要保证 IIR 滤波器的传递函数的极点都在单位圆内**，这样系统才会稳定，IIR 滤波器才会正常工作。

在 IIR 滤波器中，a_0可以被直接视为量化系数。量化理论和实现的过程和 IIR 滤波器一致。在本工程中a_0系数可以由 PS 端配置。

对于 FIR 滤波器具体实现，由 PS 端产生系数数据，输入 FIR 的系数（即系统的冲激响应），PS 端通过 AXI4 Lite 协议逐个传送给自编的 FIR_init ip 核，的 FIR_init ip 核会将这些系数转化成符合 FIR ip 要求核的 AXI4 Stream 协议，将这些系数配置进入 FIR ip 核，用 FIR ip 核对 ADC 输入的波形进行滤波，通过 DAC 输出滤波后的波形。AXI4 Lite 协议的参数如下表所示：

寄存器	功能	数据范围
slv_reg0-9	系数参数 h0-h9	-2048-2047，整数
slv_reg10	传输完成标志	0 和 1，上升沿有效

关于 FIR ip 核的初始化 ip 核，fir_init，参考 **2.2.4 部分** FFT ip 核的初始化算法，**使用状态机的方式配置 ip 核**。不过有几点不同：一是在传入系数时，FIR 的滤波器是逐个传入的。不可能每一次传入时均重新配置 FIR ip 核，这样会造成 FIR ip 核工作不稳定。因此，另外 PS 端在配制完成后需要传输一个完成信号 reload_en，当出现上升沿，才会由状态 S0 转为状态 S1。其二，FIR ip 核的 m_axis_reload_tdata 数据有多个，存在 tlast，因此在 S1 状态后面需要加一个 tlast 状态 S2，在传输最后一个数时拉高 m_axis_reload_tlast。其三，要在状态 S3 拉低

m_axis_reload_tvalid 和 m_axis_reload_tlast 同时拉高 m_axis_config_tvalid，告诉 ip 核配置生效。

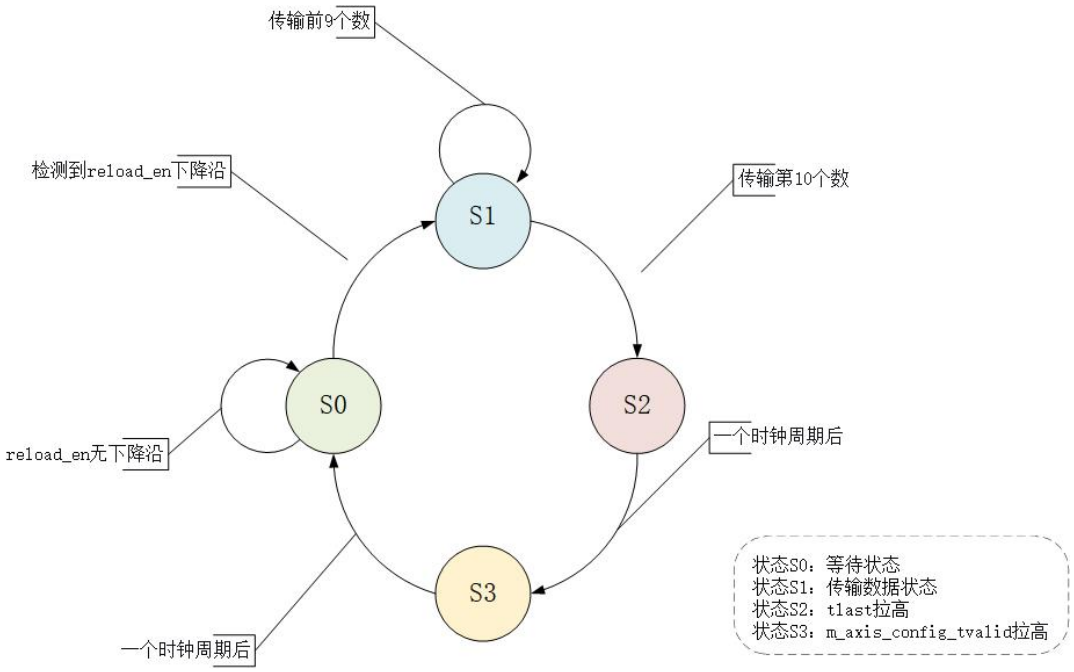


图 2-21FIR_init 状态转移图

对于 FIR_init ip 核在系数配置生效时与 FIR ip 核通信实现的具体时序时序，如图 2-22 所示。

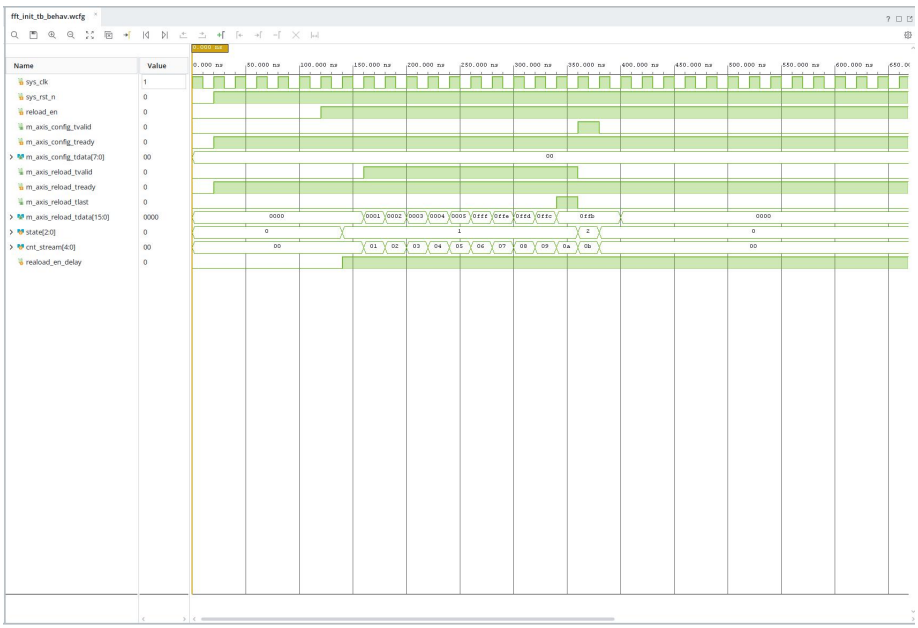


图 2-22FIR_init 单次系数配置实现的时序

对于 IIR 滤波器具体实现，用户除了可以配置全部的系数以外，量化系数也可以配置。IIR 滤波器采用自编 IP 核，采用直接型架构，分为零点系数和极点系数，最终结果为零点系数减去极点系数在除以 y[n]的系数a_0。

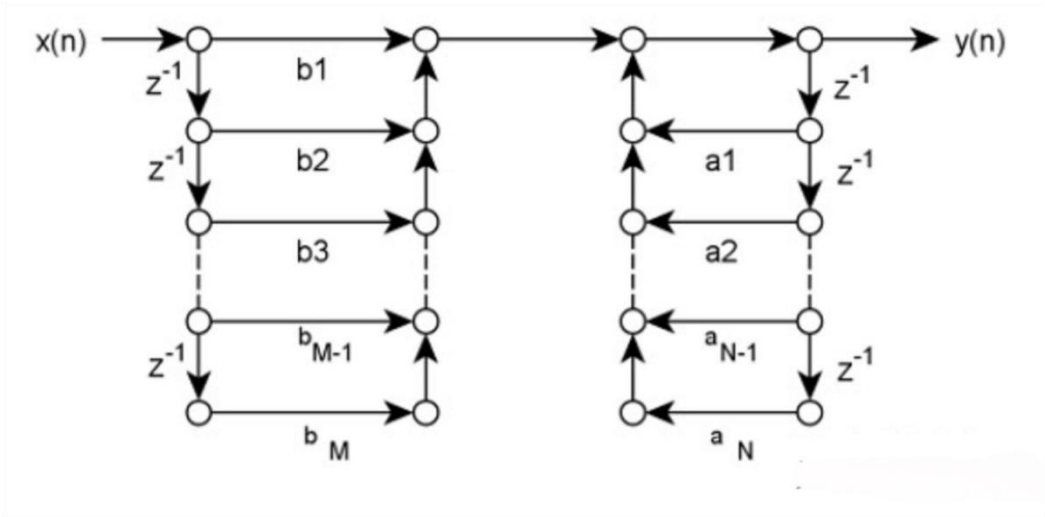


图 2-23 直接型 IIR 滤波器流程图

对于 IIR 滤波器的系数配置，由于是自编 ip 核，相关 RTL 代码一被集成进 ip 核中，无需外挂额外的 ip 核。所有的系数由 PS 端通过 AXI4_Lite 直接配置进入 IIR ip 核。对于 IIR 滤波器的参数 AXI4 Lite 协议的参数如下表所示：

寄存器	功能	数据范围
slv_reg0-7	系数 b0-b7	-2048-2047，整数
slv_reg7-15	系数 a0-a7	-2048-2047，整数

2.2.6 LCD 显示器功能集群介绍

配备 5 寸 RGB565 LCD 触控屏，全触控操作替代物理按键。多种功

能一屏切换显示，UI 界面随功能模式动态重构。使用 AMD 官方提供的 VDMA IP 核，对数据传递进行配置。自定义数据转换 IP 核，将 RGB888 转换为 RGB565，对 LCD 进行适配。5 寸 LCD 触控屏通过 40Pin FPC 排线与 PL 端连接。

VDMA 采用 Register Direct(寄存器直接模式)，只使能读通道，且无中断控制，一直使能传输，在 PS 端往 DDR 对应的地址写值即可改变对应位置像素点的颜色。同时，为了减轻 PL 端资源的负担，VDMA 的帧缓存数量设置为 1，将屏幕画面切换等 PS 端的操作负担加大，代码数量增加，以给予硬件更多的资源空间。

视频时序发生器(VTC)根据事先计算好的行同步、场同步等相关时序参数，控制 LCD 完成每一行和每一帧的显示。VTC 时钟信号通过分频器分频主时钟信号输出 33.3MHz 作为输入，最终正常情况下 LCD 的帧率可以达到 60fps 左右。

2.3 软件系统介绍（ARM/PS）

2.3.1 软件整体介绍

PS 端是整台仪器的总控制端与全部人机交互实现的地方。它一边驱动和调度 PL 里的采集与运算，一边在 LCD 液晶屏上实现多种功能的仪器界面的显示。

2.3.2 Chirp-Z 变换

Chirp-Z 变换的定义如下:

设有限长序列为：

$$x[n], \quad n = 0,1,\ldots,N-1$$

其 Z 变换为：

$$X(z) = \sum_{n=0}^{N-1} x[n] z^{-n}$$

Chirp-Z 变换计算 M个点 z_k上的 Z 变换：

$$X_k = X(z_k) = \sum_{n=0}^{N-1} x[n] z_k^{-n}, \quad k = 0,1,\ldots,M-1$$

这些采样点取为

$$z_k = AW^{-k}$$

其中

$$A = A_0 e^{j\theta_0},$$

$$W = W_0 e^{j\varphi_0}$$

于是：

$$z_k = A_0 W_0^{-k} e^{j(\theta_0 - k\varphi_0)}$$

其中

A_0 ：起始半径；

φ_0 ：起始角度；

W_0 ：半径变化率；

φ_0 ：角度步进量；

M ：输出点数，可以与 N 不同。

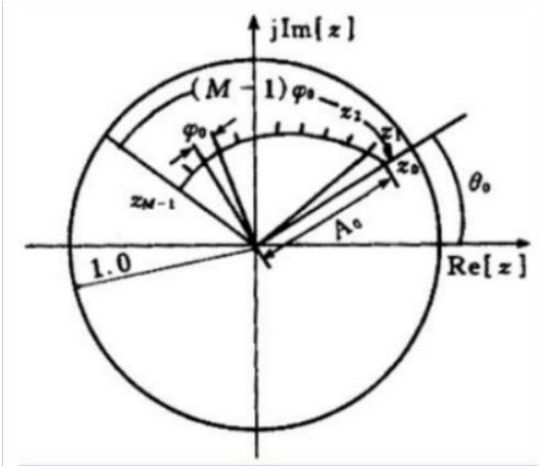


图 2-24 CZT 关键参数示意图

Chirp-Z 变换的起始频率和终止频率分别为f_{start}和f_{end}，频率步进df可以用以下公式表示

$$df = \frac{f_{end} - f_{start}}{M - 1}$$

在工程中，如果测量的为非衰减信号，A_0一般取 1，W_0取 1，即均在单位圆$e^{j\omega}$上采样，A和W可以由如下公式得到：

$$A = e^{j2\pi f_{start}}$$

$$W = e^{-j2\pi \frac{f_{end} - f_{start}}{M-1}}$$

将z_k带入X_k,有

$$X_k = \sum_{n=0}^{N-1} x[n] A^{-n} W^{nk}$$

取恒等式

$$nk = \frac{n^2 + k^2 - (k-n)^2}{2}$$

代入可得

$$X_k = W^{k^2/2} \sum_{n=0}^{N-1} \left(x[n] A^{-n} W^{n^2/2} \right) W^{-(k-n)^2/2}$$

定义

$$g[n] = x[n] A^{-n} W^{n^2/2}$$

$$h[m] = W^{-m^2/2}$$

所以有：

$$X_k = W^{k^2/2} \sum_{n=0}^{N-1} g\,[n] h[k-n]$$

即可以表示为$g[k]$和$h[k]$的卷积。梳理一下，Chirp-Z 算法的流程如下：

1.输入序列乘以 chirp 项$A^{-n}W^{n^2/2}$；

2.与另一个 chirp 序列$W^{-m^2/2}$做卷积；

3.输出结果再乘以 chirp 项 $W^{k^2/2}$

其中卷积可以使用 FFT ip 核硬件加速计算。原理如下：

$$\mathcal{F}(g[n] * h[m]) = G(r) \cdot H(r)$$

即可以有如下表示

$$G(r) = FFT(g[n])$$

$$H(r) = FFT(h[m])$$

$$g[k] * h[k] = IFFT(G(r) \cdot H(r))$$

需注意,FFT 计算的是循环卷积，当满 FFT 点数 L 满足以下条件时，才能用循环卷积表示线性卷积：

$$L \geq M + N - 1$$

其中M为$h[m]$的长度，N为信号输入长度。

算法流程如下图所示：

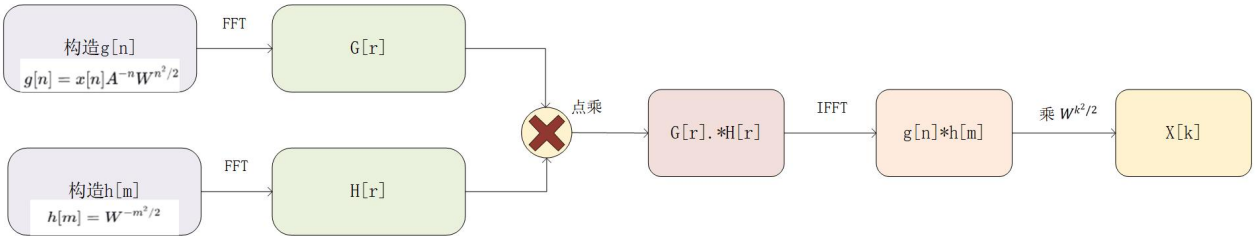


图 2-25　Chirp-Z 算法流程示意图

对于本工程软件具体实现，可以遵循以下顺序：

1. 选择 FFT 长度L

2. 构造序列$g[n]$,将 g[n]补零到长度 L

3. 构造序列 h[m]:0 到 M-1 按照公式填入，负索引以用循环方式放入长度 L 的数组。即

$$h[\mathrm{L}-m]=W^{-m^2/2}\quad m=0,1,2..N-1$$

其余位置补零。

4. 量化序列$g[n],h[m]$：转换为整数类型的数据,使其尽量覆盖 FFT ip 核最大输入允许的位数范围。

5. 打包序列$g[n],h[m]$:拼接实部和虚部，使其符合 FFT ip 要求的输入格式

6. 分别传入 PL 硬件 FFT 加速器

7. 解包序列$G(r),H(r)$ 抽取实部和虚部，除以量化值恢复真实值。

8. $G(r).*H(r)$

9. 打包、量化序列 G(r).*H(r) ，要求同上

10. 改变 FFT ip 核的工作状态为 IFFT，传入硬件 FFT 加速器

11. 解包序列$g[n]*h[m]$，要求同上，FFT ip 核的 IFFT 的功能仅是相位因子取共轭，输出数据不会除点数，最后还要除以点数 L。

12. 乘$W^{k^2/2}$，取前M项得到结果

本计算过程考虑到运算主要的时间消耗在两次 FFT 和一次 IFFT 恰好 PL（FPGA）对 FFT 的运算加速效果显著，同样的大点数运算往往 ARM 硬核花的时间往往 FPGA 的数倍至数十倍。故我们将两次 FFT 和一次 IFFT 放入硬件加速计算。而在其它其它的处理，运算量较小，多是长度为 L 的复数乘法，算上通信开销，FPGA 加速效果有限。且涉及高精度浮点数计算时，不在 FPGA 擅长的计算范围。故其它运算均放在 PS 端 ARM 硬核实现。

Chirp-Z 变换的作用是可以精细化 FFT 的频谱。在关注的频谱范围内，以极低的时间和硬件资源代价下达到增加 FFT 点数 N 的效果，消除 FFT 的栅栏效应，优化频谱仪的频率和幅度精确度，理论上可以任意精细。但是,需要注意的是，Chirp-Z 变换的结果只是让 DFT（离散傅里叶变换）在 DTFT（离散时间傅里叶变换）图像上的采样点增加，并不优化真实物理的频率分辨率。当两个信号频率之间的间隔小于真实物理的频率分辨率时，Chirp-Z 变换的频率和幅度的结果仍然不可信，能提高物理的频率分辨率的方法必须增加采样点数。

如图 2-24 所示，在三个不同频率的输入信号叠加 FFT 和 chirp-z 变换效果对比 MATLAB 仿真图。可以看到，在 40KHz-60KHz

频率段，chirp-z 变换可以准确测量信号的幅度和频率位置，误差极小。而 FFT 可以看到受到栅栏效应的影响，完全无法确认其频率和位置。

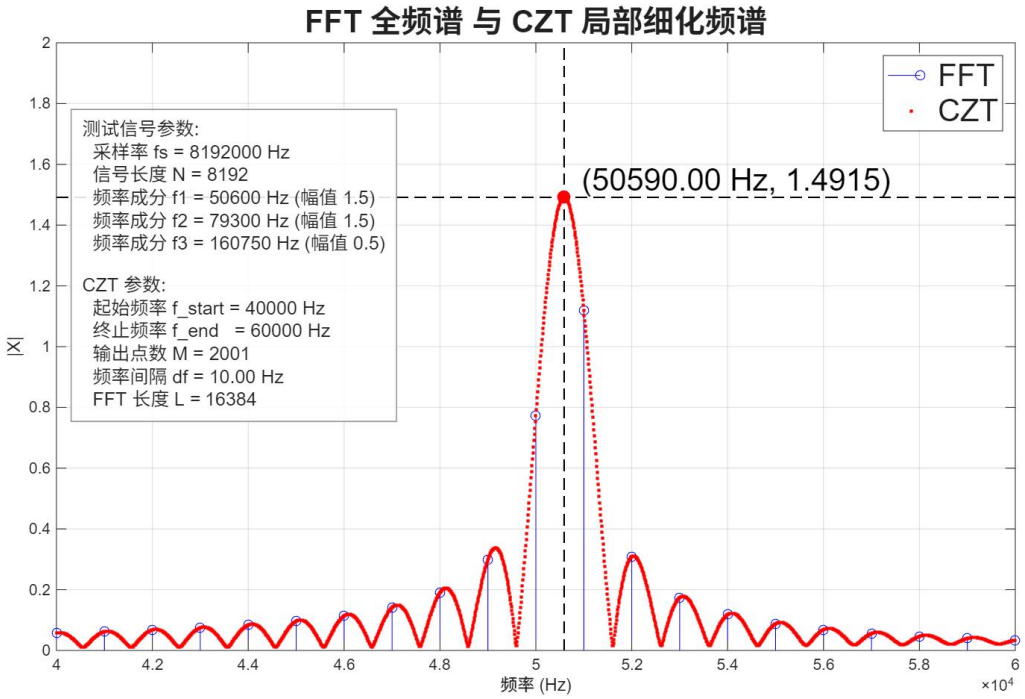


图 2-26 CZT 与 FFT 的 MATLAB 仿真对比

此外 Chirp-Z 变换通过更改A和 W，可以测量带指数衰减信号的频谱，在其它工程中应用广泛。

2.3.3 示波器 Sinc 插值

受 ADC 硬件资源限制，其最大采样率为 64Msps。采样信号频率为 1MHz 时，其一个周期内的采样点数只有：

$$N = \frac{f_s}{f_m} = 64$$

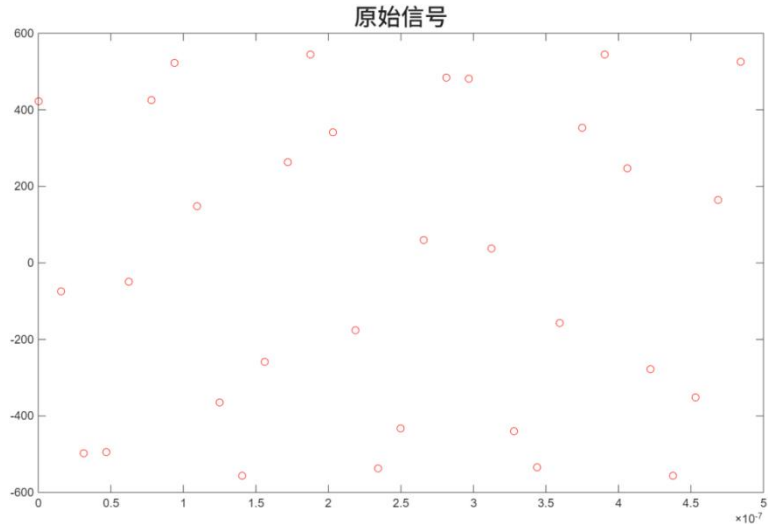
此时直接用原始采样数据在屏幕上绘制波形，已经会出现明显的锯齿状，波形不平滑。所以引入 Sinc 插值算法，平滑波形。该算法的理论公式如下：

$$x(t) = \sum_{n=-\infty}^{\infty} x(nT)\frac{\sin[\frac{\pi}{T}(t-nT)]}{\frac{\pi}{T}(t-nT)}$$

公式的计算过程本质上是采样信号与 Sinc 函数进行卷积运算，那么为了让还原的波形更加准确、平滑，则将抽头数和插值点数提高。同时为了保障示波器的刷新率，使用 FFT 进行加速卷积运算，使还原波形又快又准。

在使用了 Sinc 插值后，基于 ADC 硬件最大采样率 64Msps，在过采样理论范围内，可以测量信号的频率达到 30MHz，测量出的幅值和频率虽有小幅跳动，但大体上测量数值很准，误差很小。

如下图所示，**为 sinc 插值原始信号 10MHz，采样率 64Msps 的，取单次采样点数 N=64 插值的 MATLAB 仿真图，**可以看到几乎还原原始正弦信号。



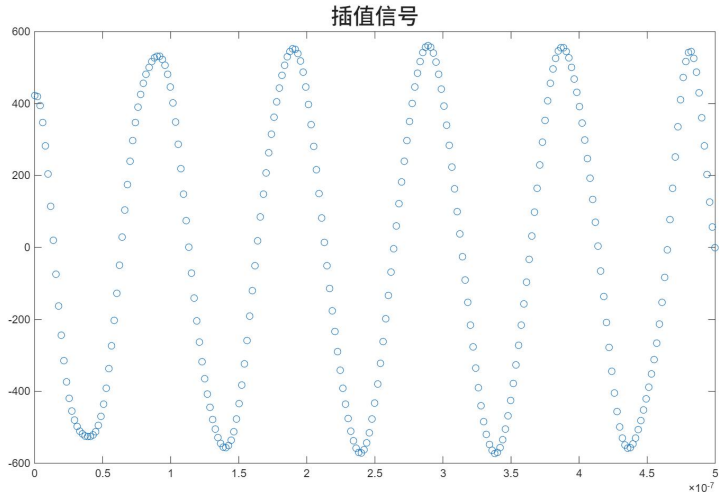


图 2-27 插值前后 MATLAB 仿真对比

2.3.4 频率特性测试仪

利用 AMD（Xilinx）官方提供的 FFT ip 核，使用信号发生器的功能在测试范围内进行正弦扫频，利用 ADC 的两路通道，一路测量信号发生器的输出信号（被测系统的输入信号）X(k)，一路得到被测系统的输出信号 Y(k)，被测系统的幅频、相频特性和传递函数，数学表达式为：

$$|H(k)| = \frac{Y(k)}{X(k)}$$

$$\phi(k) = \phi_y(k) - \phi_x(k)$$

通过曲线拟合可以得到$|H(\omega)|$和$\phi(\omega)$，则传递函数如下：

$$H(j\omega) = |H(\omega)| \cdot e^{j\phi(\omega)}$$

冲激响应则通过对传递函数求逆傅里叶变换得到：

$$h[n] = IFFT(H[k])$$

阶跃响应则通过对冲激响应(离散)积分得到：

$$s[n] = \sum_{i=0}^{n} h[i]$$

实际实现时，连硬件连接关系接如下图所示利用装置自身的 DDS 模块输出可调的正弦扫频信号，将 DAC 输出端口一路接被测系统输入，被测系统输出接一路 ADC 输入，另一路直接接另一路 ADC 输入端口，作为参考信号。完成搭建后，可自主设置扫频频率范围和扫频步长。当扫频运行结束后，装置会自动计算和拟合被测系统的幅频响应曲线、相频响应曲线、冲激响应曲线、阶跃响应曲线以及系统传递函数。曲线可通过按键切换显示。

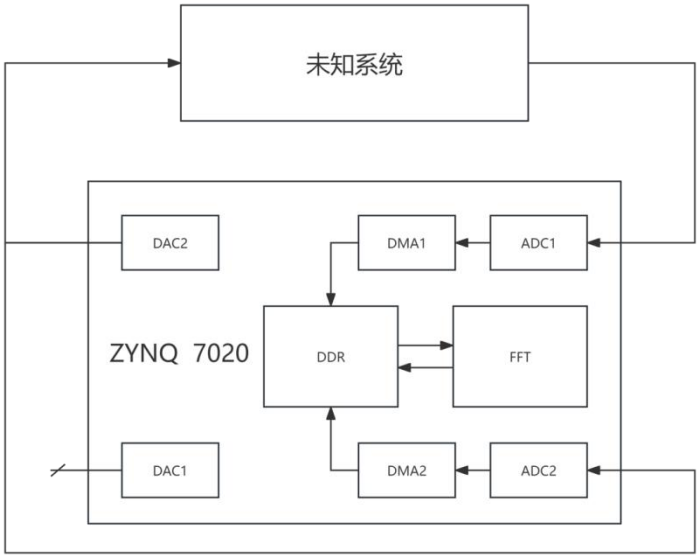


图 2-28 频率特性测试仪硬件数据通路

计算拟合得到的幅频响应曲线会自动显示系统的截止频率和系统的传递函数，幅频和相频曲线界面有光标显示，通过左右移动光标位置可查看对应扫频步长的点的相关数值。如下图所示，为频率特性测试仪的完整的程序流程图。

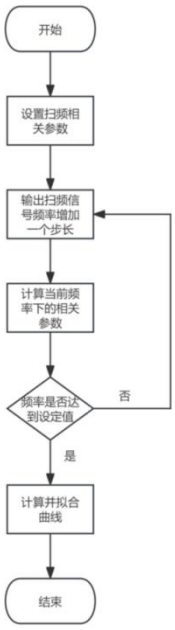


图 2-29 频率特性测试仪程序流程图

2.3.5 Zoom-FFT 软件部分

软件端根据用户设置的中心频率和抽取倍率完成参数校验与采样长度计算，将原始采样数据存入 DDR。随后，软件配置硬件处理参数并启动 DMA，调度 HLS 预处理与 FFT 运算。结果回传后，软件完成频谱数据校验、幅相计算和频率坐标恢复，输出目标频段的分析结果。PS 使用旋转递推生成混频正余弦，每 1024 点重新定相以限制漂移，并将 I/Q 量化到 16 位。

生成相位：

$$[\theta[n] = \frac{2\pi f_c n}{F_s}]$$

将实数采样信号分别与余弦、负正弦相乘，得到正交分量：

$$[I[n] = x[n]\cos\theta[n]]$$

$$[Q[n] = -x[n]\sin\theta[n]]$$

组成复数基带信号：

$$[u[n] = I[n] + jQ[n] = x[n]e^{-j2\pi f_c n/F_s}]$$

该操作将频率 f 搬移至 f-fc。PS 将生成的 I/Q 数据通过 DMA 送入 HLS 模块。

频率轴使用抽取后的采样率，并按自然顺序 FFT 输出构造有符号频率索引：

$$k_s = \begin{cases} k, & 0 \le k < N/2 \\ k-N, & N/2 \le k < N \end{cases}, \quad f[k] = f_c + \frac{k_s F_s}{DN} \quad (5)$$

幅度换算须补偿周期 Hann 的相干增益和 output_shift。对镜像分离、通带增益近似为 1 且接近频格中心的实数正弦，若 Cw 为窗相干增益、s 为输出右移位数，则可近似按下式恢复幅度：

$$\hat{A}[k] \approx \frac{2^{s+1}}{NC_w}\sqrt{Re(Y[k])^2 + Im(Y[k])^2} \quad (6)$$

图 2-30 按职责划分软件模块：main 负责本帧计划、ADC 记录和频谱解释，zfft.c/.h 封装入口检查、PS 混频及 DMA/HLS/FFT 事务。先配置并维护缓存，再依次启动 S2MM、HLS 和 MM2S；TX、RX 与 HLS 完成后，核对 RX 实收字节数及 HLS status，存在错误或饱和时拒绝本帧结果。

输入与输出缓冲的容量、互不重叠和 DMA 可访问性由调用方保证；zfft 入口检查指针、32 字节对齐和有效原始点数。

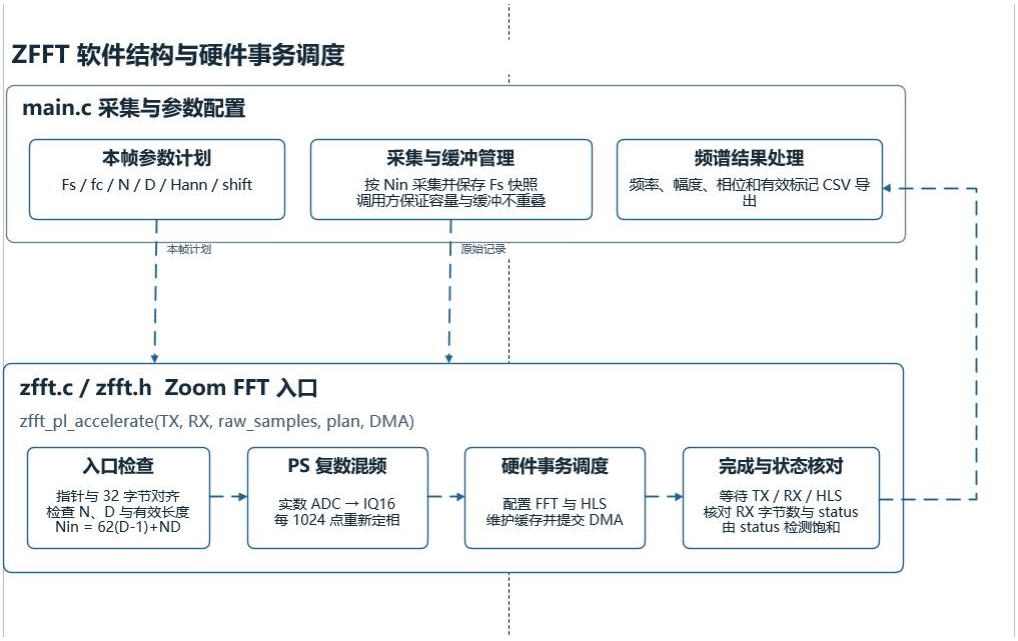


图 2-30ZFFT 软件模块结构与软硬件事务调度

2.3.6 人机交互

所有的人机交互操作均在 LCD 屏上进行，此 LCD 屏为电容式触摸屏，触摸信号通过 PL 端的一路 I2C 进行采集，后下发到 PS 端进行相关驱动和控制的编写。

使用时，操作者可通过点击屏幕上的虚拟按钮进行与该界面和功能相关的操作，以及切换界面、弹出子窗口、调用数字键盘进行相关参数输入等。

第三部分 性能与资源优化

3.1 性能与资源优化概述

ZYNQ 7020 资源有限，在我们开发过程中，经常遇到 LUT、DSP 和 BRAM 资源不够用的情况。我们在开发的过程中务必在权衡性能的情况下，对每一个自编 IP 核官方 IP 和进行资源优化，减少不必要的资源浪费。同时，不够良好的 verilog 设计会造成时序中不满足要求，进而导致时钟频率无法提高;对于 PS 端软件层面，冗余的驱硬件动代码也会影响硬件性能的发挥。下面，我们将通过几个例子，阐述我们性能与资源优化的过程。

3.2 DDS 输出时序优化

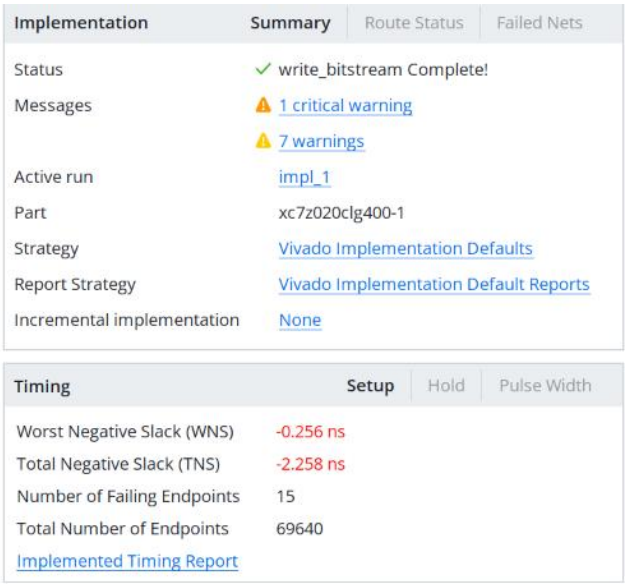
为了能灵活调整 DDS 波形的输出幅度，在输出信号时，我们要对输出信号进行幅度缩放。进行幅度缩放需要精心较为大量的计算。如果我们不加处理，进行布局布线时会造成时序违规，最差负时序裕量（WNS）会为负。因此，我们起初在幅度计算时，会采用流水线的形式，分级计算。确保时序不违例。

起初，我们在 DAC 工作时钟为 50MHz 时，采用以下写法：

```
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        mult_stage1 <= 23'd0;
        mult_stage2 <= 31'd0;
        dac_data_1 <= 14'd0;
    end else begin
        mult_stage1 <= (dac_data_mid-8191)* amplitude+8192000;
```

```
        mult_stage2 <= mult_stage1 /100;
        if (mult_stage2> 16383)
            dac_data_1 <= 14'd16383;
        else
            dac_data_1 <= mult_stage2[13:0];
    end
end
```

其中，dac_data_mid 为波形数据表输出数据，amplitude 为幅度控制字，由 PS 端发送，范围为 0-100（线性对应放缩幅度）。在 50MHz 情况下，vivado 编译正常，没有任何时序违例的警告。然而，当我们想要优化性能，提高 DAC 时钟频率，将其提高到 120MHz 时，编译后出现时序违例，如下图所示：



Implementation	Summary	Route Status	Failed Nets
Status	✓ write_bitstream Complete!		
Messages	⚠ 1 critical warning		
	⚠ 7 warnings		
Active run	impl_1		
Part	xc7z020clg400-1		
Strategy	Vivado Implementation Defaults		
Report Strategy	Vivado Implementation Default Reports		
Incremental implementation	None		

Timing	Setup	Hold	Pulse Width
Worst Negative Slack (WNS)	-0.256 ns		
Total Negative Slack (TNS)	-2.258 ns		
Number of Failing Endpoints	15		
Total Number of Endpoints	69640		
Implemented Timing Report			

于是，我们决定对我们的 verilog 代码进行优化。其一，将下面这句计算过冗长的 verilog 代码再拆分为两级流水线

```
    mult_stage1 <= (dac_data_mid-8191)* amplitude+8192000;
```

修改后为：

```
        mult_tmp   <= (rom_dout_r - 8191) * amp_r;
```

```
        mult_stage1 <= mult_tmp + 1048576;
```

其二，将 ROM 波形表和幅度再加一级流水线：

```
   rom_dout_r <= dac_data_mid;
        amp_r      <= amplitude;
```

其三，将 amplitude 的范围由 0-100 改为 0-128，在对幅度进行缩放时直接用算数右移 7 位取代除以 100，提高速度。

```
mult_stage2 <= mult_stage1 >> 7;
```

于是，总体代码如下图所示：

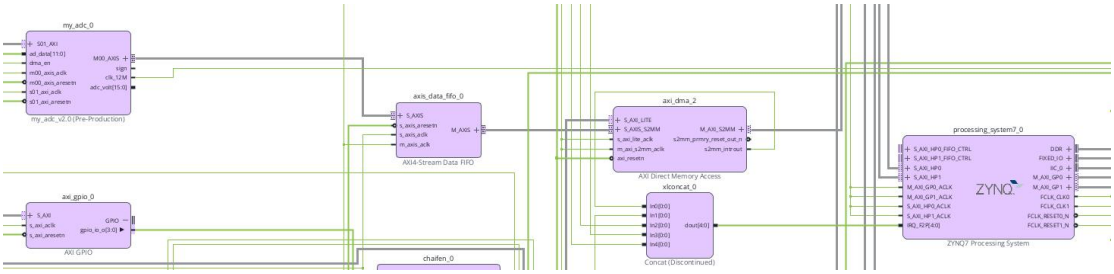
```
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        rom_dout_r  <= 14'd0;
        amp_r       <= 8'd0;
        mult_tmp    <= 24'd0;
        mult_stage1 <= 24'd0;
        mult_stage2 <= 32'd0;
        dac_data_1  <= 14'd0;
    end else begin
        rom_dout_r <= dac_data_mid;
        amp_r      <= amplitude;
        mult_tmp   <= (rom_dout_r - 8191) * amp_r;
        mult_stage1 <= mult_tmp + 1048576;
        mult_stage2 <= mult_stage1 >> 7;
      if (mult_stage2 > 32'd16383)
            dac_data_1 <= 14'd16383;
        else
            dac_data_1 <= mult_stage2[13:0];
    end
```

```
end
```

时序违例也随之解除。DDS 可以在 120MHz 下正常工作。相比于 50MHz，在输出高频波形时，波形明显更加平滑。

3.3 BRAM 资源优化

在 PL-PS 通信中，AXI4 stream data FIFO 使用频繁。FIFO 既能让数据跨时钟域稳定传输，又可以为 AXI DMA 提供数据缓冲，。最保险的方式是将 FIFO 的容量设置为 16384 以 ADC 采集数据->PS 这条路径为例，两个通道则需要两个 FIFO。如下图所示



对于 FIFO 的容量设置，我们更加希望 FIFO 的容量等于希望单次传输的最大数据数量，因为这样可以保证数据有足够空间缓冲。然而，如果按照这么设计，我们的芯片的资源完全无法支撑 ADC 单次 65536 个数据采集和单次 FFT 至少 16384 点。我们在实践中发现，我们可以大幅度调低 FIFO 的容量，完全可以让数据在 PL 和 PS 之间稳定传输。经实践验证，当 FIFO 的容量在 4096 时，足以支撑本项目所有的 PL-PS 之间的数据传输，对 BRAM 资源优化明显。

对于 BRAM 资源消耗的对比，以只令 ADC->PS 通路的两个 FIFO 优前容量为 16384 时，两者消耗的资源如下：

	优化前	优化后
BRAM 使用数量	126	116
BRAM 占比	90%	73%

3.4 AXI 传输速度优化

对于 vitis 内置的例程和很多公开资料，对于单次触发 AXI DMA 传输接收数据，其代码是这么写的：

```
        RxDone = 0;
        Error = 0;
       //刷新 Data Cache 将指定地址范围内的缓存数据强制写回内存，确保内存数据与缓存数据一致
      Xil_DCacheFlushRange((UINTPTR)RxBufferPtr, MAX_PKT_LEN);
       //开启 DMA 传输
       XAxiDma_SimpleTransfer(&AxiDma, (UINTPTR) RxBufferPtr,
                       MAX_PKT_LEN, XAXIDMA_DEVICE_TO_DMA);
       Status   =   Xil_WaitForEventSet(POLL_TIMEOUT_COUNTER, NUMBER_OF_EVENTS, &Error);
          if (Status == XST_SUCCESS) {
              if (Status == XST_SUCCESS && !RxDone) {
                 xil_printf("Receive error %d\r\n", Status);
                 goto Done;
              }
           }
       Status=Xil_WaitForEventSet(POLL_TIMEOUT_COUNTER, NUMBER_OF_EVENTS, &RxDone)
          if (Status != XST_SUCCESS) {
             xil_printf("Receive failed %d\r\n", Status);
```

```
            goto Done;
        }
```

我们在实践中发现，当大规模多次触发传输时，其传输速度远低于理论值。经检查，本代码中 RxDone 和 Error 均在 AXI DMA 的接收完成中断服务函数中被改变。且只会有变量被置为 1，用于指示传输的成功与否。因此，假设传输早已成功，对于下面这句话：

```
  Status=Xil_WaitForEventSet(POLL_TIMEOUT_COUNTER,
NUMBER_OF_EVENTS, &Error);
```

因为传输是成功的，Error 不会被置为 1，程序会一直等待到时间超出 POLL_TIMEOUT_COUNTER（一个宏定义，确定的数）才会执行下一句话。因此，会有大量的时间消耗在等待超时。

我们的 AXI DMA 主要用于 ADC 数据采集和硬件加速 IP 核的数据的传输与接收，只要我们每次以同样的方式发起传输，传输一定会成功，不可能会失败。因此，我们可以完全不需要判断 Error 这个变量，只需要当 RxDone 被置 1 时马上执行接下来的操作，即可节省大量的时间，修改后的代码如下：

```
        RxDone = 0;
        Error = 0;
       //刷新 Data Cache 将指定地址范围内的缓存数据强制写回内存，
确保内存数据与缓存数据一致
      Xil_DCacheFlushRange((UINTPTR)RxBufferPtr,
MAX_PKT_LEN);
       //开启 DMA 传输
       XAxiDma_SimpleTransfer(&AxiDma, (UINTPTR) RxBufferPtr,
                        MAX_PKT_LEN, XAXIDMA_DEVICE_TO_DMA);
```

```
        Status   =   Xil_WaitForEventSet(POLL_TIMEOUT_COUNTER,
NUMBER_OF_EVENTS, &RxDone)
          if (Status != XST_SUCCESS) {
             xil_printf("Receive failed %d\r\n", Status);
             goto Done;
          }
```

当 POLL_TIMEOUT_COUNTER 设定为 100000U 时，我们在频率特性测试仪进行 2000 次 16384 点的 AXI DMA 数据传输时，传输时间差距如下：

	优化前	优化后
传输时间	240s 左右	20s 左右

3.5 Chirp-Z 卷积运算精度优化

我们为了确保 FFT/IFFT 的准确性，我们在 FFT ip 核配置的时候，将其设置为不缩放。由于输入 FFT ip 核的数据必须是 16 位整型实部数据+16 位整型虚部数据。在加速 chirp-Z 卷积运算时，和 ADC 采样数据不同，原始的输入数据带有实部数据和虚部数据两项且均有小数点。前期我们直接将数据四舍五入取整输入 FFT ip 核进行计算，发现精度差异较大，不能满足我们的需求。

于是，我们发现，在 ADC 原始数为 0-4095 的输入范围内，以加速卷积运算的 IFFT 为例，实部和虚部的数值。远小于 FFT IP 和允许输入的范围的 0-65535，我们完全可以根据 FFT 计算的线性性，将原始数据取一个合适的倍数进行放大后，再送入 FFT ip 核进行计算。计

算完成后，再将结果除以这个放大倍数。C 语言的代码实现如下：

```
/**
 * @brief  计算 IFFT 输入打包所需的定标系数 k_in / k_out，保证不越界的条件下精度最优。
 * @param  x     待打包的双精度复数数组
 * @param  len   点数
 * @param  k_in  输出：输入定标（打包时乘）
 * @param  k_out 输出：输出定标（解包时乘）
 * @return 无
 */
static void plfft_ifft_scaling(const czt_complex_t *x, int len, double L, double *k_in, double *k_out)
{
    const double kin_max = 65535.0;
    double maxabs = complex_max_abs(x, len);//寻找最大值
    double kin;

    if (maxabs <= 0.0) {
        kin = 1.0;                    /* 频谱全零，任意合法值即可 */
    } else {
        kin = 32767.0 / maxabs;
        if (kin > kin_max) {
            kin = kin_max;
        }
    }

    *k_in  = kin;
    *k_out = (1.0 / L) / kin;           /* 由于 IFFT ip 核不会除以
```

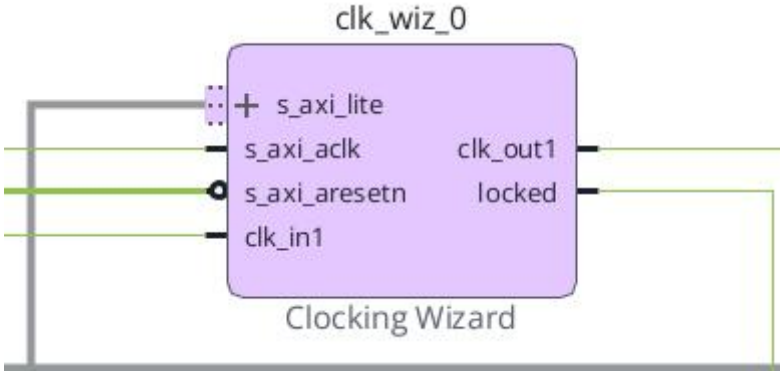
```
长度 L,故除以长度 L 与除以 k_in 可以合并进行*/
}
```

我们将一个信号的 ADC 原始值进行 IFFT 后再进行 IFFT，比较两者的 ADC 原始值的差异，有以下结论：

	优化前	优化后
ADC 原始值的差异	±30	±1

3.5 时钟输出精度优化

我们为实现采样时钟可以随时变化，我们使用带有 AXI4 Lite 的 clk_wiz ip 核，其可以在工作中频率随时被改变。



其配置代码在参考公开资料后，先是将 VCO 频率固定在 1000MHz，然后再计算分频系数，将这个分频系数，写入对应的寄存器，其代码如下：

```
void clk_wiz_cfg(u32 BaseAddr,double freq)
{

    XClk_Wiz_Config *clk_cfg_ptr;
    double div_factor = 0;
```

```
      u32 div_factor_int = 0,dviv_factor_frac=0;
      u32 clk_divide = 0;
      u32 status = 0;

       clk_cfg_ptr=XClk_Wiz_LookupConfig(BaseAddr);
       //初始化 XCLK_Wiz
   XClk_Wiz_CfgInitialize(&clk_wiz_inst,clk_cfg_ptr,clk_cfg_pt
r->BaseAddr);
       //配置时钟倍频
      XClk_Wiz_WriteReg(clk_cfg_ptr->BaseAddr,CLK_CFG0_OFFSET,
0x00000a01);  //10 倍频，1 分频
       //计算分频系数
      div_factor = CLK_WIZ_IN_FREQ * 10 / freq;
      div_factor_int = (u32)div_factor;
      dviv_factor_frac = (u32)((div_factor - div_factor_int) *
1000);
      clk_divide = div_factor_int | (dviv_factor_frac<<8);
       //配置分频系数
      XClk_Wiz_WriteReg(clk_cfg_ptr->BaseAddr,CLK_CFG2_OFFSET,
clk_divide);
       //加载重配置的参数
      XClk_Wiz_WriteReg(clk_cfg_ptr->BaseAddr,CLK_CFG23_OFFSET,
0x00000003);
      while(1){
         status                                              =
XClk_Wiz_ReadReg(clk_cfg_ptr->BaseAddr,CLK_SR_OFFSET);
         if(status&0x00000001)    //Bit0 Locked 信号
            return ;
      }
```

使用这种方式配置，会造成一个问题。就是输入的信号必须由1000MHz 分频而来，受到分频系数必须是 0.125 的倍数的限制。这个分频后未必是能达到目标频率最准的方式。这对 FFT、chirp-Z 等多个功能的计算分析精度有非常大的影响。

经过考虑,我们采用如下优化方式:直接记录我们常用频率 vivado 中的 M/D/O 配置方案方案，在 clk_wiz 的驱动程序中打好表。如下面代码所示：

```
static const clk_wiz_entry_t clk_wiz_table[] = {

      /*  freq_hz      0x200       0x208       VCO(MHz)   M / D / O */

        {   5000000u, 0x04002005u, 0x00000080u,  640.0000f },  /* 32.000 / 5 / 128.000 */

        {   6000000u, 0x04FA2605u, 0x0005F47Fu,  765.0000f },  /* 38.250 / 5 / 127.500 */

        {   6144000u, 0x05772C06u, 0x00057778u,  739.5833f },  /* 44.375 / 6 / 120.375 */

        {   8000000u, 0x04000A01u, 0x0000007Du, 1000.0000f },  /* 10     / 1 / 125.000 */

        {   8192000u, 0x05773C08u, 0x00047D5Cu,  754.6875f },  /* 60.375 / 8 /  92.125 */

        {  16000000u, 0x04000A01u, 0x0005F43Eu, 1000.0000f },  /* 10     / 1 /  62.500 */

        {  16384000u, 0x06712B05u, 0x0004FA35u,  872.5000f },  /* 43.625 / 5 /  53.250 */

        {  32000000u, 0x04000A01u, 0x0004FA1Fu, 1000.0000f },  /* 10     / 1 /  31.250 */

        {  32768000u, 0x076B1002u, 0x0006EE19u,  843.7500f },  /* 16.875 / 2 /  25.750 */

        {  48000000u, 0x05F43105u, 0x00067114u,  990.0000f },  /* 49.500 / 5 /  20.625 */

        {  49152000u, 0x06712B05u, 0x0006EE11u,  872.5000f },  /* 43.625 / 5 /  17.750 */

        {  64000000u, 0x04000A01u, 0x0006710Fu, 1000.0000f },  /* 10     / 1 /  15.625 */

        {  65536000u, 0x076B1002u, 0x00076B0Cu,  843.7500f },  /* 16.875 / 2 /  12.875 */

    };
```

当我们要使用这个频率时，直接调用已经存在表里的方，往 0x200 和 0x208 两个寄存器配置对应的值即可。

经实测，采用该方案时钟精度相比于原方案提升极大。下表显示优化前后实际的 PL 时钟差异;

	设定时钟频率	优化前实际频率	优化后实际频率
8.192MHz	8.192MHz	8.188332MHz	8.191995MHz
16.384MHz	16.384MHz	16.359918MHz	16.384977MHz
32.768MH	32.768MHz	32.786885MHz	32.766990MHz

第四部分 大模型协作记录

4.1 协作模型与工具声明

使用的大模型/Agent 平台：DeepSeek-V41

本地/云端部署方式：通过 API 调用构建的智能体。

使用场景概述：PL 端均由团队成员独立完成，未使用 AI。PS 端根据团队成员写好的有关界面布局、硬件接口、算法的相关函数，为 AI 描述写好的函数功能，以及 LCD 屏幕有关的参数，让它去调用这些函数，根据团队成员做好的屏幕设计和操控逻辑，来完成屏幕的绘制。

4.2 核心交互案例与自我纠错轨迹

案例一：示波器 Sinc 插值平滑波形的设计

初始提示词：

示波器直接使用原始采样数据,频率稍高时会出现锯齿。引入 Sinc 插值算法，让原始数据与 Sinc 函数做卷积运算，再显示波形。

模型初步回答与问题：

大模型按照这个思路写出了相关代码。但考虑到波形和数据的刷新时间，无法按照理论取所有点进行运算。AI 写出的代码中预留了地方，让我选择抽头数和插入点数。上板实测后，发现波形确实平滑了，但高频信号的波形跳动严重，甚至会出现波形紊乱的现象。同时方波阶跃处插值过多，导致阶跃处跳动严重。

自我纠错轨迹：

1.**诊断：**将现象告诉大模型，并告知若无法修复方波的问题，那么方

波就取消插值，方波不插值影响不大。

2.Prompt 调整：检查 Sinc 插值的相关算法和波形显示，按道理说在低于奈奎斯特频率的信号插值出来的波形都是平滑不怎么跳动的。还有当输入信号检测为方波时取消插值。

3.最终修正结果：AI 检查出具体的问题是在调用写好的显示驱动时，输入的数据和调用顺序出现问题，并将它改正。最终的波形非常平滑，不会出现锯齿，到了奈奎斯特频率附近才会出现明显的抖动。

案例二：修复切换屏幕界面出现的闪屏问题

初始提示词：

PL 端的 vDMA IP 核采用的是单帧缓存，为了优化或防止出现切换界面闪屏和花屏的问题，我让 vDMA 的传输的 DDR 地址分成两部分，每次改写只在不正在显示的地址处改，切换时直接切换地址，等于是在 PS 做多帧缓存。你试一下，将显示方式改为这种。

模型初步回答与问题：

大模型对相关驱动进行了替换。写好代码后进行上板验证。整体情况比之前要好不少，但在关键的显示位置还是有闪屏问题。

自我纠错轨迹：

1.诊断：将现象汇报给大模型后，它阐述这是单帧缓存必然出现的问题，在资源充裕的情况下，还是要多帧缓存。基于现在资源不充裕的情况，AI 给的思路依然是只有一份地址，每次切界面清屏，延时等待 DDR 数据改写，然后连续灌数据 ，收尾。

2.Prompt 调整：按照 AI 给出的思路进行编写，同时注意复查。

3.最终修正结果：花屏和闪屏的问题被大幅修正，绝大多数的重要切换和刷新屏幕不再出现这种问题，即使有时可能还会出现，但几乎不影响整个设备的正常使用。

案例三：曲线拟合的 HLS 算子在 PS 端的调用

初始提示词：

先查看 PL 端的曲线拟合的 HLS 算子，同时相关驱动和注释已经由团队成员写好，仔细查看驱动和注释，并调用函数完成频率特性分析仪功能界面的曲线拟合。

模型初步回答与问题：

AI 成功地在 PS 端调用了写好的函数，并在界面上显示出对应的拟合曲线，但显示出的拟合曲线存在瑕疵，准度有点不高。

自我纠错轨迹：

1.诊断：将现象汇报给 AI 后，AI 表示它严格地使用了我们团队成员写好的函数，没有自己编写驱动，理论上不会有问题。

2.Prompt 调整：注意该拟合系统的拟合阶数固定为 2 阶。重点检查调用函数的输入参数是否满足注释所写。拟合冲激响应和阶跃响应曲线时，只取前 1/4 的点，后续的点用处不大。

3.最终修正结果：检查出的问题是 AI 调用函数时，写入的参数存在一定问题，以及拟合冲激响应和阶跃响应曲线时，点数取得过多，问题已经修复。最终拟合出的系统相关曲线平滑且与理论曲线相近。

4.3 协作效果评估与反思

正面效果：

对于我们这种复杂的逻辑设计和驱动撰写已经独立完成的情况，让 AI 调用我们写好的驱动去完成对应的功能以及最重要的 LCD 屏幕的显示和界面的设计，可谓十分完美。在保证高正确率的情况下，相比人为的手写界面设计的代码，效率可是十分高效，为我们节约了大量的时间。

局限性分析：

现阶段而言，人工智能工具仍然有很大的局限性。例如相比人类而言：架构设计不优；代码中隐性 bug 极多，甚至影响装置的性能；长上下文状态出现“降智”问题；不人工干预的话，会经常出现修一个问题后出现多个问题；面对公开资料不多的领域，经常会给出完全错误的答案和解决方案。

这种在 PS 端让 AI 进行大量代码编写的情况，会让 AI 以及开发人员很难读懂代码并进行自我修改，同时还会造成代码堆积，形成“屎山”代码。修改一个小问题，也必须让 AI 进行，而且时间久了，AI 对话的上下文过长，AI 会不记得之前写的代码，让修改变得困难。

4.4 团队成员原创部分说明

团队自主完成的核心工作：

系统 PL 侧的所有内容，包括 AMD 官方 IP 核的参数设计和调用、自定义 IP 核的设计、Block Design 的设计等均由团队学生成员自主完成。PS 端根据官方 vitis 软件依据 PL 侧的设计所生成的 BSP，团

队学生成员自主完成了大部分软硬件接口的底层驱动程序的编写，包括 GPIO 初始化、DMA 传输、所有中断配置、串口初始化、可写入 Bram、自定义 IP 核参数实时修改等。项目所采用硬件加速的算法的软件代码，包括 FFT 输出数据计算和显示算法、Chrip-Z 变换算法，也由团队学生成员编写。

AI 与团队成员工作边界说明

本作品的核心不依赖 AI。PL 侧系统集成、FPGA 硬件架构、非通用的 RTL 代码和 IP 核均为本团队学生成员独立设计。

PS 端仅有人机交互界面的设计和需求由学生成员提出，AI 依据学生成员的需求以及学生成员自行设计的接口驱动代码和算法函数进行编写。它写的东西用于 LCD 界面绘制、按键逻辑、参数格式化等外围代码的初稿，以及编译错误或越界与显示异常的排查。AI 不参与算法与公式的推导，Chrip-Z、数字滤波器等功能的数学定义与计算由团队学生成员自己推导、验算并编写代码。AI 不参与算法底层的软硬件接口的驱动程序编写。

AI 生成的代码都要经过人工审查、编译无警告和报错且上板实测、人工排查问题和修复，经由学生团队确认达到性能指标要求和没有明

第五部分 技能包提炼过程

提炼来源：

在 PS 端配置相关驱动时，我们并没有过多的与 AI 进行交流，许多的驱动编写，包括 DMA 传输，多中断配置等，均由我们团队自己，在相关平台上寻找信息，自己基于官方样例代码进行实践和踩坑，最后完成。我们将踩的一些坑和自己的相关经验，整合起来，写成 skill 包。

技能包成果及具体内容：

在参考了 AMD Ross 套件中的 Skill 包的写法后，我们撰写了以下几个 Skill 包：

Skill 1：PS 端的 AXI DMA 传输配置

内容说明：提供一组经过验证的提示词模板和相关代码的模板工程，强制模型在生成代码时先查看 PL 端的相关通路，并根据用户工程的实际情况模仿模板工程撰写代码模块。

适用场景：需要大模型在 PS 端生成完整的 AXI DMA 传输模块。

使用方法：将该 Skill 包整个文件放入对应模型的路径或按照大模型要求的操作进行。

失效条件：针对 SG 模式下的 DMA 不适用。

Skill 2：PS 端的多中断配置

内容说明：提供一组经过验证的提示词模板和相关代码的模板工程，

强制模型在生成代码时先查看 PL 端开启的所有中断，并根据用户工程的实际情况和要求模仿模板工程撰写代码模块。

适用场景：需要大模型在 PS 端生成所有中断的完整配置模块以及中断服务函数的接口。

使用方法：将该 Skill 包整个文件放入对应模型的路径或按照大模型要求的操作进行。

失效条件：非 PS 端或不使用任何中断。

Skill 3：PL 端使用 AXI vDMA 使能 LCD 显示

内容说明：提供一组经过验证的提示词模板，在进行该 Skill 的主要工作前，大模型要获取用户的 LCD 屏幕的相关信息，用于确定 VTC 和 AXI Stream to Video Out 的相关参数。再根据当前用户的 Vivado 工程情况进行检查并告知用户如何搭建。大模型本身不直接对 Vivado 工程进行改写。

适用场景：需要大模型帮助用户在 PL 端检查和搭建 LCD 显示通路。

使用方法：将该 Skill 包整个文件放入对应模型的路径或按照大模型要求的操作进行。

失效条件：不适用于让大模型直接改写文件或者用户的屏幕严重不适配 AMD 官方的 IP 核。

验证有效性与复用边界：

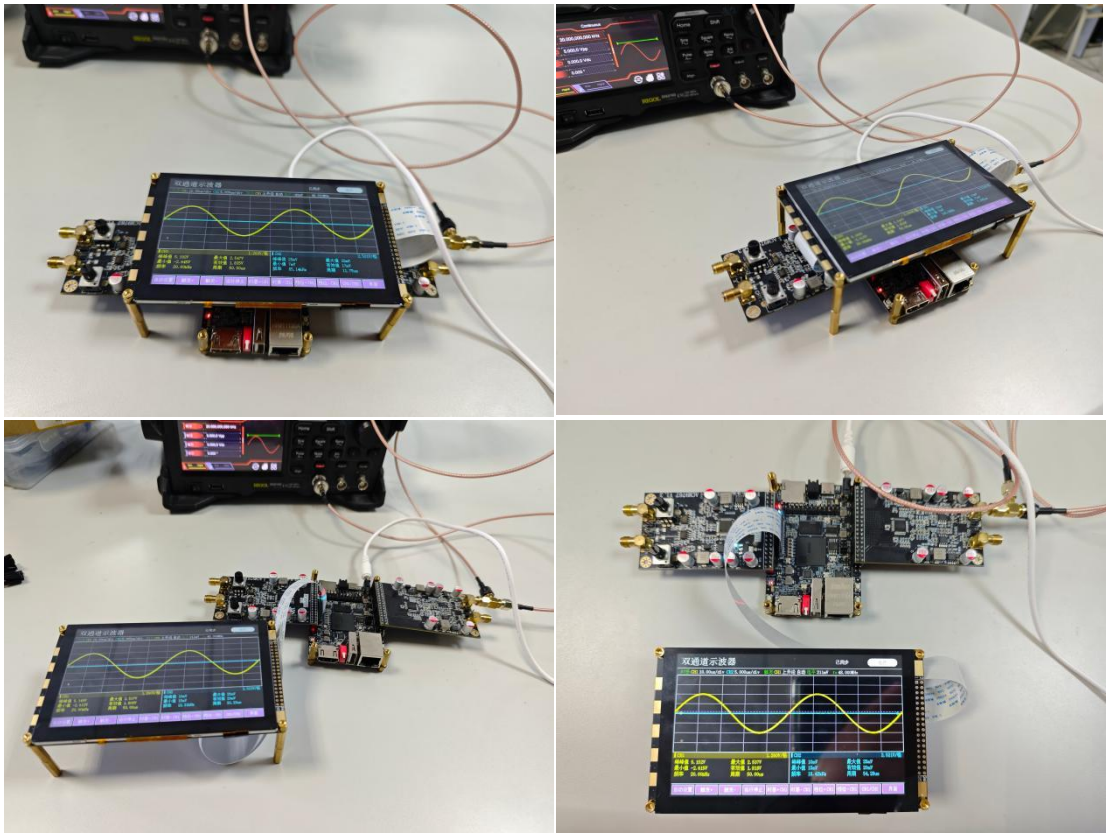
首先，我们将写好的 Skill 包装载到另一个大模型上，这里我们用的是豆包。然后我们再换一个拥有这个功能的工程，不用原始工程。让大模型读完工程后，输入相关提示词，大模型成功地调用了这个 Skill 包，且生成出的代码基本上可以直接使用，人为只要修改一点点。

在边界地划定上，一个 Skill 包只能负责一件事，不能同时做好几件事。同时这些 Skill 包不会依赖于本队特定的题目，只要使用了对应的功能和框架，vivado、vitis 框架，DMA 传输，中断配置等都可以使用。

第六部分　完成情况及性能参数

6.1 整体介绍

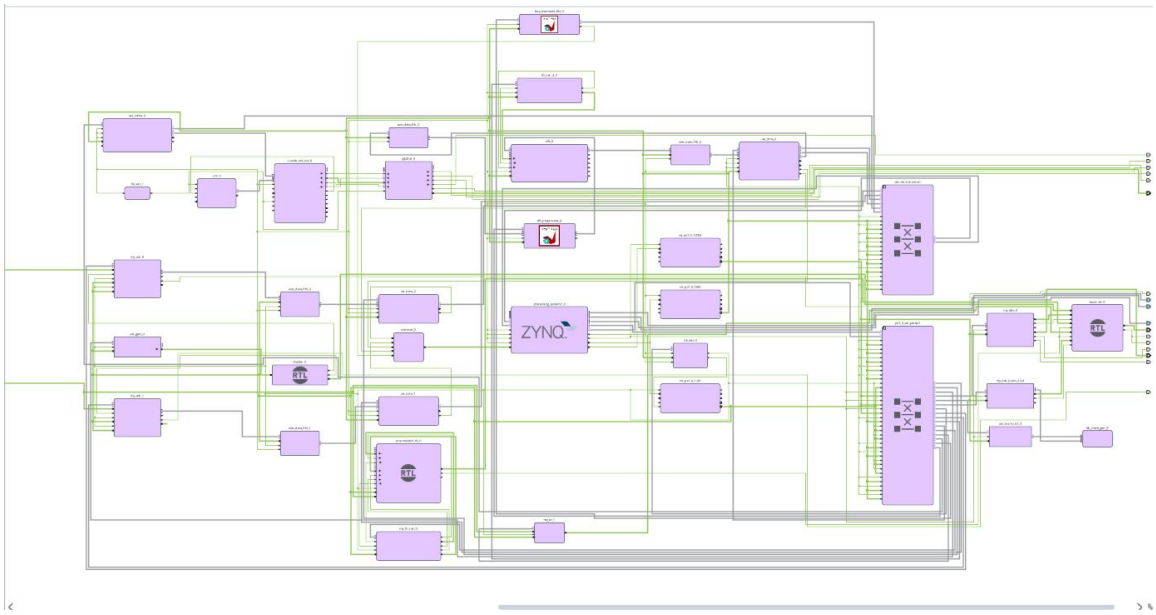
下面是装置的整体展示。整体装置由四部分构成，分别为 ZYNQ 7020 最小系统开发板、65Msps 采样率 AD9238 双通道 ADC 模块、125Msps 采样率 AD9767 双通道 DAC 模块，5 英寸 TFT LCD 液晶屏。达到了我们所追求的高集成度的需求，不需要任何额外的装置。



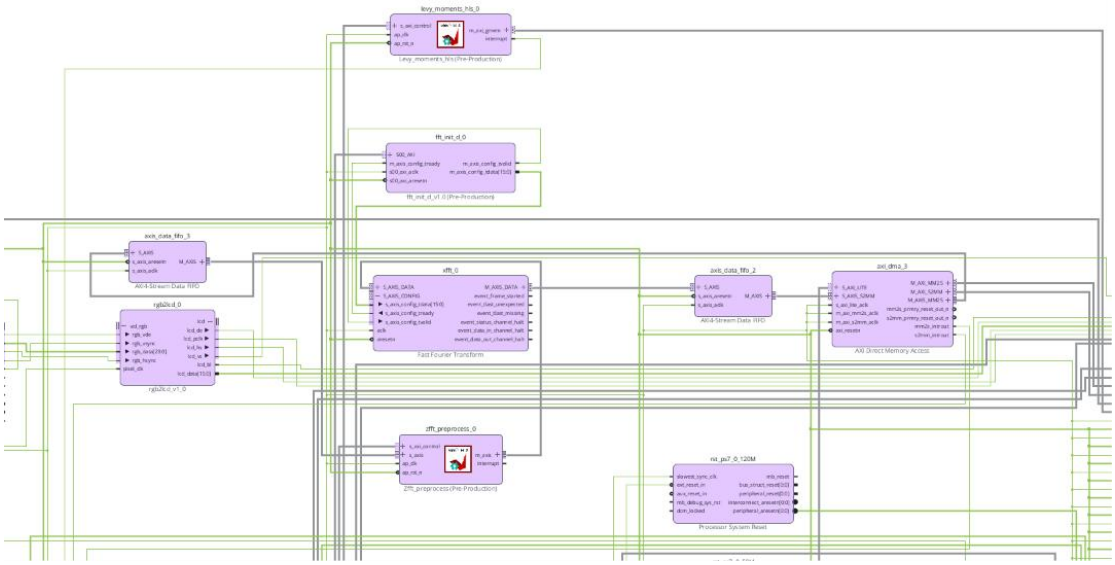
6.2 工程成果

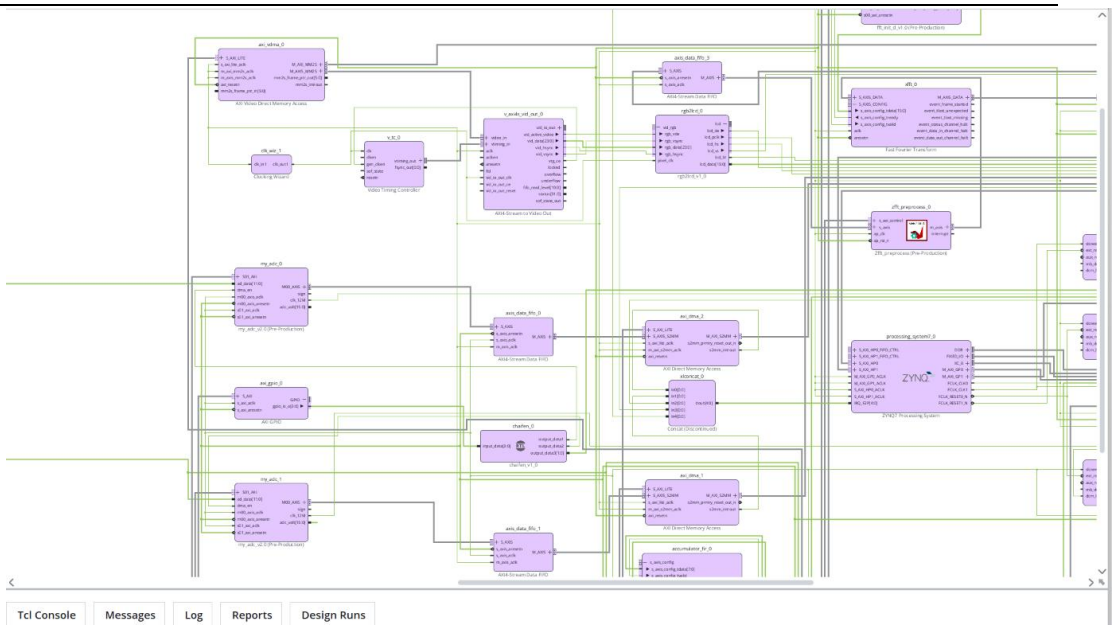
6.2.1 PL 侧 FPGA 硬件设计成果

下面是 vivado 工程的 block design 展示。基于 vivado26.1 版本开发。



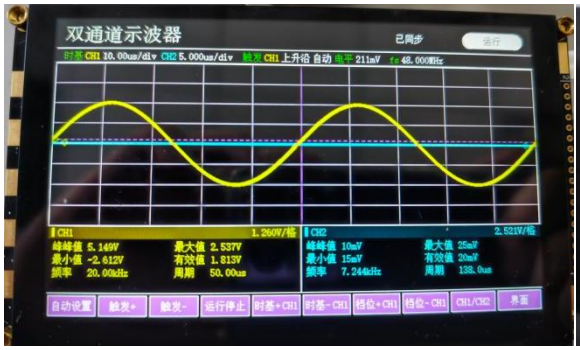
细节展示：

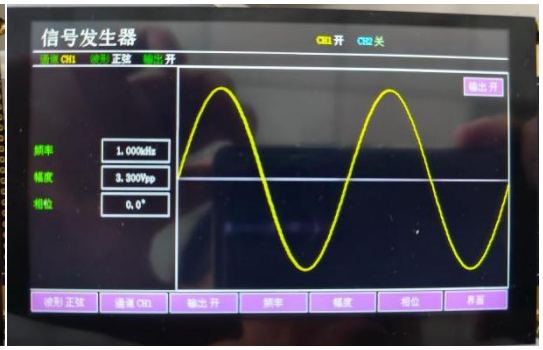


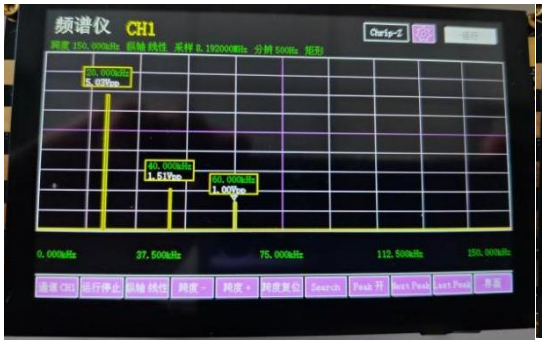


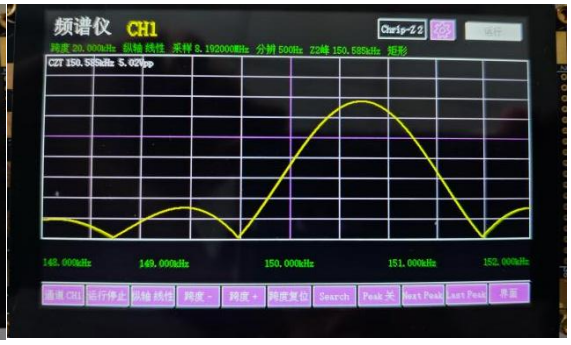
6.2.1 软件界面设计成果

下面依次为示波器，信号发生器、频谱仪(普通FFT)、频谱仪(chirp-Z）频率特性测试仪和数字滤波器功能的界面：











下面为部分参数配置界面展示：



6.3 特性成果（功能、性能参数等量化指标）

6.3.1 信号发生器

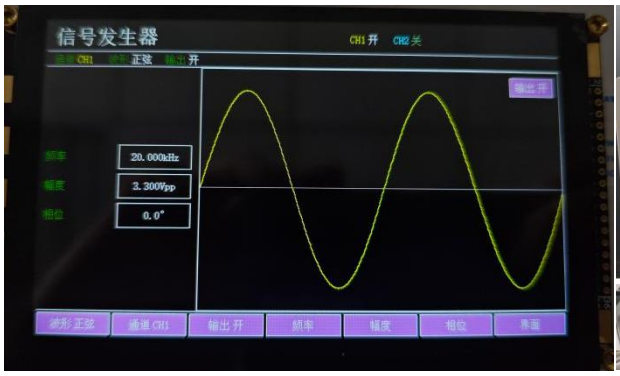
正弦波测试

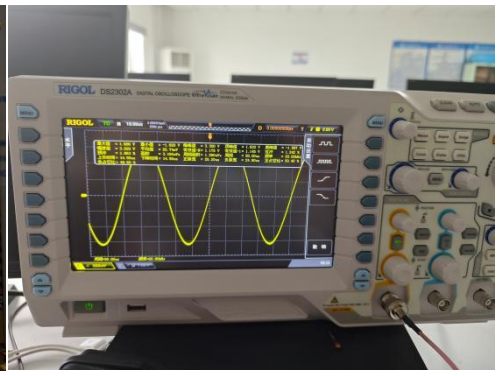
波形类型	设置频率	设置峰峰值	实际频率	实际峰峰值	是否失真
正弦波	20.000KHz	3.300Vpp	20.00KHz	3.300V	否
正弦波	1.000MHz	7.500Vpp	1.000MHz	7.440V	否
正弦波	5.000MHz	2.500VPP	5.000MHz	2.440V	否

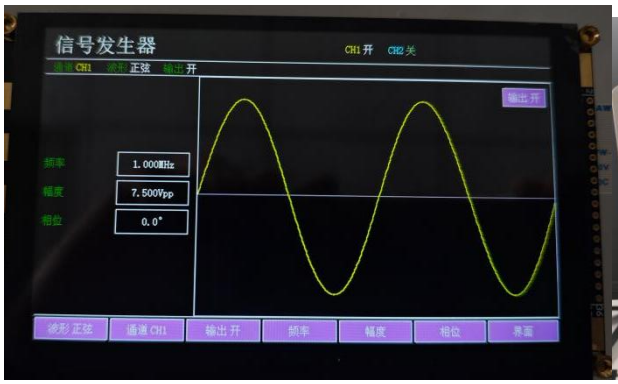
正弦波	10.000MHz	2.500Vpp	10.000MHz	1.980V	否
正弦波	20.000MHz	2.000Vpp	20.000MHz	0.776V	否

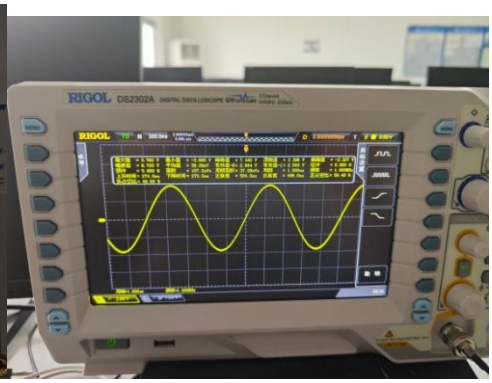
注：在频率大于 10MHz 时，受到 DAC 前端模拟低通滤波器的频率特性影响，不保证幅度的准确性。但是能保证正弦波不失真且频率稳定。

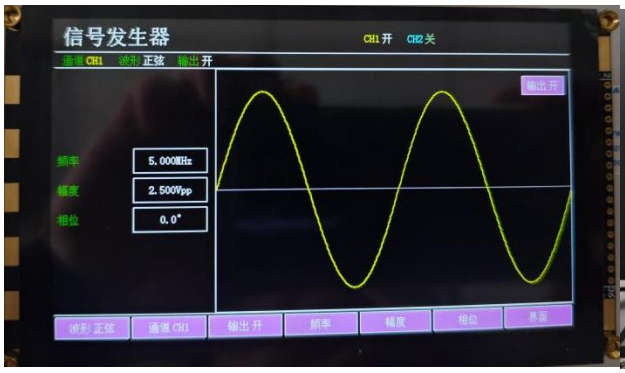
测试图片,左侧为装置设置图片，右侧为台式示波器测试装置输出：

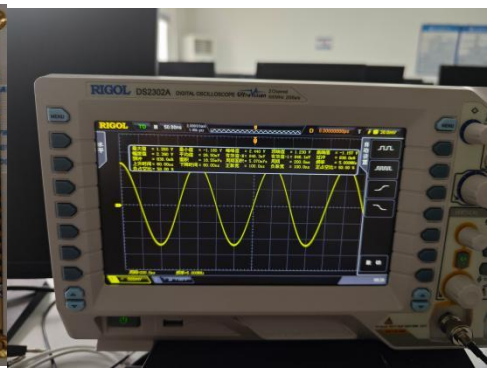


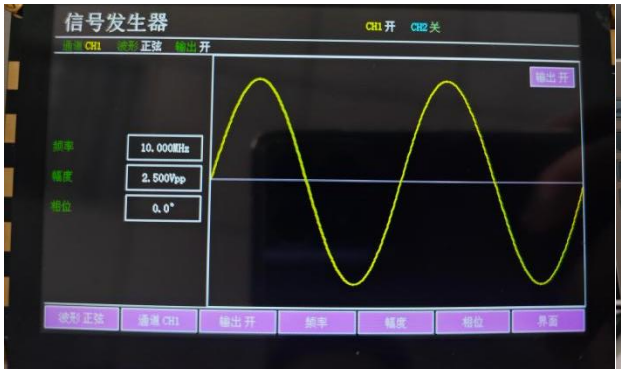


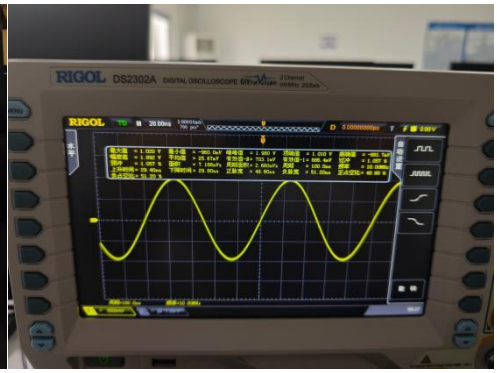


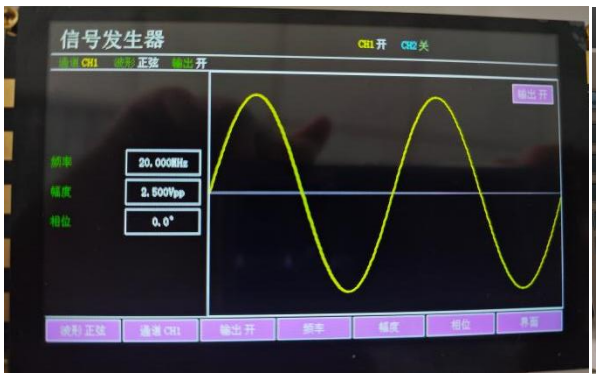


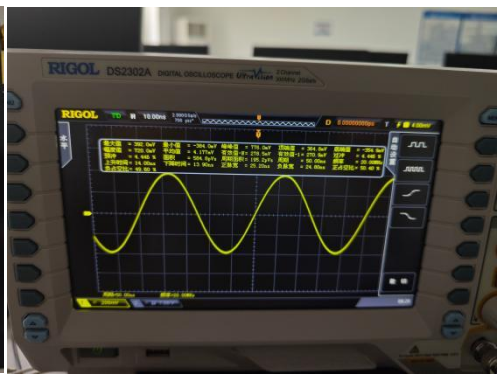






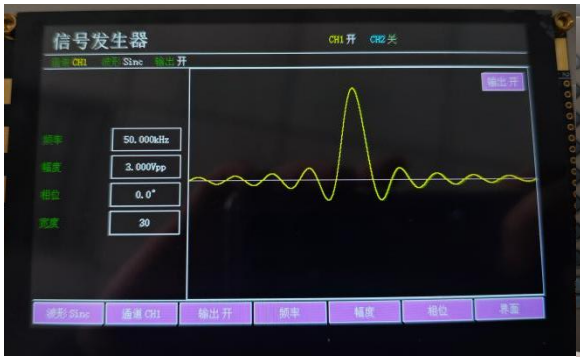






其他波形测试-Sinc

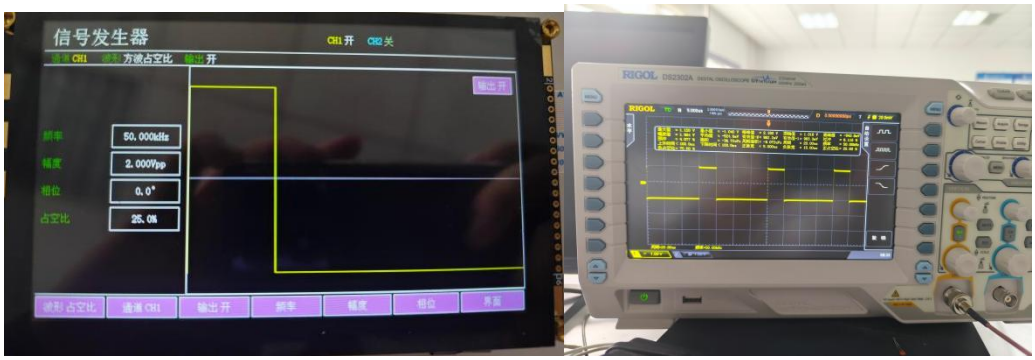
波形类型	设置频率	设置峰峰值	实际频率	实际峰峰值	是否失真
Sinc	50.00kHz	3.300V	50.00kHz	3.020V	否





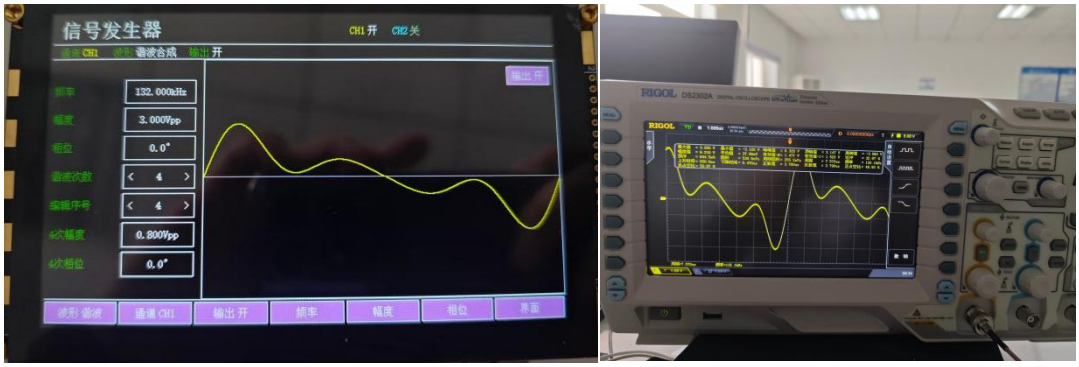
其他波形测试

波形类型	设置频率	设置峰峰值	设置占空比	实际频率	实际幅度	实际占空比
方波	50.000kHz	2.000Vpp	25.0%	50.00kHz	2.100Vpp	25.00%



其他波形测试

波形类型	设置频率	设置峰峰值	理论峰峰值	实际峰峰值	实际频率	是否失真
多次谐波	132KHz	3Vpp		6.320V	132.1kHz	否
	264KHz	1.8Vpp				
	528KHz	2.5VPP				
	660KHz	0.8Vpp				



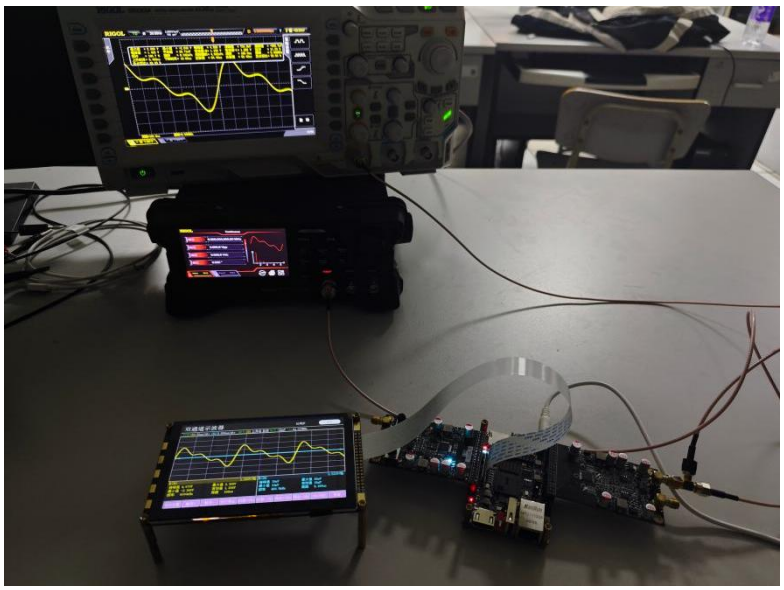
其他波形测试

波形类型	参数类型	设置频率	设置峰峰值	是否失真
AM 调制波形	调制波	20.00kHz	1.50V	否
	载波	400.00kHz	3.00V	



6.3.2 示波器

本装置的示波器功能展示如下图所示，**使用台式信号发生器输出信号，分别接台式示波器和本装置，与示波器对比输出结果**。



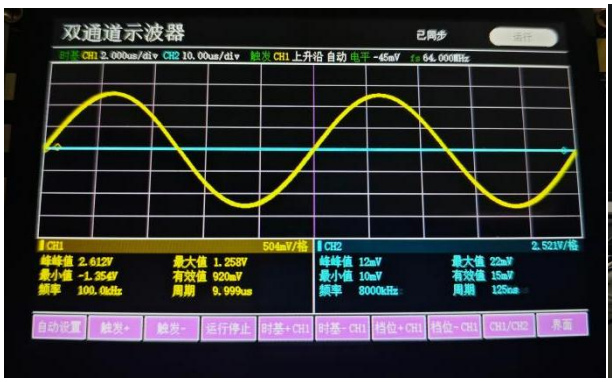
通入波形参数：

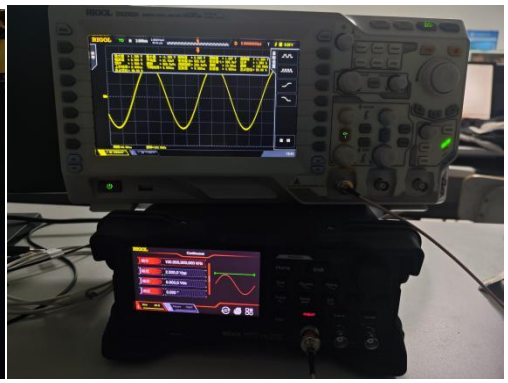
序号	波形类型	设置频率	通入通道	设置峰峰值	占空比
1	正弦波	100KHz	1	2.5V	-
2	三角波	20KHz	1	2.5V	-
3	方波	20KHz	1	2.5V	50%

4	正弦波	25MHz	1	2.5V	-
5	多次谐波	6MHz	1	3V	-
		12MHz		1.2V	-
		18MHz		1.2V	-
6	正弦波	20KHz	1	5V	-
	方波	20KHz	2	5V	50%

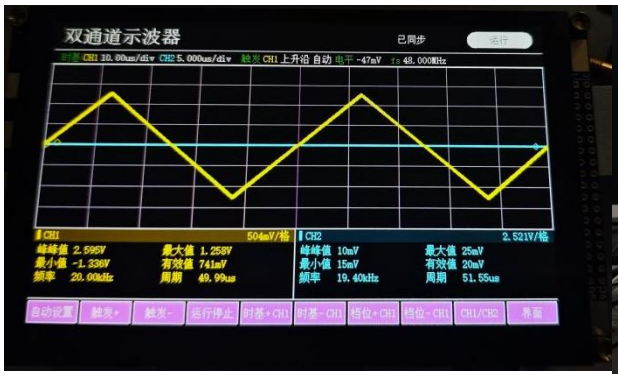
下面为显示效果，右侧为本装置显示，左侧为仪器设置：

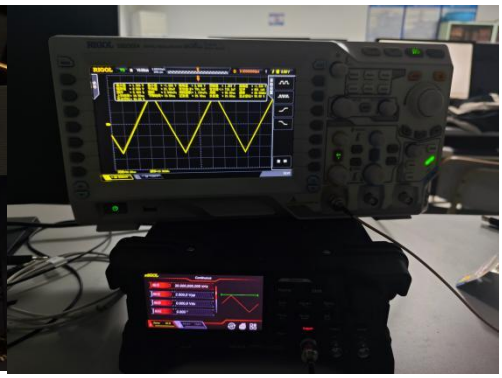
序号 1 正弦波 100KHz：



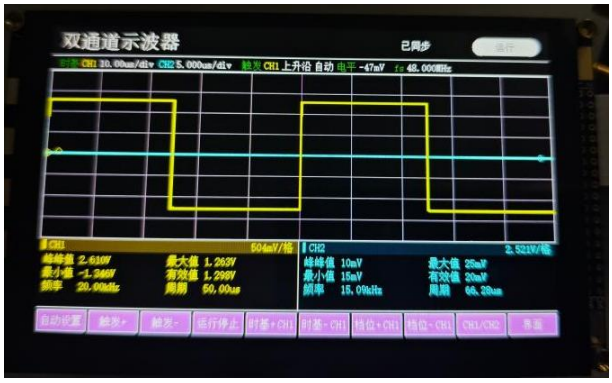


序号 2 三角波 20KHz：



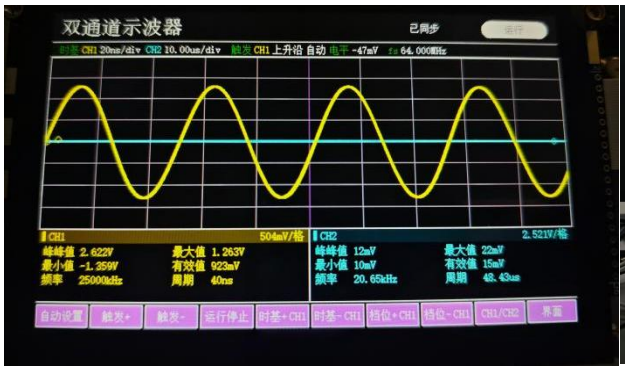


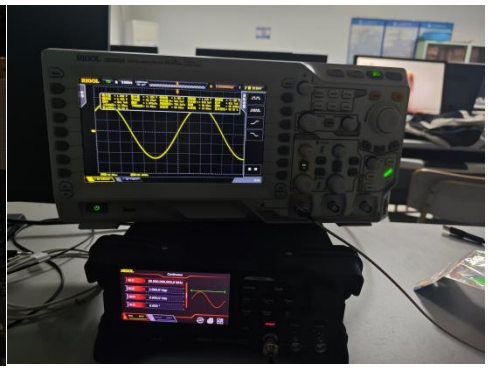
序号 3 方波 20KHz：



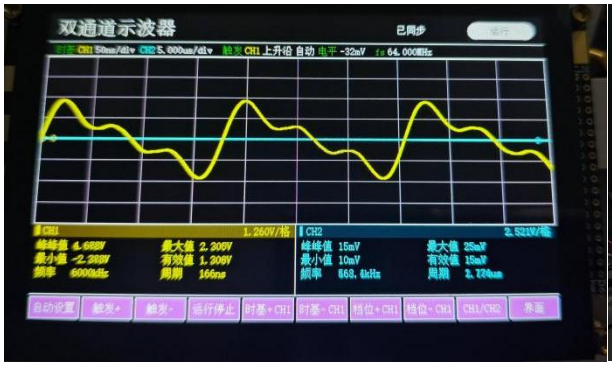


序号 4 正弦波 25MHz：



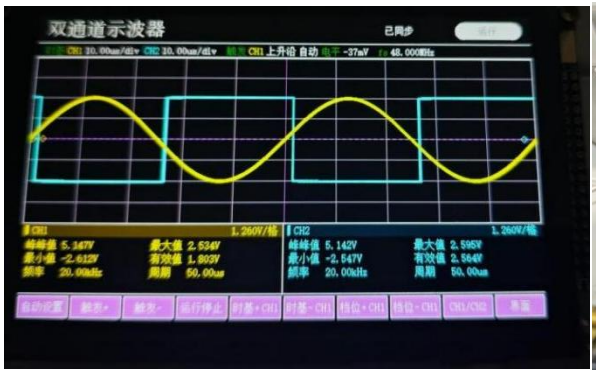


序号 5 多次谐波：





序号 6 多通道：





6.3.3 实时频谱分析仪

由于本团队没有频谱仪相关设备，**故直接使用台式信号发生器设定的值作为参照。**

6.3.3.1 普通 FFT：

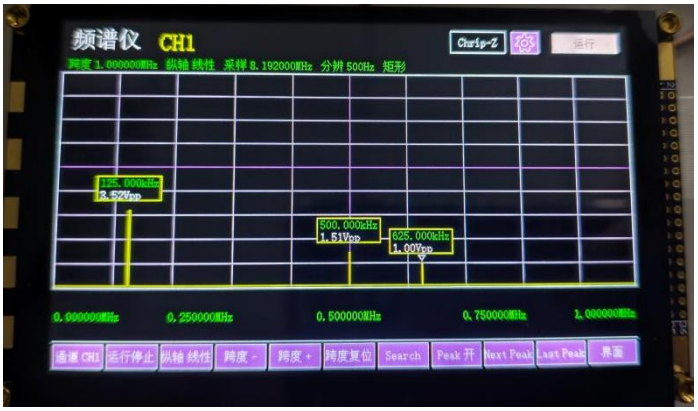
序号	波形类型	设置频率	设置峰峰值	测量频率	测量峰峰值	测量峰峰值（加窗）
1	多次谐波	125KHz	3.5V	125KHz	3.52V	-
		500KHz	1.5V	500KHz	1.51V	-
		625KHz	1.0V	625KHz	1.00V	-
2	多次谐波	2MHz	4.0V	2MHz	4.02V	4.02V
		8MHz	2.0V	8MHz	1.97V	2.02V
		10MHz	1.5V	10MHz	1.45V	1.52V

序号 1 实验图片：

台式信号发生器设置：



装置图片：

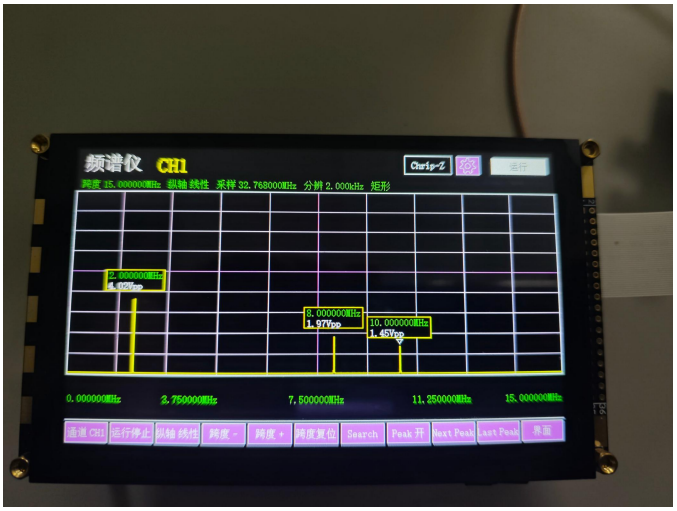


序号 2 实验图片：

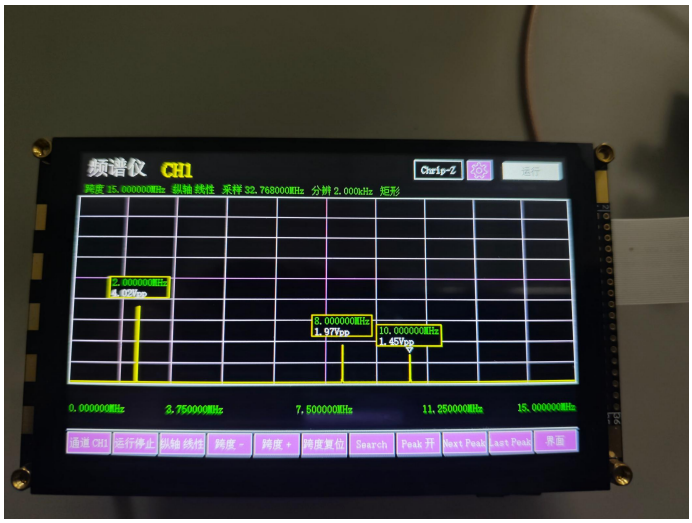
台式信号发生器设置：



未加窗：



加窗（汉宁窗）：



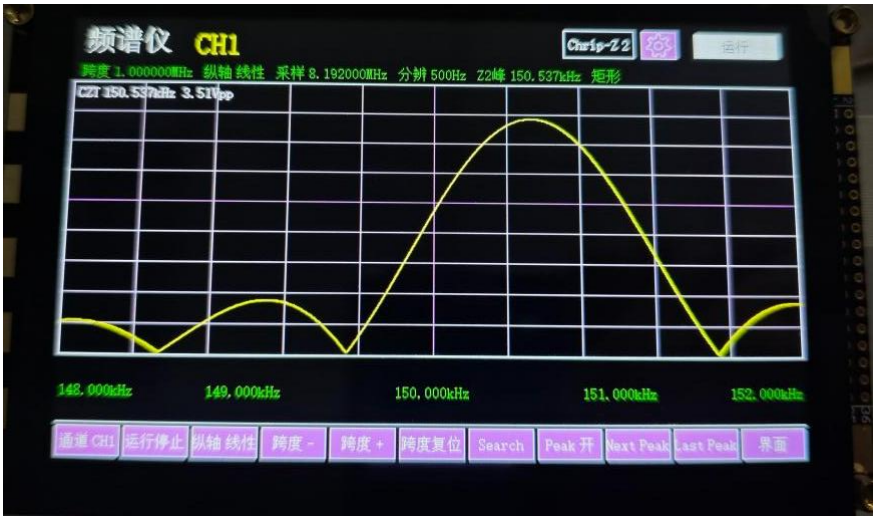
6.3.3.2 线性调频 Z 变换（Chirp-Z）部分：

序号	波形类型	设置频率	设置峰峰值	测量频率	测量峰峰值
1	多次谐波	150.535KHz	3.50V	150.537KHz	3.51V
		301.070KHz	1.50V	301.072KHz	1.50V
		451.605KHz	1.00V	451.610KHz	1.00V
2	多次谐波	2.000385MHz	4.00V	2.000387MHz	4.02V
		4.000770MHz	2.00V	4.000776MHz	2.03V
		6.001155MHz	1.50V	-	-

序号 1 仪器设置参数如下图所示：



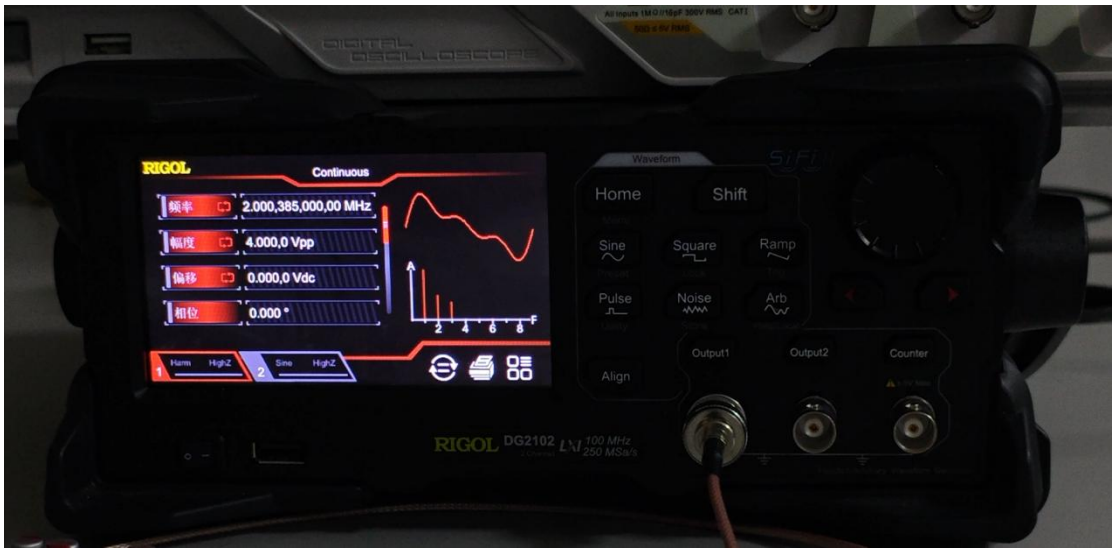
序号 1 本装置图片如下图所示：



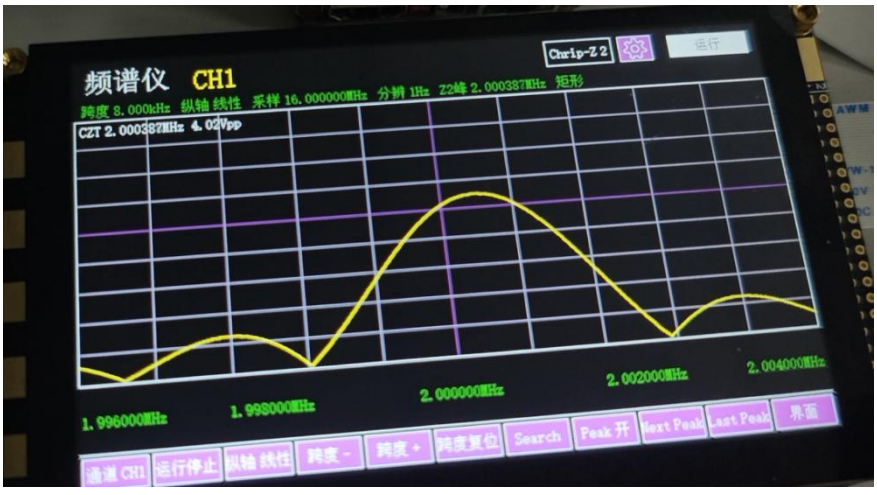


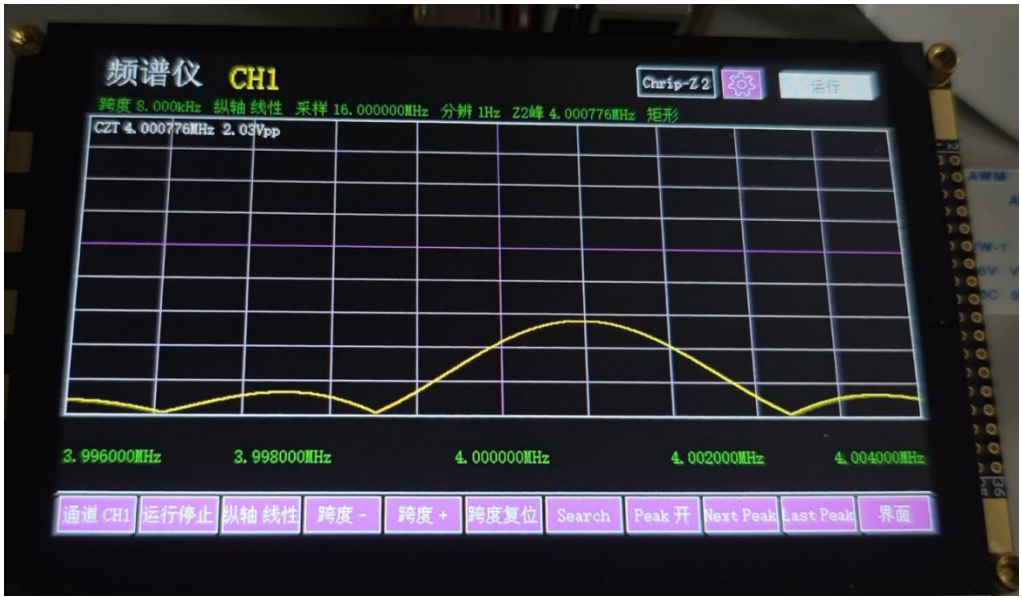


序号 2　仪器设置参数如下图所示：



序号 2 本装置图片如下图所示：



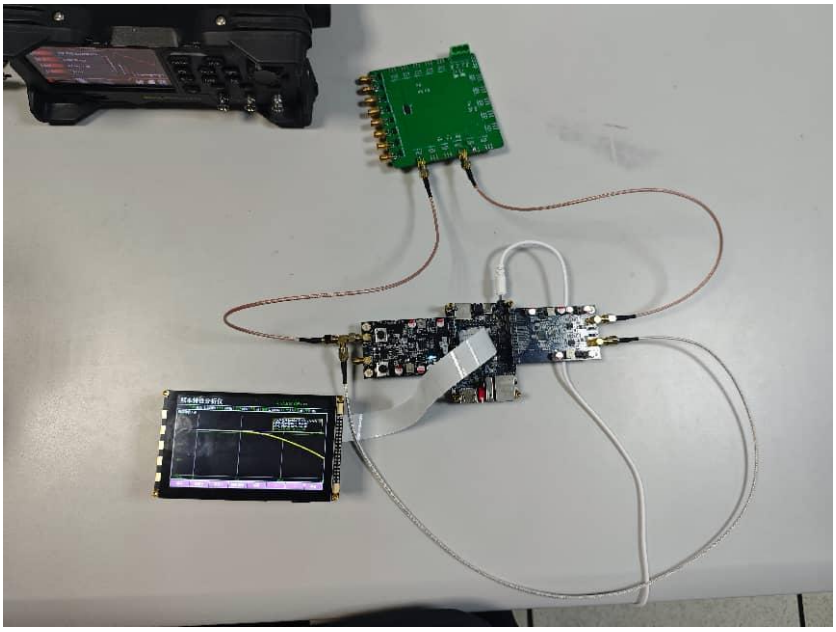


Zoom FFT 部分：

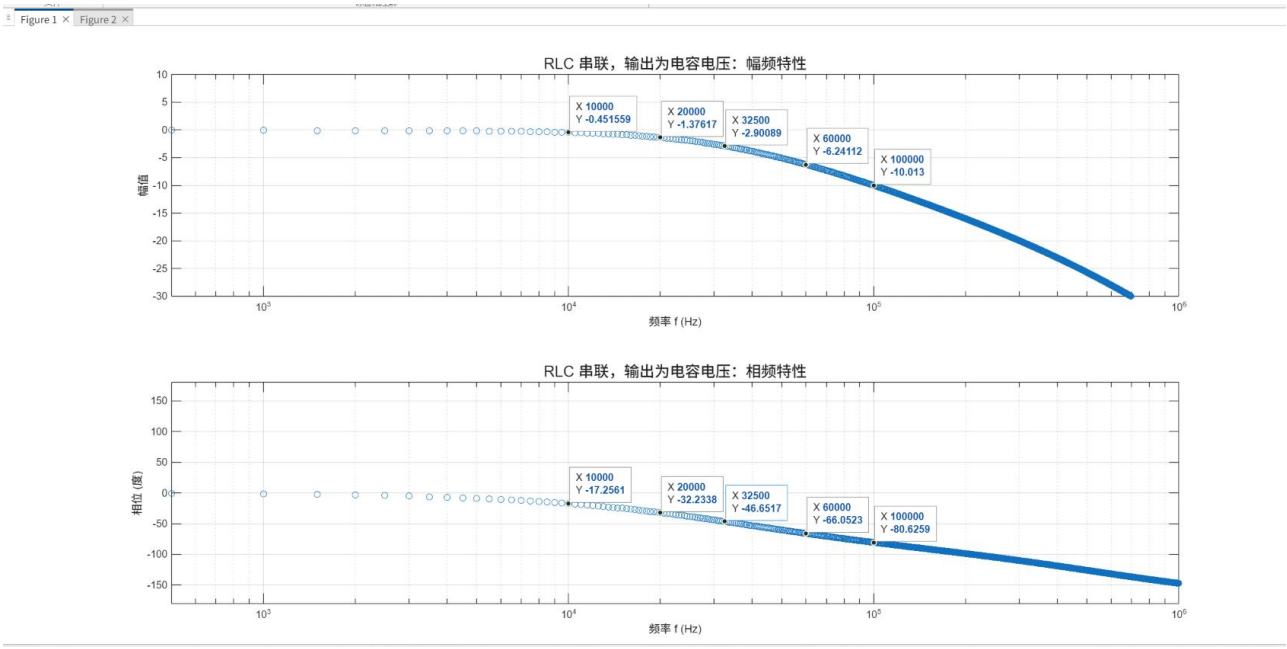
波形类型	参数类型	设置频率	设置峰峰值	测量频率	测量峰峰值
叠加波形	正弦波	1.000015MHz			
	正弦波	999.985KHz			

6.3.4 频率特性测试仪

频率特性测试仪连线方式如下图所示：



测试数据选取由 RLC 构成的二阶无源滤波器系统，测试数据 1，取 R=1kΩ，L=55uH，C=10nF ，扫频步进：500Hz，扫频范围：500Hz-1MHz，理论分频率特性的图如下图所示：

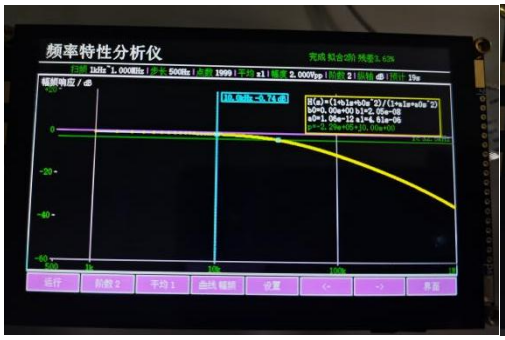


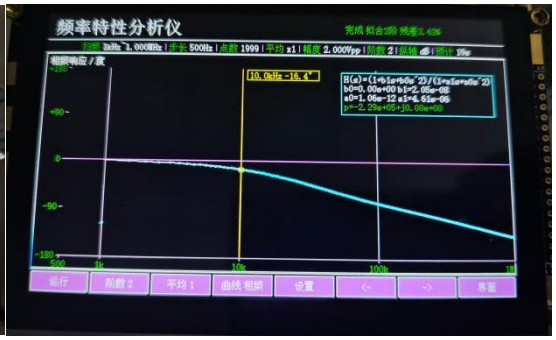
由于器件值和电路连接的影响，实际系统频率特性与理论不同，最佳方案是使用专业台式频率特性测试仪，由于本团队无该设备，故采用理论值做对比：

频率	理论增益/dB	实测增益/dB	理论相移/度	实测相移/度
10KHz	-0.45	-0.74	-17.3	-16.4
20Khz	-1.38	-1.60	-32.2	-30.6
截止频率；32.5KHz	-2.90	-3.02	-46.7	-44.4
60KHz	-6.24	-6.19	-66.1	-63.4
100KHz	-10.01	-9.84	-80.6	-77.9

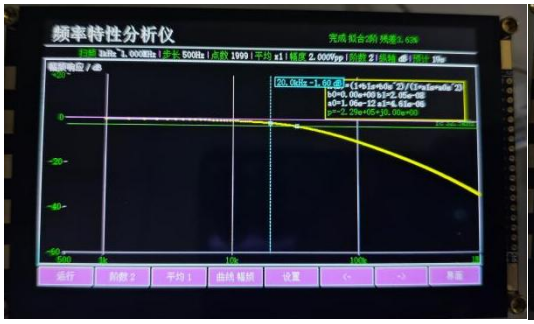
装置图片，左侧为幅频曲线，右侧为相频曲线：

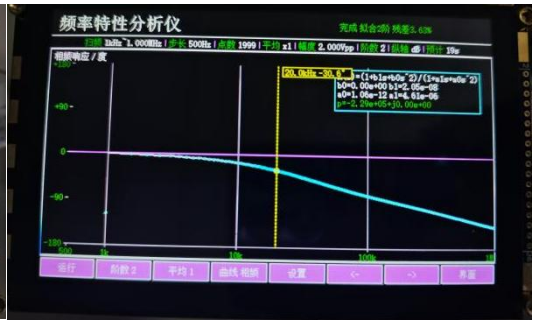
10KHz：



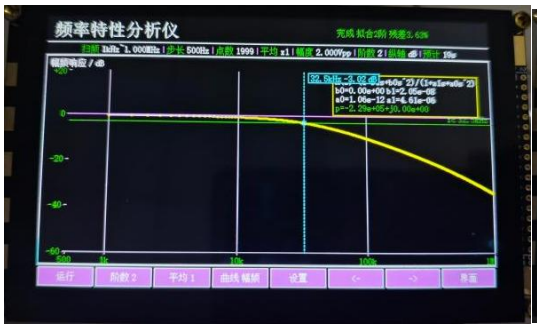


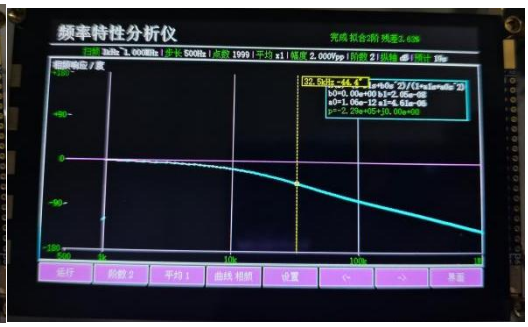
20KHz：



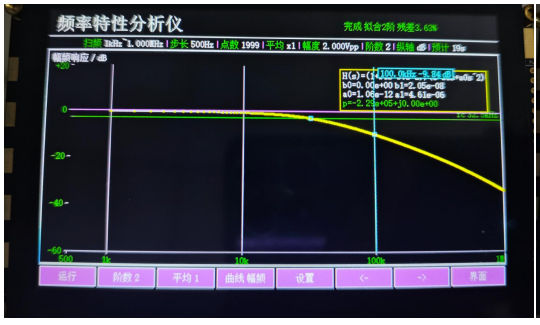


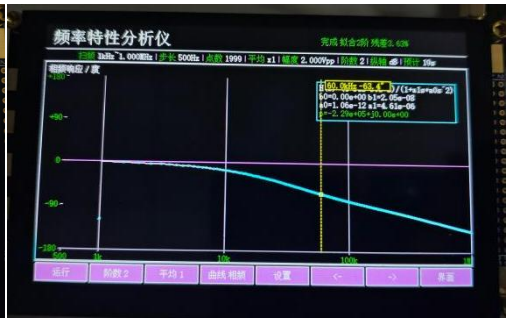
32.5KHz：



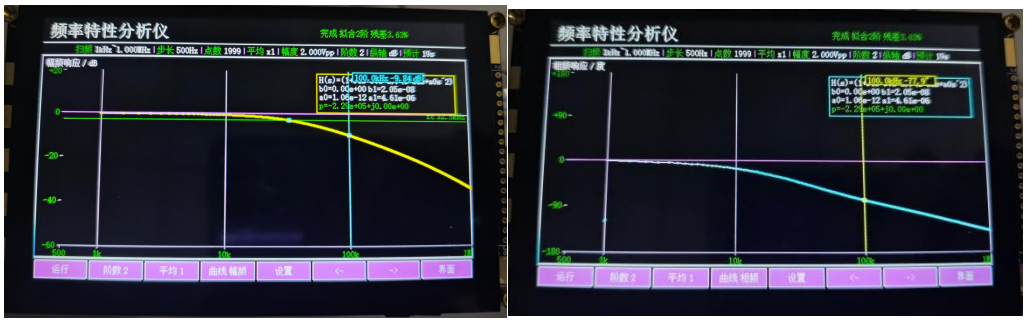


60KHz：





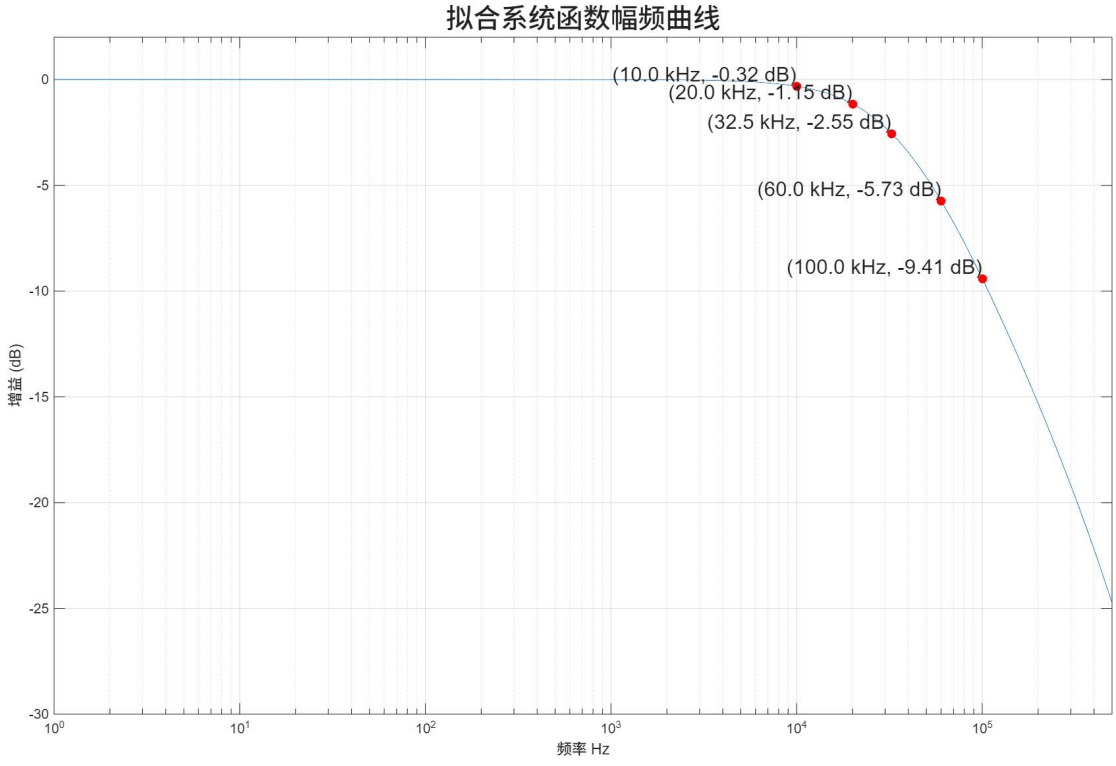
100KHz：

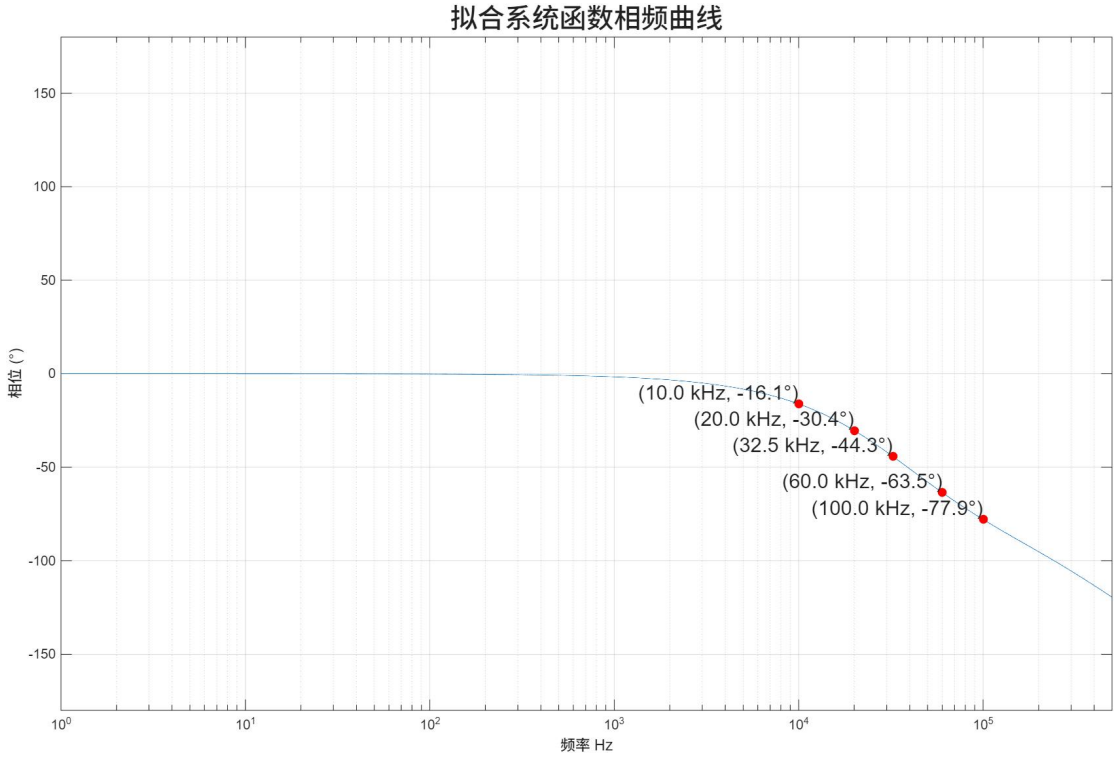


由图可知，当选择系统拟合阶数在二阶时，装置测量的系统拟合的传递函数公式为：

$$H(s)=\frac{2.05\times10^{-8}s+1}{1.06\times10^{-12}s^{2}+4.61\times10^{-6}+1}$$

该公式的幅频曲线和相频曲线及对应频率点的增益和相位值如下图所示:





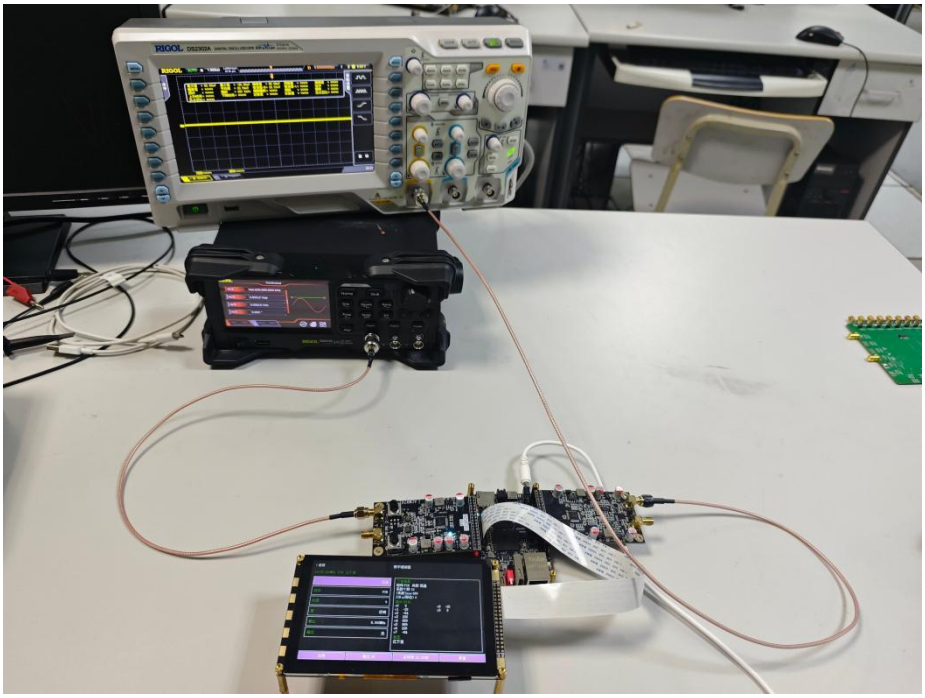
实际系统频率特性与理论不同，且受到其他因素干扰，实际系统的阶数可能不止二阶的原因，误差在可以接受的范围。

6.3.5 滤波器

设计参数：

滤波器类型	窗函数类型	设置截止频率	设置采样频率	设置阶数	量化位数
低通滤波器	汉明	8MHz	32MHz	9	有符号 12bit

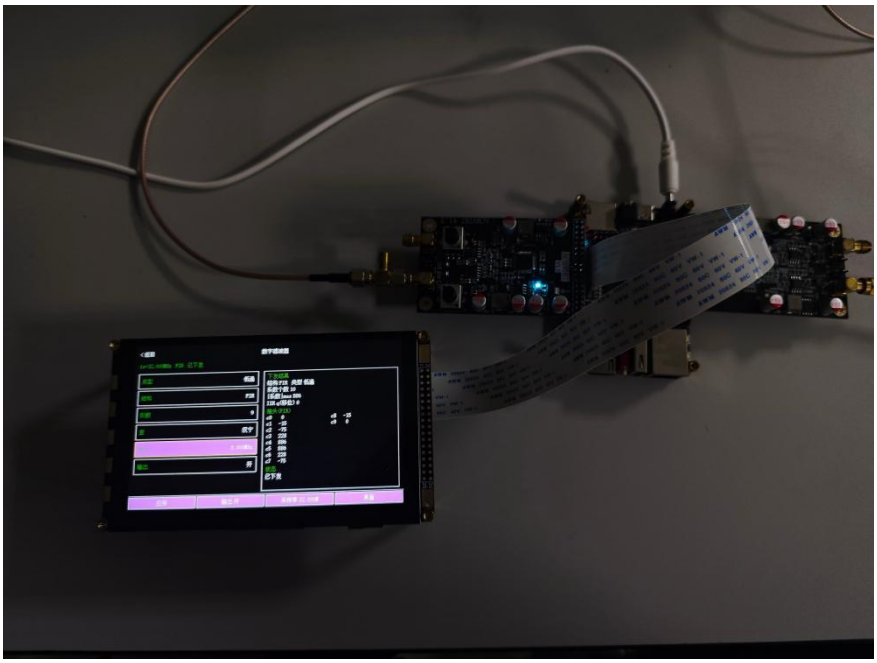
本装置连线方式，输入接 ADC 通道 2，输出接 DAC 通道 1：

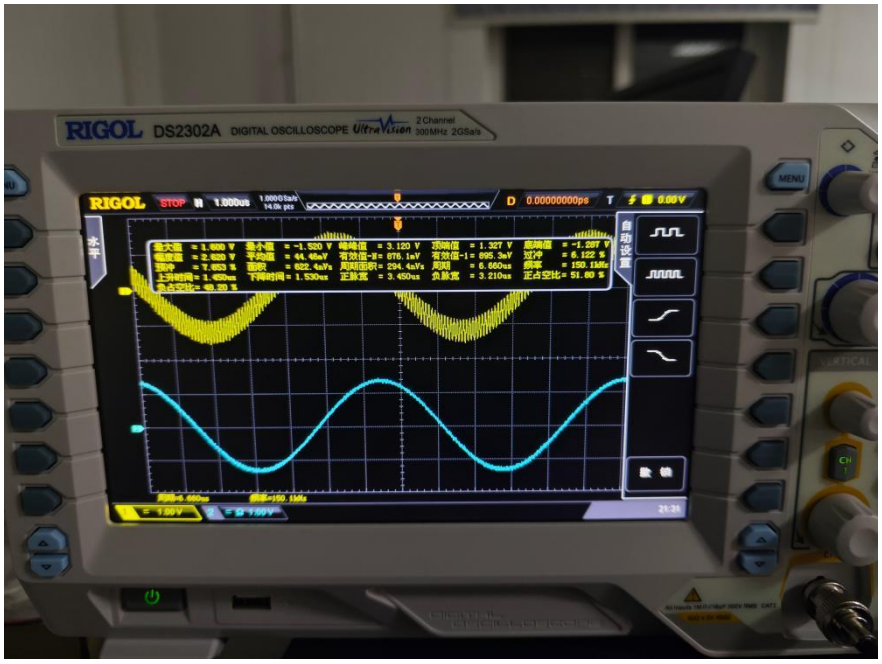


实测数据：

波形类型	参数类型	设置频率	设置峰峰值	是否滤除
叠加波形	正弦波	150KHz	3V	否
	正弦波	12MHz	0.6V	是

实测结果图片；




RIGOL DS2302A DIGITAL OSCILLOSCOPE UltraVision 2 Channel 300MHz 2GSa/s
RIGOL
MENU

第七部分 总结

7.1 可扩展性

本系统为软件定义的综合信号测试平台，若进一步提高 ADC 的采样率和 DAC 输出的速率，采用诸如 AD963x 系列的超高频 ADC/DAC。可以升级为软件定义无线电平台，通信的数字信号处理与本系统有诸多相似之处。可以实现数字上下变频、数字调制解调，基带信号处理，完成通信功能。此外，如果进一步增加 ADC、DAC 的通道数量（可以提高到几十个甚至几百个），升级更强大的 FPGA 芯片（如 RFSoC）可以实现 Massive-MIMO、波束赋形等更复杂的多天线数字通信技术以及数字相控阵雷达等感知设备。

7.2 心得体会

通过本项目，团队成员把信号与系统、数字信号处理、数字电子技术以及通信原理等诸多课堂所学理论知放在真实的 FPGA 上实现，不仅极大加深了团队成员对理论知识的掌握 而且加强了团队成员的动手实践能力。同过本项目，我们可以深切感受到一个项目，一个理论从理论成立到实际实现需要付出巨大的精力和时间。

例如，在实现 FFT 的时候，我们在数字信号课堂上学的可能只是 FFT 的流图、频谱泄露以及频率分辨率这类的理论的概念。但我们精细实现时，我们需要考虑到以下诸多问题：官方提供的 ip 核数据格式是什么样的？如何对原始数据量化，以最大利用 FFT 的 IP 核资源达到最大精度？FFT/IFFT 每一级的缩放对精度有什么影响？如何配

置才能满足我们的精度需求？如何控制时钟输出的精确度以减少频谱泄露？如何缩短 DMA 传输的时间？等等。这些问题我们只能通过官方提供的数据手册加上我们的不断的尝试才能得到最优方案，这远比我们学习 FFT/DFT 这些理论知识要花费更多的时间。

此外，相比于前几年，人工智能生成技术不断完善。不少大语言模型可以直接生成能运行的代码，人工智能工具逐渐在开发过程起到辅助作用。本项目也尝试将部分 PS 端软件的工作交给人工智能工具完成。但是，现阶段而言，人工智能工具仍然有很大的局限性。例如相比人类而言：架构设计不优；代码中隐性 bug 极多，甚至影响装置的性能；长上下文状态出现“降智”问题；不人工干预的话，会经常出现修一个问题后出现多个问题；输出面对公开资料不多的领域，经常会给出完全错误的答案和解决方案。因此，在使用人工智能工具辅

第八部分 复现说明

本工程提供了极易复现工程的脚本。并且读者可以查看、修改、优化 PS 和 PL 端盖的工程设计。

如读者需要复现本工程，需要准备一块 ZYNQ7020 的开发板，AD9238 模块、AD9767 模块以及 800x480 RGB565 TFT LCD 屏幕，触控使用 IIC 通信。同时连接的引脚要满足 XDC 引脚约束文件的定义。

这个**装有资源文件夹的路径尽量短，最好放在 D 盘或任意盘的根目录下，路径不带有中文和空格。**电脑已下载好 **Vitis 26.1 和 Vivado 26.1**，然后遵循以下步骤：

1.依次打开 Software_Defined_Signal_Test_and_Validation_Platform 文件夹,build 文件夹，点击 create_vitis_workspace 的 windows 批处理文件。如下图所示。





2.等待约 1-2 分钟，将在 build 文件夹内自动创建 vitis 的 workspace，同时自动打开 vitis 工程。打开 vitis 工程平台编译完成，可查看代码。

注意：中间可能会出现了连接 vitis 服务失败，但脚本会制动启用兼容补丁，耐心等待即可。

3.连接板卡，点击 run 即可下载程序，即可查看运行效果，读者也

可以将程序固化到 Flash 中。

注意;为了防止路径过长，脚本会自动：

1. 在工程所在盘根目录建 `D:\vws\<工程名>_<短哈希>`目录；

2. 建一个目录联接点指向工程的 build 目录；

3. 让 Vitis 通过 D:\vws\xxxx\Vitis_WorkSpace（约 35 字符，纯 ASCII）工作；

4. 如果 XSA 路径本身含空格/中文，先把它复制到 `D:\vws\xxxx_hw\`再用。

所以，如果需要删除工作区。建议使用 clean_vitis_workspace 这个 Windows 批处理文件。双击即可清理。



本工程的复现只需要 vitis，不依赖 vivado，如果读者想要查看 vivado 工程，可以点击 create_vivado_project 批处理文件。**等待 2-3 分钟，vivado 会自动打开本工程**，同时在 build 文件夹内创一个存放 prj 的文件夹。



名称	修改日期	类型	大小
create_vitis_workspace	2026/10/31 19:48	Windows 批处理...	5 KB
create_vitis_workspace	2026/10/31 21:57	PY 文件	81 KB
create_vivado_project	2026/10/31 22:04	Windows 批处理...	5 KB
create_vivado_project	2026/10/31 20:58	Altium Script Do...	15 KB
find_vivado	2026/10/31 22:03	Windows Power...	5 KB
README	2026/10/31 22:04	Markdown 源文件	15 KB

附录 参考文献

[1]吴镇扬.数字信号处理[M].高等教育出版社,2024.

[2]王金明.数字系统设计与 Verilog HDL[M].电子工业出版社,2021.